Mahavir Education Trust's
Shah & Anchor Kutchhi Engineering College
Autonomous Institute Affiliated to University of Mumbai
Bestowed with the University of Mumbai's Best College Honor A.Y. 2024-25
www.sakec.ac.in

UG Program in Electronics and Computer Science

Honours in Emerging Area offered by Department of Artificial Intelligence and Data Science

Micro-Project Title

ClinXAI: Transparent Multi-Modal Clinical Diagnostics Using LIME and SHAP

Student Details:

Name: [STUDENT NAME]

PRN: [PRN]

Class and Division: [CLASS / DIVISION]

Academic Year: 2026-27

Subject: Trustworthy and Responsible AI (Course Code: **ADCOR1PC303**)

Guide: Mr. Milind S. Khairrnar

Submitted as partial fulfillment of the term-work for the subject,
Trustworthy and Responsible AI (Course Code: **ADCOR1PC303**)

Mahavir Education Trust's
Shah & Anchor Kutchhi Engineering College
Autonomous Institute Affiliated to University of Mumbai
Bestowed with the University of Mumbai's Best College Honor A.Y. 2024-25
www.sakec.ac.in

UG Program in Electronics and Computer Science

Honours in Emerging Area offered by Department of Artificial Intelligence and Data Science

Micro-Project Title

ClinXAI: Transparent Multi-Modal Clinical Diagnostics Using LIME and SHAP

Student Details:

Name: [STUDENT NAME]

PRN: [PRN]

Class and Division: [CLASS / DIVISION]

Academic Year: 2026-27

Subject: Trustworthy and Responsible AI (Course Code: **ADCOR1PC303**)

Guide: Mr. Milind S. Khairrnar

Submitted as partial fulfillment of the term-work for the subject,
Trustworthy and Responsible AI (Course Code: **ADCOR1PC303**)

Mahavir Education Trust's
Shah & Anchor Kutchhi Engineering College
Autonomous Institute Affiliated to University of Mumbai
Bestowed with the University of Mumbai's Best College Honor A.Y. 2024-25
www.sakec.ac.in

UG Program in Electronics and Computer Science

Honours in Emerging Area offered by Department of Artificial Intelligence and Data Science

Certificate

This is to certify that the report of the project entitled

ClinXAI: Transparent Multi-Modal Clinical Diagnostics Using LIME and SHAP

is a bonafide work of

Name of student	Class	Roll No.
[STUDENT NAME]	T. Y.	[ROLL NUMBER]

submitted to the

Shah & Anchor Kutchhi Engineering College, Mumbai

during semester V in partial fulfilment of the requirement for the term-work of the subject, **"Trustworthy and Responsible AI" in Third Year, Semester V of B. Tech. Degree in**

Artificial Intelligence and Data Science

Guide

Mr. Milind S. Khairrnar

Mahavir Education Trust's
Shah & Anchor Kutchhi Engineering College
Autonomous Institute Affiliated to University of Mumbai
Bestowed with the University of Mumbai's Best College Honor A.Y. 2024-25
www.sakec.ac.in

UG Program in Electronics and Computer Science

Honours in Emerging Area offered by Department of Artificial Intelligence and Data Science

Approval for Project Report for T. Y. Semester V

This project report entitled THIRD YEAR MICRO-PROJECT, "ClinXAI: Transparent Multi-Modal Clinical Diagnostics Using LIME and SHAP" by Student Name [STUDENT NAME] is approved for semester V in partial fulfilment of the requirement for the CIAP for the subject Trustworthy and Responsible AI (Course Code: **ADCOR1PC303**).

Examiner

1._____

Date:

Place:

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Name of student	Class	Roll No.	Signature
[STUDENT NAME]	T. Y.	[ROLL NUMBER]	

Date:

Place:

COURSE ARTICULATION & OUTCOME ALIGNMENT SHEET

This sheet maps the ClinXAI micro-project to the Course Outcome alignment template supplied in the official *Technical Micro-Project Report Guidelines* for Trustworthy and Responsible AI (ADCOR1PC303), Section 4 (NBA SAR Criterion 3). The first five columns reproduce the official template text unchanged. The last two columns state whether each component applies to ClinXAI and give the evidence produced by this project.

Project Activity Component	Mapped CO	Bloom's Level	Mapped POs / PSOs	Measurable Performance Benchmark	Applicability to ClinXAI	ClinXAI Evidence
Explainability Architecture & Feature Attribution (SHAP / LIME)	CO 1	BL 2 (Understand) / BL 3 (Apply)	PO1, PO2, PO5, PSO1	Global summary & local force plots generated with zero syntax error	**Applicable — core of this project**	Global SHAP bar and beeswarm plots; local LIME, waterfall and force plots for 5 case studies; SHAP additivity error 4.4 × 10^{-16}; 24/24 automated tests passed
Fairness Audit & Debiasing Pipeline (Reweighing / Equalized Odds)	CO 2	BL 3 (Apply) / BL 4 (Analyze)	PO1, PO2, PO3, PO5, PSO1	Disparate impact improved >= 0.80; TPR gap minimized towards 0.00	Not applicable	Not implemented; the dataset has no demographic attributes (see Section 6.4)
Adversarial Defense & Evasion Fortification (FGSM / Rob. Training)	CO 3	BL 4 (Analyze) / BL 5 (Evaluate)	PO1, PO2, PO3, PO5, PSO2	Adversarial training maintains > 60% accuracy under epsilon=0.20	Not applicable	Not implemented
Privacy Vulnerability & Membership Inference Defense (MIA / DP)	CO 3	BL 4 (Analyze) / BL 5 (Evaluate)	PO1, PO2, PO4, PO6, PSO2	Shadow model attack accuracy audited; differential noise calibrated	Not applicable	Not implemented
Legal Compliance Audit & AI Model Card Generation (DPDP / EU AI)	CO 3	BL 4 (Analyze) / BL 5 (Evaluate)	PO2, PO6, PO7, PO8, PSO3	Completed technical Model Card aligned with EU AI Act Annex III	Not applicable as a deliverable	No model card or compliance audit was produced; Chapter 6 discusses principles only

Project Activity Component	Mapped CO	Bloom's Level	Mapped POs / PSOs	Measurable Performance Benchmark	Applicability to ClinXAI	ClinXAI Evidence
End-to-End Responsible AI Deployment & Interactive UI Dashboard	CO 1 - CO 3	BL 5 (Evaluate) / BL 6 (Create)	PO2, PO3, PO5, PO9, PSO3	Fully responsive live dashboard deployed on Streamlit or Gradio	**Applicable (explainability scope)**	Seven-tab Streamlit dashboard run locally (`streamlit run app.py`); 34/34 real-browser checks passed. Not deployed to a public server; layout verified at desktop width only

Only the first and last rows apply to this project. For the dashboard row, ClinXAI covers the explainability outcomes (CO 1); it does not implement the CO 2 and CO 3 activities (fairness debiasing, adversarial defence, privacy attacks or a compliance model card).

CIAP 25-MARK RUBRIC EVALUATION SHEET

The assessment rubric below is reproduced verbatim from Section 5 of the official guidelines (*Standard 25-Mark CIAP Evaluation Rubric*). Marks are awarded by the assessment committee only.

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Definition & Scope (15%)	Clearly defined problem with a focused and achievable scope.	Problem identified, but scope may be unclear or too broad/narrow.	Problem vaguely defined, or scope missing.	Problem not clearly identified or missing.
Design & Methodology (20%)	Innovative and well-structured approach with justification for chosen methods.	Appropriate design and methodology, but limited justification.	Basic design and methodology, with some justification.	Weak or missing design and methodology.
Implementation & Results (25%)	Project completed effectively with clear and well-presented results.	Project completed, but results unclear or poorly presented.	Project partially completed with limited or unclear results.	Project not completed or results missing.
Analysis & Interpretation (20%)	Thorough analysis with insightful interpretation of results.	Analysis attempts were made, but interpretation could be improved.	Basic analysis with limited interpretation.	Weak or missing analysis and interpretation.
Report & Presentation (20%)	Clear, concise, and well-organized report with effective presentation.	Report generally well-written but may lack clarity or organization. Presentation could be improved.	Report lacks clarity or organization. Presentation needs significant work.	Poorly written report with ineffective or missing presentation.

Evaluation record (to be completed by the assessment committee)

Criteria	Weight	Level awarded (1–4)	Marks awarded	Remarks
Problem Definition & Scope	15%			
Design & Methodology	20%			
Implementation & Results	25%			
Analysis & Interpretation	20%			
Report & Presentation	20%			
Total (CIAP — 25 Marks)	100%			

Evaluator's name and signature: _____

Date: _____

ABSTRACT

Machine-learning models can predict whether a breast tumour is malignant from cytology measurements with high accuracy. Ensembles such as Random Forests, however, behave as black boxes: the clinicians expected to act on a prediction cannot see why it was made. This micro-project, ClinXAI, builds and evaluates an end-to-end transparent diagnostic pipeline on the Breast Cancer Wisconsin (Diagnostic) dataset, which contains 569 patients and 30 numeric nuclear features computed from digitised fine-needle-aspirate images. Labels are re-encoded so that malignant is the positive class, and a stratified 80/20 split (random_state = 42) keeps 114 patients unseen during training. A 300-tree Random Forest is trained and compared with a standardised Logistic Regression baseline. Each prediction is explained in two complementary ways. LIME fits a proximity-weighted linear surrogate to 5,000 seeded perturbations around the patient. TreeSHAP computes exact Shapley attributions in probability units, which drive local waterfall and force plots as well as global feature importance. A seven-tab Streamlit dashboard presents predictions, explanations, model evaluation, what-if analysis and case studies. On the held-out set the Random Forest achieved 97.37% accuracy, 100% precision, 92.86% recall, an F1-score of 96.30% and a ROC-AUC of 0.9944 (TN = 72, FP = 0, FN = 3, TP = 39). SHAP additivity held with a maximum error of 4.4×10^{-16}. Across all 114 test patients, LIME and SHAP shared on average 3.66 of their top-five features (mean Jaccard similarity 0.62, mean Spearman correlation 0.70), and 28.9% of patients showed at least one directional disagreement. Agreement was complete for confident predictions but collapsed for the two missed malignancies studied in detail (Patients #86 and #73), where LIME's local fit fell to $R^2 \leq 0.06$. The study shows that explanations must themselves be audited before they are trusted in clinical decision support.

Keywords: Explainable Artificial Intelligence; LIME; SHAP; TreeSHAP; Random Forest; Breast Cancer Diagnosis; Clinical Decision Support

TABLE OF CONTENTS

Course Articulation & Outcome Alignment Sheet vi

CIAP 25-Mark Rubric Evaluation Sheet viii

Abstract ix

List of Figures xiv

List of Tables xvii

List of Abbreviations xix

Chapter 1: Problem Definition & Operational Relevance 1

1.1 Problem Statement and Background 1

1.2 Clinical and Societal Relevance 1

1.3 Project Objectives 2

1.4 Project Scope and Operational Boundaries 3

Chapter 2: Theoretical Formulation, Literature Review & Mathematical Framework 4

2.1 Explainable Artificial Intelligence 4

2.2 Random Forest Classification 4

2.3 LIME: Local Interpretable Model-Agnostic Explanations 5

2.4 LIME Perturbation and Local Surrogate Formulation 5

2.5 SHAP: Shapley Additive Explanations 6

2.6 Shapley Game-Theoretic Attribution 7

2.7 TreeSHAP Explanation Method 7

2.8 LIME versus SHAP: Mathematical and Practical Trade-offs 8

2.9 Comparative Literature Review 8

Chapter 3: System Architecture, Design Pipeline & Workflow Models 10

3.1 Overall System Architecture 10

3.2 Dataset and Feature Description 10

3.3 Data Preprocessing Pipeline 12

3.4 Random Forest Diagnostic Prediction Pipeline 12

3.5 LIME Explanation Workflow 13
3.6 SHAP Explanation Workflow 14
3.7 Streamlit User Interaction Flow 14
3.8 System Architecture Diagram and Data Flow Diagram 15
Chapter 4: Implementation Details, Algorithms & Technological Artifacts 18
4.1 Development Environment and Technology Stack 18
4.2 Dataset Loading and Target Mapping 18
4.3 Model Training Implementation 19
4.4 LIME Explainer Implementation 20
4.5 SHAP Explainer Implementation 21
4.6 LIME–SHAP Comparison Implementation 21
4.7 Streamlit Interface Implementation 22
4.8 Input Validation and Error Handling 23
4.9 Code Structure and Module Responsibilities 25
Chapter 5: Experimental Evaluation, Testing, Trade-off Analysis & Empirical Results 26
5.1 Experimental Methodology 26
5.2 Dataset Characteristics and Train/Test Split 26
5.3 Accuracy, Precision, Recall, F1 and ROC-AUC 27
5.4 Confusion Matrix and ROC Curve 27
5.5 Global SHAP Feature Importance 28
5.6 Individual Patient LIME Explanations 30
5.7 Individual Patient SHAP Explanations 32
5.8 LIME–SHAP Case Study Comparison 34
Case 1 — Clearly malignant (Patient #70) 34
Case 2 — Clearly benign (Patient #74) 35
Case 3 — Borderline prediction (Patient #100) 36
Case 4 — Ranking divergence, a missed malignancy (Patient #86) 37

Case 5 — Direction disagreement, a missed malignancy (Patient #73) 38
5.9 Explanation Divergence Analysis 39
5.10 Model Performance and Interpretability Trade-offs 42
5.11 Operational Verification Screenshots 43
Chapter 6: Societal Impact, Ethical Governance, Safety & Security Audits 52
6.1 Importance of Transparency in Clinical AI 52
6.2 Human Oversight and Clinical Decision Support 52
6.3 Patient Privacy and Responsible Data Usage 53
6.4 Model Bias, Dataset Limitations and Generalisation 53
6.5 Safety Considerations and False Negatives 53
6.6 Relevant Ethical and Regulatory Considerations 54
6.7 Alignment with UN Sustainable Development Goal 3 55
Chapter 7: Conclusion, Engineering Limitations & Future Enhancements 56
7.1 Summary of Work Completed 56
7.2 Achievement of Project Objectives 56
7.3 Technical Limitations 57
7.4 Future Enhancements 57
References 59
Appendix A: Complete Source Code Listings / Core Configuration Files 62
A.1 requirements.txt — Pinned dependencies (11 lines) 62
A.2 .streamlit/config.toml — Streamlit theme and server configuration (13 lines) 62
A.3 src/__init__.py — Package marker (3 lines) 62
A.4 src/config.py — Central configuration (41 lines) 62
A.5 src/data_loader.py — Data loading, mapping, validation and split (124 lines) 63
A.6 src/model.py — Model training, evaluation and persistence (109 lines) 65
A.7 src/explain_lime.py — LIME explanations (94 lines) 67
A.8 src/explain_shap.py — SHAP explanations (80 lines) 68

A.9 src/comparison.py — LIME–SHAP comparison and case selection (125 lines) ... 70
A.10 src/visualization.py — Figures (298 lines) ... 72
A.11 train_model.py — Training script (89 lines) ... 77
A.12 evaluate.py — Explanation evaluation script (183 lines) ... 78
A.13 app.py — Streamlit dashboard (488 lines) ... 81
A.14 tests/test_pipeline.py — Automated test suite (282 lines) ... 90
Appendix B: Raw Diagnostic Trace Logs, Audit Tables & Verification Sheets ... 95
B.1 Environment Record ... 95
B.2 Training Console Log ... 95
B.3 Evaluation Console Log ... 95
B.4 Automated Test Log ... 96
B.5 Independent Output Validation (35 checks) ... 97
B.6 Real-Browser Verification Sheet (34 checks) ... 98
B.7 SHAP Additivity Record ... 99
B.8 Model Metrics Record ... 99
B.9 Case-Study Audit Table ... 100
B.10 Descriptive Analysis Record ... 100

LIST OF FIGURES

Figure 3.1: ClinXAI architecture — tabular clinical diagnostics with two complementary explainers (generated by `evaluate.py`)....10

Figure 3.2: Dataset distribution: (a) class counts for the full set and the stratified train/test split; (b–c) class-wise histograms of two discriminative features (report figure generated from the dataset)....11

Figure 3.3: Per-patient explanation workflow: LIME lane (top), SHAP lane (bottom) and the comparison step....13

Figure 3.4: Streamlit user-interaction flow implemented in `app.py`....14

Figure 3.5: Level-1 data-flow diagram of ClinXAI (scripts and files as implemented)....15

Figure 4.1: ClinXAI dashboard window (real screenshot, Microsoft Edge, 1440 × 900): sidebar patient selector and LIME settings, banner, seven tabs and disclaimer....23

Figure 5.1: Random Forest confusion matrix on the 114-patient test set....28

Figure 5.2: ROC curves of the Random Forest (AUC 0.994) and the Logistic Regression baseline (AUC 0.996)....28

Figure 5.3: Global feature importance: mean |SHAP| on the test set (probability units)...29

Figure 5.4: SHAP summary (beeswarm) plot: impact of each feature on P(malignant) for every test patient....30

Figure 5.5: LIME explanation for Patient #70 (clearly malignant)....31

Figure 5.6: LIME explanation for Patient #86 (false negative)....31

Figure 5.7: SHAP waterfall plot for Patient #70....32

Figure 5.8: SHAP waterfall plot for Patient #86 (false negative)....33

Figure 5.9: SHAP force plot for the borderline Patient #100 (minor features grouped as 'other features')....33

Figure 5.10: LIME vs SHAP for Patient #70 (full agreement)....35

Figure 5.11: LIME vs SHAP for Patient #74 (full agreement)....35

Figure 5.12: LIME vs SHAP for the borderline Patient #100 (two direction disagreements)....36

Figure 5.13: LIME vs SHAP for Patient #86 (false negative; no shared top-5 feature, four sign conflicts) 38

Figure 5.14: LIME vs SHAP for Patient #73 (false negative; four direction disagreements) 39

Figure 5.15: LIME–SHAP agreement (top-5 Jaccard and Spearman ρ) across the five case studies 39

Figure 5.16: Explanation agreement (a) and LIME fidelity (b) against model confidence for all 114 test patients; false negatives marked (descriptive analysis of test_set_lime_vs_shap.csv) 40

Figure 5.17: Overview tab: objective, dataset description, scope note, workflow diagram and educational-use disclaimer 43

Figure 5.18: Diagnostic Prediction tab for Patient #70: output label, P(malignant) = 1.000, risk gauge and the uncalibrated-score note 44

Figure 5.19: LIME Explanation tab for Patient #70: rule chart, surrogate metrics and plain-English reading guide 44

Figure 5.20: SHAP Explanation tab: global importance and beeswarm plots 45

Figure 5.21: SHAP Explanation tab: additivity metrics, waterfall plot and contribution table for Patient #70 45

Figure 5.22: SHAP Explanation tab: static force plot for Patient #70 46

Figure 5.23: LIME vs SHAP tab for Patient #86: agreement metrics (0/5 overlap, 4 disagreements) and side-by-side chart 46

Figure 5.24: LIME vs SHAP tab for Patient #86: ranking and direction table, warning and divergence reasons 47

Figure 5.25: Model Evaluation tab: metrics, confusion matrix, ROC curves and baseline comparison 48

Figure 5.26: Case Studies tab: aggregate agreement, case overview chart and the expanded borderline case 49

Figure 5.27: What-if analysis on Patient #100: lowering worst area from 906.5 to 500 flips the output to benign (P = 0.363) 50

Figure 5.28: Error handling: a worst area of 6000 (training maximum 4254) triggers the extrapolation warning 50

LIST OF TABLES

Table 1.1: Project objectives and their measurable success criteria 2

Table 2.1: Theoretical and practical comparison of LIME and TreeSHAP as used in ClinXAI..... 8

Table 2.2: Comparative summary of explanation approaches relevant to ClinXAI..... 9

Table 3.1: The ten nuclear characteristics (each expressed as mean, error and worst) 11

Table 3.2: Selected feature statistics by class (from dataset_overview.csv) 11

Table 3.3: Contents of the serialised model bundle models/clinxai_rf_bundle.joblib 13

Table 3.4: Dashboard tabs and their purpose..... 15

Table 3.5: Component interface specification 15

Table 4.1: Technology stack (exact versions used and tested) 18

Table 4.2: Defensive-engineering guardrails and their verified behaviour 24

Table 4.3: Module responsibilities and size (lines of code, including comments and blank lines)..... 25

Table 5.1: Class composition of the full dataset and the stratified split 26

Table 5.2: Held-out test-set performance (114 patients, positive class = malignant, threshold 0.5) 27

Table 5.3: Ten most important features by mean |SHAP| (outputs/metrics/shap_global_importance.csv) 29

Table 5.4: Summary of the five case studies (same patient, same model, same class for both methods)..... 34

Table 5.5: Patient #70: LIME top-5 conditions and SHAP top-5 attributions..... 34

Table 5.6: Patient #74: LIME top-5 conditions and SHAP top-5 attributions..... 35

Table 5.7: Patient #100: LIME top-5 conditions and SHAP top-5 attributions..... 36

Table 5.8: Patient #86: LIME top-5 conditions and SHAP top-5 attributions..... 37

Table 5.9: Patient #73: LIME top-5 conditions and SHAP top-5 attributions..... 38

Table 5.10: Aggregate LIME–SHAP agreement over the 114 held-out patients 39

Table 5.11: Distribution of the number of shared top-5 features per test patient 40

Table 5.12: All false negatives in the test set and their explanation-agreement profile 40

Table 5.13: LIME seed stability for the five case studies (mean / minimum pairwise top-5 Jaccard) 41

Table 5.14: Performance and interpretability trade-off between the two models....42

Table 5.15: Verification summary of the implemented system....51

Table 6.1: Safety, robustness and security audit of the prototype 54

Table 6.2: Illustrative alignment with ethical and regulatory frameworks (not a compliance claim)....54

Table 7.1: Achievement of the project objectives defined in Section 1.3 56

Table B.1: Real-browser click-through results (Playwright + Microsoft Edge)....98

Table B.2: Contents of outputs/case_studies/case_studies.csv (numeric columns; direction-disagreement lists and observations are given in Section 5.8)....100

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
AUC	Area Under the (ROC) Curve
BL	Bloom's (Taxonomy) Level
CIAP	Continuous Internal Assessment for Practical
CM	Confusion Matrix
CO	Course Outcome
CPU	Central Processing Unit
CSV	Comma-Separated Values
CV	Cross-Validation
DFD	Data-Flow Diagram
DPDP	Digital Personal Data Protection (Act, India, 2023)
EU	European Union
FN / FP	False Negative / False Positive
FNA	Fine-Needle Aspirate
HTML	HyperText Markup Language
ICMR	Indian Council of Medical Research
IEEE	Institute of Electrical and Electronics Engineers
ISO/IEC	International Organization for Standardization / International Electrotechnical Commission
JSON	JavaScript Object Notation
LIME	Local Interpretable Model-agnostic Explanations
LR	Logistic Regression
NBA	National Board of Accreditation
PNG	Portable Network Graphics
PO / PSO	Program Outcome / Program-Specific Outcome
R^2	Coefficient of determination
RF	Random Forest
ROC	Receiver Operating Characteristic
SAR	Self-Assessment Report
SDG	(UN) Sustainable Development Goal
SHAP	SHapley Additive exPlanations
TN / TP	True Negative / True Positive
TPR	True Positive Rate
TreeSHAP	Tree-specific polynomial-time SHAP algorithm
UCI	University of California, Irvine
UI	User Interface
UN	United Nations
WHO	World Health Organization
XAI	Explainable Artificial Intelligence

CHAPTER 1: PROBLEM DEFINITION & OPERATIONAL RELEVANCE

1.1 Problem Statement and Background

Breast cancer is the most commonly diagnosed cancer in women worldwide. The World Health Organization reports about 2.3 million new diagnoses and 670,000 deaths in 2022 [1]. A common diagnostic step is the fine-needle aspirate (FNA): a thin needle draws cells from a breast mass, and a cytologist judges whether the cells look benign or malignant. In the early 1990s Street, Wolberg and Mangasarian showed that this judgement can be supported by computer vision. They digitised FNA slides, traced the outline of each cell nucleus, and summarised ten shape and texture measurements per sample [2]. The resulting Breast Cancer Wisconsin (Diagnostic) dataset [3] is the data used in this project.

Modern classifiers reach high accuracy on such data, but the most accurate ones, such as tree ensembles and neural networks, are opaque. A Random Forest with 300 trees contains thousands of decision nodes (10,976 in the model built here) [4]. No clinician can trace by hand how a particular patient's measurements led to a *malignant* or *benign* output. This is the **black-box problem** in clinical decision support. A prediction that cannot be explained cannot be checked, challenged or responsibly acted upon [5], [6].

Post-hoc explanation methods try to close this gap without giving up accuracy. Two of the most widely used are LIME [7], which approximates the model locally with a simple linear surrogate, and SHAP [8], which assigns each feature a game-theoretic share of the prediction. In practice they are often used as if they were interchangeable sources of truth, yet they can disagree with each other [9].

Problem statement. Given the 30 nuclear measurements of a patient's FNA sample, ClinXAI must (i) predict the probability that the tumour is malignant with a complex tree ensemble, (ii) explain that individual prediction with both LIME and SHAP for the same target class, (iii) summarise the model's global behaviour, and (iv) measure where and why the two explanation methods disagree, especially on clinically risky edge cases.

1.2 Clinical and Societal Relevance

The operational failure this project targets is not a wrong prediction alone. It is a wrong prediction that *looks* well-justified. Three concrete failure modes motivate the work:

- **Unexplained output.** A probability score with no reasons invites either blind trust (automation bias) or blanket rejection. Neither supports safe care.
- **A false negative hidden behind a confident explanation.** A missed malignancy delays treatment. If the attached explanation looks plausible, the error is harder to catch. Chapter 5 shows that the explanations are least consistent on exactly these patients.
- **Single-explainer reliance.** Teams usually adopt one explanation method. When two well-founded methods give different reasons for the same prediction, at least one of them misrepresents the model [9]. That disagreement is invisible unless it is measured.

Standard practice falls short in two ways. Intrinsically interpretable models such as logistic regression are transparent but constrain the model's form. Opaque models paired with a single unaudited explanation offer accuracy but only an illusion of transparency. Regulators increasingly require transparency and human oversight for high-risk AI. The EU Artificial Intelligence Act treats AI that acts as a safety component of medical devices as high-risk [10], and the WHO's guidance on AI for health lists transparency, explainability and intelligibility among its core principles [11]. Without auditable explanations, such tools are hard to justify ethically, legally or clinically.

1.3 Project Objectives

The project pursued four measurable engineering objectives:

Table 1.1: Project objectives and their measurable success criteria

No.	Objective (Bloom's action verb)	Measurable criterion
O1	**Implement and evaluate** a Random Forest diagnostic classifier with malignant as the positive class, and compare it with an interpretable Logistic Regression baseline.	Accuracy, precision, recall, F1, ROC-AUC and confusion matrix reported on 114 held-out patients; no test data used for training.
O2	**Apply** LIME and TreeSHAP to explain individual predictions of the malignant-class probability, and **construct** global SHAP importance.	Local explanations for every test patient; SHAP additivity verified to an error below 10^{-6}.
O3	**Quantify and analyse** agreement between LIME and SHAP explanations.	Top-5 overlap, Jaccard similarity, Spearman rank correlation, direction agreement and LIME seed stability for all 114 patients and five programmatically selected case studies.
O4	**Design and validate** an interactive dashboard that presents predictions and explanations transparently.	Seven-tab Streamlit application verified by automated tests and a real-browser click-through.

1.4 Project Scope and Operational Boundaries

In scope. Binary malignant/benign prediction from the 30 tabular features. Random Forest and Logistic Regression models. LIME (tabular) and SHAP (TreeExplainer) explanations of P(malignant). Quantitative LIME–SHAP comparison. Five reproducible case studies. A local Streamlit dashboard. Automated tests and report-ready evidence files.

Out of scope. The system does not accept raw microscope images, so it is not an image classifier. It makes no calibrated risk estimates and gives no treatment recommendations. It has no clinical or external validation, no fairness audit (the dataset has no demographic attributes), no adversarial or privacy attacks, and no public deployment.

Interpretation of the title. The word *multi-modal* refers to the two complementary explanation modalities, a local surrogate (LIME) and game-theoretic attribution (SHAP), presented in both visual and tabular form. The input data is a single tabular modality, although it is derived from images. Genuine image plus tabular fusion is identified as future work (Section 7.4).

Assumptions and constraints. The dataset labels are taken as ground truth. Features are assumed to be measured the same way as in the original study. All computation runs on a CPU-only Windows 11 PC with Python 3.12, with no GPU, internet service or external API. The dataset is small (569 patients), so the test set has only 114 patients, and every metric carries sampling uncertainty.

CHAPTER 2: THEORETICAL FORMULATION, LITERATURE REVIEW & MATHEMATICAL FRAMEWORK

2.1 Explainable Artificial Intelligence

Explainable AI (XAI) studies methods that make a model's behaviour understandable to humans [12], [13]. Doshi-Velez and Kim define interpretability as the ability to explain or present a model's reasoning in understandable terms to a human [14]. Lipton cautions that the word covers several different properties that should be stated explicitly [15]. Three distinctions organise the field:

- **Intrinsic vs post-hoc.** Some models are interpretable by construction, such as sparse linear models or short decision trees. Others are explained after training by a separate method, as LIME and SHAP do.
- **Local vs global.** A local explanation justifies one prediction for one patient. A global explanation describes the model's behaviour over a population.
- **Model-agnostic vs model-specific.** Model-agnostic methods only query predictions, as LIME does. Model-specific methods exploit internal structure, as TreeSHAP does with tree paths.

Explanations are judged against several desiderata. This report uses five of them precisely. **Local fidelity** is how closely an explanation reproduces the model near the instance. **Consistency** means that if a model relies more on a feature, its attribution should not decrease. **Stability** means that repeated or slightly perturbed explanations stay similar. **Comprehensibility** concerns how easily a human can read the explanation. **Computational cost** is the time needed to produce it. Rudin argues that for high-stakes decisions an interpretable model is preferable to an explained black box [5]. ClinXAI tests that trade-off directly with its Logistic Regression baseline.

2.2 Random Forest Classification

A Random Forest [4] is an ensemble of T decision trees. Each tree is grown on a bootstrap sample of the training set, and each split considers only a random subset of features. In this project T = 300, splits use the Gini criterion, max_features = "sqrt" ($\lfloor\sqrt{30}\rfloor$ = 5 candidate features per split), and trees are grown until their leaves are pure

(max_depth = None, min_samples_leaf = 1). Scikit-learn [16] averages the class-probability estimates of the trees:

$$f(x) = \hat{P}(y = 1 \mid x) = (1/T) \cdot \Sigma_{t=1..T}\ p_t(y = 1 \mid x) \qquad (2.1)$$

Here $p_t(y = 1 \mid x)$ is the fraction of malignant training samples in the leaf of tree t that x reaches. Because the leaves are almost always pure, this is close to the fraction of trees voting malignant. The predicted class is malignant when $f(x) \geq 0.5$. Each split is chosen to minimise the weighted Gini impurity of the child nodes:

$$G(\text{node}) = 1 - \Sigma_{k \in \{0,1\}}\ p_k^2 \qquad (2.2)$$

where p_k is the proportion of class k in the node. Averaging many de-correlated trees reduces variance and gives strong accuracy. The price is opacity: the trained forest has 10,976 nodes and 5,638 leaves, with a mean tree depth of 7.4 and a maximum depth of 12. The baseline, Logistic Regression, models $P(y = 1 \mid x) = \sigma(w^T\tilde{x} + b)$, where $\tilde{x}$ is the standardised feature vector and $\sigma(u) = 1/(1 + e^{-u})$. Its coefficients can be read directly.

2.3 LIME: Local Interpretable Model-Agnostic Explanations

LIME [7] explains a single prediction f(x) by learning a simple, interpretable model g that imitates f only in the neighbourhood of x. It is model-agnostic: it needs only the ability to call `predict_proba` on new inputs. For tabular data, the LIME library used here (version 0.2.0.1) [17] works in three stages. It maps each continuous feature to an interpretable representation by discretising it into quartile bins learnt from the training data. It generates perturbed samples around the patient and queries the black-box model on them. It then fits a weighted linear model whose coefficients become the explanation, for example "`worst area > 1122.50` adds +0.137 to P(malignant)".

2.4 LIME Perturbation and Local Surrogate Formulation

LIME selects the explanation g from a class G of interpretable models (here, linear models) by solving

$$\xi(x) = \text{argmin}_{g \in G}\ \ L(f, g, \pi_x) + \Omega(g) \qquad (2.3)$$

where f is the black-box model (the Random Forest's P(malignant)). L measures how unfaithful g is to f in the neighbourhood of x. π_x is a proximity kernel that defines that neighbourhood. $\Omega(g)$ penalises the complexity of g. The fidelity term is a proximity-weighted squared loss over N perturbed samples z_i with interpretable representations z'_i:

$$L(f, g, \pi_x) = \Sigma_{i=1..N}\ \pi_x(z_i) \cdot (\ f(z_i) - g(z'_i)\ )^2 \qquad (2.4)$$

$$\pi_x(z) = \exp\left(-D(x,z)^2 / (2\sigma^2)\right), \quad \sigma = 0.75 \cdot \sqrt{d} \qquad (2.5)$$

Here D is the Euclidean distance in LIME's standardised representation and d = 30 is the number of features, so $\sigma = 0.75 \cdot \sqrt{30} \approx 4.108$. The library implements the kernel as $\sqrt{(\exp(-D^2/\sigma^2))}$, which is algebraically identical to Equation (2.5). With quartile discretisation, $z'_j \in \{0, 1\}$ indicates whether perturbed sample z falls in the *same* quartile bin as the patient for feature j. The surrogate is linear in these indicators:

$$g(z') = w_0 + \sum_{j=1..d} w_j \cdot z'_j, \quad \Omega(g) = \lambda \cdot \|w\|^2 \ \text{(ridge, } \lambda = 1) \qquad (2.6)$$

Each coefficient w_j is the explanation weight for feature j's condition. A positive value pushes the surrogate toward malignant, and a negative value pushes it toward benign. LIME also reports the weighted coefficient of determination R^2 of the surrogate on its own samples. **R^2 is the only built-in fidelity signal**: a low R^2 means the linear surrogate does not reproduce the forest's behaviour near this patient. Because the N samples are random, the explanation depends on the random seed and on N. This sensitivity to perturbations is a documented weakness [18]. ClinXAI fixes the seed (42) and N (5,000) for reproducibility and measures stability separately (Section 5.9).

2.5 SHAP: Shapley Additive Explanations

SHAP [8] unifies several attribution methods under the class of **additive feature attribution** explanations. Each one expresses the explanation as a linear function of simplified binary inputs $z' \in \{0,1\}^M$, where $z'_j = 1$ means feature j is present:

$$g(z') = \phi_0 + \sum_{j=1..M} \phi_j \cdot z'_j \qquad (2.7)$$

Lundberg and Lee proved that only one choice of attributions φ_j satisfies three desirable properties together, and that choice is the Shapley value from cooperative game theory:

- **Local accuracy (additivity).** The attributions sum exactly to the prediction, $f(x) = \varphi_0 + \sum_j \varphi_j$, where the base value $\varphi_0 = E[f(X)]$.
- **Missingness.** A feature that is absent receives zero attribution.
- **Consistency.** If a model changes so that a feature's marginal contribution never decreases, its attribution never decreases.

For a classifier explained in probability units, Equation (2.7) gives the identity checked in this project:

$$P(\text{malignant} \mid x) = \phi_0 + \sum_{j=1..30} \phi_j(x) \qquad (2.8)$$

2.6 Shapley Game-Theoretic Attribution

Shapley [19] asked how to divide the payout of a cooperative game fairly among players. In SHAP, the players are the M features, a coalition $S \subseteq F$ is a subset of features whose values are known, and the payout of a coalition is the model's expected output given only those features, $v_x(S) = E[f(X) \mid X_S = x_S]$. Feature j's Shapley value averages its marginal contribution over all orderings in which features can join the coalition:

$$\phi_j(x) = \Sigma_{S \subseteq F \setminus \{j\}} \left[|S|! \cdot (M - |S| - 1)! / M! \right] \cdot \left[v_x(S \cup \{j\}) - v_x(S) \right] \qquad (2.9)$$

The Shapley value is the unique attribution satisfying the axioms of **efficiency** (attributions sum to $v_x(F) - v_x(\emptyset)$), **symmetry** (interchangeable features receive equal credit), **dummy** (a feature that never changes the payout receives zero), and **additivity** (attributions for a sum of games are the sums of the attributions). Evaluated exactly, Equation (2.9) needs 2^M coalitions per patient. For M = 30 that is $2^{30} \approx 1.07 \times 10^9$ coalitions, which is impractical without a model-specific algorithm.

2.7 TreeSHAP Explanation Method

TreeSHAP [20] computes Equation (2.9) **exactly and in polynomial time** for tree models. It pushes all coalitions through each tree simultaneously, tracking the proportion of training samples (the *cover*) that follow each branch. Its cost is $O(T \cdot L \cdot D^2)$ per explained sample, where T is the number of trees, L the maximum number of leaves and D the maximum depth. ClinXAI uses the default *tree-path-dependent* variant (`shap.TreeExplainer(model)`). It needs no background dataset, because it estimates the conditional expectation $v_x(S)$ from the training-sample cover stored in each node.

Two properties make TreeSHAP especially suitable here. First, Shapley values are linear in the model, so the forest's attribution is the average of the per-tree attributions, $\varphi_j(RF) = (1/T)\ \Sigma_t\ \varphi_j(tree_t)$. Second, a scikit-learn classification tree's raw output is a class-probability fraction. The SHAP values, the base value and `predict_proba` therefore share **probability units**, and Equation (2.8) can be verified numerically (Section 5.7). The base value is the forest's expected output over its bootstrap training samples, 0.3747. That is close to the training prevalence of malignancy, 170/455 = 0.374. The path-dependent variant reflects the *model's* learned structure. It is not a causal statement about tumour biology, and with strongly correlated features it can spread credit in ways that differ from interventional variants that use a background dataset.

2.8 LIME versus SHAP: Mathematical and Practical Trade-offs

Both methods produce a signed weight per feature for the same patient and the same target class. The weights answer different questions, so they **cannot be compared in magnitude**. A LIME weight is the effect of a binary *bin indicator* in a weighted linear surrogate over a sampled neighbourhood. A SHAP value is an exact share of the prediction relative to the base value. Table 2.1 summarises the practical differences.

Table 2.1: Theoretical and practical comparison of LIME and TreeSHAP as used in ClinXAI

Property	LIME (tabular)	TreeSHAP (path-dependent)
Model access	Model-agnostic: calls predict_proba only	Model-specific: reads tree structure
Explanation unit	Weight of a quartile-bin indicator in a ridge surrogate	Additive share of P(malignant) relative to base value φ_0
Local fidelity	Approximate; measured by surrogate R^2 (mean 0.27 in this project)	Exact: $\varphi_0 + \Sigma\varphi_j = f(x)$ (error 4.4×10^{-16})
Consistency guarantee	None	Yes (Shapley axioms)
Stability	Varies with random seed and sample count	Deterministic for a fixed model
Baseline / reference	Other bins under the training distribution	E[f(X)] over training cover
Handling of correlation	Perturbs features independently (can create unrealistic samples)	Conditional expectation along tree paths
Cost per patient (here)	5,000 model calls + ridge fit (≈ 0.2–0.3 s)	Polynomial and exact; all 114 test patients are explained once at dashboard start-up (total start-up 6–8 s)
Global view	Not native	Mean $\lvert\varphi_j\rvert$ importance, beeswarm

ClinXAI quantifies agreement with three measures, computed only for the same patient and the same class. Let A and B be the sets of top-k features ranked by |weight| for LIME and |φ| for SHAP, with k = 5:

$$J(A, B) = |A \cap B| \,/\, |A \cup B| \qquad (2.10)$$

$$\rho = 1 - 6 \cdot \Sigma_j d_j^2 \,/\, (n \cdot (n^2 - 1)) \qquad (2.11)$$

J is the Jaccard similarity: 1 means identical top-5 sets and 0 means no shared feature. ρ is Spearman's rank correlation between the |LIME| and |SHAP| rankings over all n = 30 features, where d_j is the rank difference of feature j (written here in its no-ties form). **Direction disagreement** counts features in A ∪ B whose LIME weight and SHAP value have opposite signs. These are measures of agreement *between explanation methods*. They are not clinical validation metrics.

2.9 Comparative Literature Review

Table 2.2 places the methods used in ClinXAI among related, well-established approaches to explaining tabular classifiers.

Table 2.2: Comparative summary of explanation approaches relevant to ClinXAI

Approach	Type / scope	Key strength	Computational cost	Main limitation
Logistic regression coefficients [5]	Intrinsic, global + local	Directly readable; faithful by construction	Very low	Restricted (linear) model form
Permutation importance [4]	Post-hoc, global	Simple, model-agnostic	Moderate (re-scoring)	No per-patient explanation; affected by correlation
LIME [7]	Post-hoc, local, agnostic	Works for any model; intuitive rules	Moderate (N model calls)	Sampling instability; no fidelity guarantee
Anchors [21]	Post-hoc, local rules	High-precision if-then rules	High (search)	Coverage may be small
KernelSHAP [8]	Post-hoc, local, agnostic	Shapley properties for any model	High (sampling coalitions)	Approximate; needs a background set
TreeSHAP [20]	Post-hoc, local + global, tree-specific	Exact, fast, consistent	Low (polynomial)	Trees only; path-dependent assumptions

Research on the reliability of these methods motivates ClinXAI's comparison. Alvarez-Melis and Jaakkola found that LIME and related methods can change markedly under small input perturbations [18]. Slack et al. showed that because LIME and KernelSHAP rely on perturbed, often off-distribution samples, a model can be built to hide its true behaviour from them [22]. Krishna et al. documented the *disagreement problem*: practitioners regularly see explanation methods disagree on top features and signs, and they have no principled way to resolve it [9]. In the medical domain, Holzinger et al. argue that explainability is a prerequisite for clinical acceptance [6]. Molnar's text gives an accessible overview of these methods [23]. ClinXAI contributes a small, fully reproducible case study of this disagreement on a clinical benchmark. It shows *where* disagreement occurs (Chapter 5) and links it to model uncertainty and missed malignancies.

CHAPTER 3: SYSTEM ARCHITECTURE, DESIGN PIPELINE & WORKFLOW MODELS

3.1 Overall System Architecture

ClinXAI is organised as a linear pipeline with a verification layer and an interactive presentation layer (Figure 3.1). Every block in the diagram corresponds to a file in the repository:

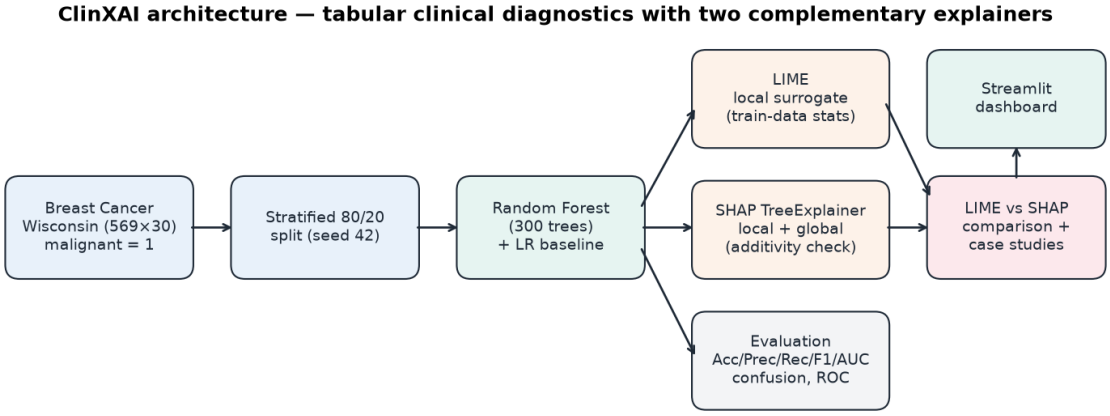


Figure 3.1: ClinXAI architecture — tabular clinical diagnostics with two complementary explainers (generated by `evaluate.py`)

- **Data layer** (`src/data_loader.py`): loads and validates the dataset, re-encodes labels (malignant = 1) and performs the stratified split.
- **Model layer** (`src/model.py`, `train_model.py`): trains the Random Forest and the Logistic Regression baseline, evaluates them and serialises a model bundle.
- **Explanation layer** (`src/explain_lime.py`, `src/explain_shap.py`): produces LIME and TreeSHAP explanations of P(malignant).
- **Comparison layer** (`src/comparison.py`, `evaluate.py`): measures LIME–SHAP agreement and selects the case studies.
- **Presentation layer** (`app.py`, `src/visualization.py`): the seven-tab Streamlit dashboard and all figures.
- **Verification layer** (`tests/test_pipeline.py`): 24 automated tests, including an application smoke test.

3.2 Dataset and Feature Description

The Breast Cancer Wisconsin (Diagnostic) dataset [3] is loaded through `sklearn.datasets.load_breast_cancer()` [16], so no manual download or credentials

are needed. It contains 569 patients: 212 malignant and 357 benign. For each FNA image, ten characteristics of the cell nuclei were computed [2], and each characteristic is summarised by three statistics: the **mean**, the **standard error** (suffix *error*) and the **worst** value (the mean of the three largest values). That gives 10 × 3 = 30 continuous, non-negative features with no missing values.

Table 3.1: The ten nuclear characteristics (each expressed as mean, error and worst)

Characteristic	Description
radius	Mean distance from the centre of the nucleus to points on its perimeter
texture	Standard deviation of grey-scale intensity values
perimeter	Length of the nuclear boundary
area	Area enclosed by the boundary
smoothness	Local variation in radius lengths
compactness	perimeter² / area − 1.0
concavity	Severity of concave portions of the contour
concave points	Number of concave portions of the contour
symmetry	Symmetry of the nucleus
fractal dimension	"Coastline approximation" − 1 (boundary complexity)

Table 3.2 lists summary statistics for six features taken from the generated `outputs/metrics/dataset_overview.csv`. Figure 3.2 shows the class balance and how the two classes separate on two representative features. Size features (area, perimeter, radius) and concavity features differ strongly between classes, although the class distributions overlap in a middle band. That band is where the model's uncertain cases lie.

Table 3.2: Selected feature statistics by class (from dataset_overview.csv)

Feature	Mean	Std. dev.	Min	Max	Mean (malignant)	Mean (benign)
worst area	880.5831	569.357	185.2	4254.0	1422.2863	558.8994
worst perimeter	107.2612	33.6025	50.41	251.2	141.3703	87.0059
worst concave points	0.1146	0.0657	0.0	0.291	0.1822	0.0744
mean concave points	0.0489	0.0388	0.0	0.2012	0.088	0.0257
worst radius	16.2692	4.8332	7.93	36.04	21.1348	13.3798
mean texture	19.2896	4.301	9.71	39.28	21.6049	17.9148

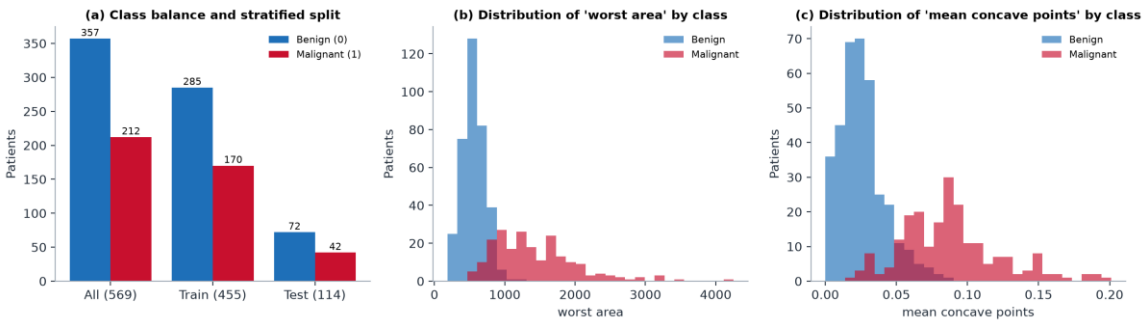


Figure 3.2: Dataset distribution: (a) class counts for the full set and the stratified train/test split; (b–c) class-wise histograms of two discriminative features (report figure generated from the dataset)

3.3 Data Preprocessing Pipeline

Preprocessing is deliberately minimal, because the data is already numeric and complete. Each step guards against a specific error:

- **Validation.** The loader checks the shape (569 × 30), the absence of NaN or infinite values, unique feature names and a binary target.
- **Label re-encoding.** sklearn encodes malignant = 0. ClinXAI derives the mapping from `target_names`, re-encodes malignant = 1 and benign = 0, and asserts the result against the raw labels. Every metric, probability and explanation therefore refers to the clinical-risk class.
- **Stratified split.** `train_test_split(test_size = 0.20, stratify = y, random_state = 42)` gives 455 training and 114 test patients with the same malignant proportion (37.4% and 36.8%).
- **No scaling for the forest.** Tree splits are invariant to monotonic feature scaling, so the Random Forest uses raw values. The Logistic Regression baseline scales inside a `Pipeline`, so the scaler is fitted on training data only.
- **Feature order.** The 30 column names and their order are stored with the model. Every prediction path (LIME's numpy callbacks, dashboard edits, dictionary inputs) rebuilds a DataFrame in that exact order.
- **Leakage control.** The model, the LIME discretisation statistics and the baseline scaler are fitted only on the training set. The 114 test patients are used only for evaluation and explanation.

3.4 Random Forest Diagnostic Prediction Pipeline

`train_model.py` trains the forest with the parameters in `src/config.py` (300 trees, Gini criterion, unlimited depth, random_state = 42, all CPU cores). It asserts that `classes_ == [0, 1]`, so column 1 of `predict_proba` is always P(malignant). It evaluates both models on the test set at a 0.5 threshold, runs 5-fold stratified cross-validation of ROC-AUC on the training set only, and writes the metrics and figures. Finally it serialises a single bundle with joblib (Table 3.3). Every later stage loads the same object, so no stage can accidentally use a different split or feature order.

Table 3.3: Contents of the serialised model bundle models/clinxai_rf_bundle.joblib

Key	Content	Used by
model	Trained RandomForestClassifier (300 trees)	evaluate.py, app.py
baseline	StandardScaler + LogisticRegression pipeline	Model Evaluation tab, ROC curve
feature_names	30 feature names in training order (checked against feature_names_in_)	All prediction paths
class_names / positive_class	["Benign", "Malignant"] / 1	Labels in UI and outputs
X_train, y_train	455 training patients	LIME statistics, range checks
X_test, y_test	114 held-out patients	Evaluation, explanations, patient selector
metrics	All test-set metrics and CV results	Model Evaluation tab
versions	scikit-learn version used for training	Reproducibility record

3.5 LIME Explanation Workflow

The upper lane of Figure 3.3 shows how one patient is explained with LIME:

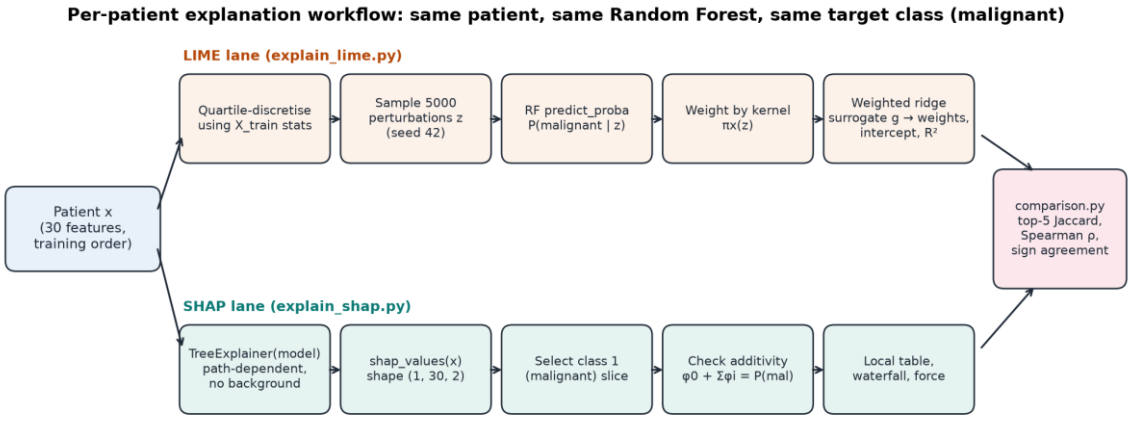


Figure 3.3: Per-patient explanation workflow: LIME lane (top), SHAP lane (bottom) and the comparison step

- Build a fresh `LimeTabularExplainer` from the training data with quartile discretisation and seed 42.
- Sample 5,000 perturbations around the patient, with bins drawn according to training frequencies.
- Query the Random Forest through a wrapper that restores feature names, and record P(malignant) for every sample.
- Weight the samples by the exponential kernel (Equation 2.5) and fit a ridge surrogate on the bin indicators (Equation 2.6).
- Return the 30 signed weights, the human-readable bin rules, the intercept, the surrogate's prediction and its R^2.

3.6 SHAP Explanation Workflow

The lower lane of Figure 3.3 shows the SHAP path. A single `TreeExplainer` is built for the trained forest. For any set of patients it returns an array of shape (n, 30, 2), one slice per class. ClinXAI selects the malignant slice and its base value, wraps them in a `shap.Explanation` object, and checks additivity against `predict_proba`. The same object drives the global bar chart and beeswarm (over the 114 test patients), the local waterfall and force plots, and the contribution table. Both lanes end in `comparison.py`, which aligns the two vectors by feature name before computing the agreement measures.

3.7 Streamlit User Interaction Flow

Figure 3.4 shows how a user interacts with the dashboard [24]. Streamlit re-runs the script on every widget change. ClinXAI therefore caches expensive objects: the model bundle, the explainer and the test-set SHAP values with `st.cache_resource`, and per-patient LIME and SHAP results with `st.cache_data` keyed by the exact feature values, seed and sample count. A change of patient or an edited biomarker refreshes every tab, while revisiting a patient is instantaneous.

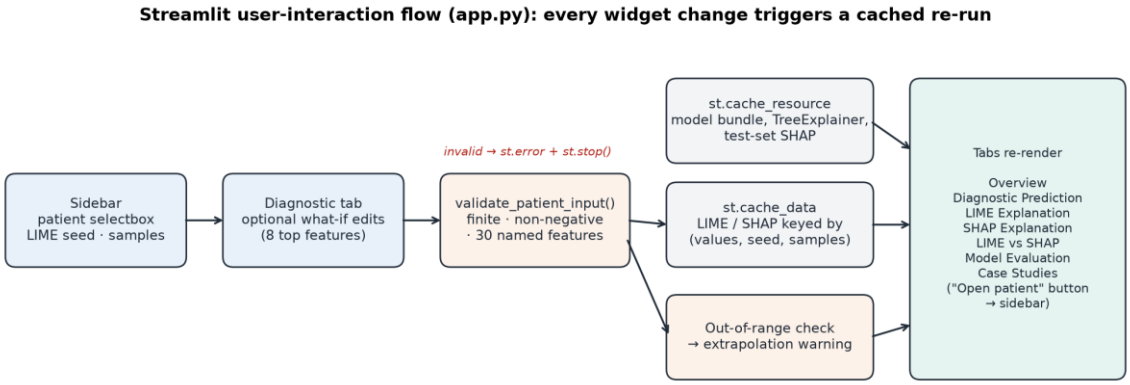


Figure 3.4: Streamlit user-interaction flow implemented in `app.py`

Table 3.4: Dashboard tabs and their purpose

Tab	Content	Purpose
Overview	Objective, dataset, scope note, workflow diagram, disclaimer	Orientation
Diagnostic Prediction	Patient header, optional what-if edits, P(malignant), risk gauge	Prediction (uncalibrated score)
LIME Explanation	Rule bar chart, surrogate metrics (R^2, intercept), top-10 table	Local surrogate
SHAP Explanation	Global bar + beeswarm; local waterfall, contribution table, force plot	Exact attribution
LIME vs SHAP	Agreement metrics, side-by-side chart, direction table, stability toggle, reasons	Explanation audit
Model Evaluation	Metrics, confusion matrix, ROC (RF vs LR), split information	Performance
Case Studies	Aggregate agreement, five cases with figures, limitations	Edge-case analysis

3.8 System Architecture Diagram and Data Flow Diagram

Figure 3.5 is a level-1 data-flow diagram of the implemented system. Processes are the four executable scripts. Data stores are the model bundle and the output folders. External entities are the dataset source and the user. The dashboard reads the model bundle and the case-study files but never writes to them, so the evidence used in this report cannot be changed by interacting with the application.

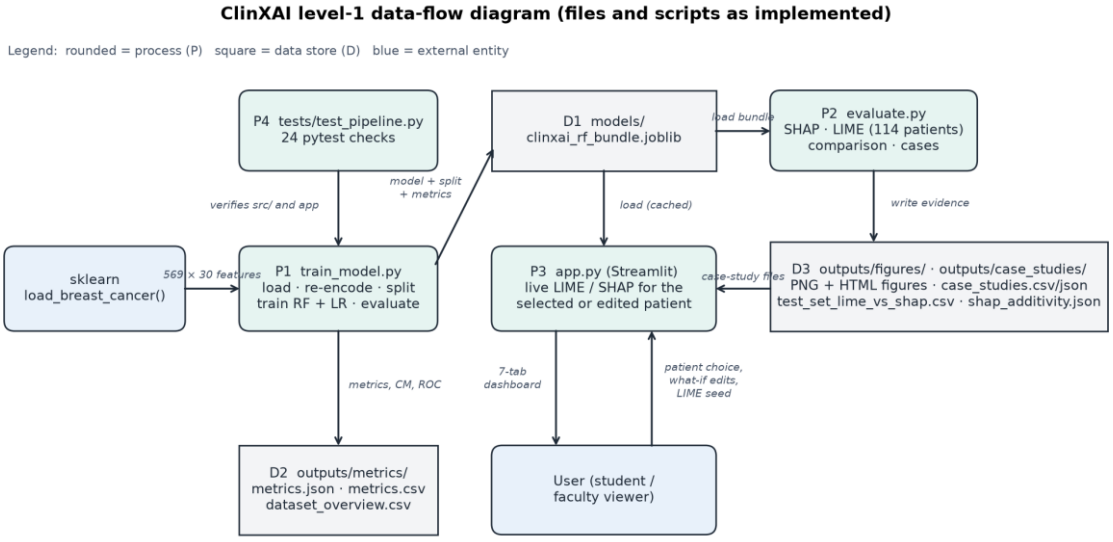


Figure 3.5: Level-1 data-flow diagram of ClinXAI (scripts and files as implemented)

Table 3.5: Component interface specification

Component	Inputs	Outputs
data_loader.load_dataset()	—	X (569 × 30 DataFrame), y (malignant = 1), feature names
data_loader.split_data()	X, y	DataSplit (X/y train and test, feature names)
model.train_random_forest()	X_train, y_train	Fitted RandomForestClassifier

Component	Inputs	Outputs
explain_lime.explain_patient_lime()	model, X_train, 1-row patient, seed, N	LimeResult: table, weights (30), intercept, local_pred, R^2, model_proba
explain_shap.compute_shap()	TreeExplainer, X	shap.Explanation (n × 30, malignant class)
comparison.compare_explanations()	LIME weights, SHAP values (same index)	Top-5 lists, overlap, Jaccard, Spearman, direction table
comparison.select_case_studies()	Per-patient summary table	List of 5 cases with selection criteria
data_loader.validate_patient_input()	dict / Series / sequence	Validated 1-row DataFrame or ValueError

The case-study output follows a fixed JSON schema. The excerpt below shows the record written for Patient #86, with the long `direction_table`, `lime_top10` and `shap_top10` arrays truncated:

Interface schema — record for Patient #86 in `outputs/case_studies/case_studies.json` (numbers shortened to 4 decimals)


```
{
  "case": 4,
  "case_type": "Ranking divergence",
  "patient_id": 86,
  "criterion": "Lowest top-5 Jaccard overlap between LIME and SHAP (ties: lowest
    Spearman)",
  "found": true,
  "true_label": "Malignant",
  "p_malignant": 0.2966,
  "shap_base_value": 0.3746,
  "shap_sum": -0.0780,
  "lime_local_pred": 0.4892,
  "lime_local_r2": 0.0274,
  "comparison": {
    "k": 5,
    "lime_top_k": [
      "worst area",
      "mean texture",
      "worst concavity",
      "worst texture",
      "fractal dimension error"
    ],
    "shap_top_k": [
      "mean concavity",
      "worst perimeter",
      "worst concave points",
      "mean concave points",
      "area error"
    ],
    "top_k_overlap": 0,
    "jaccard_top_k": 0.0,
    "spearman_abs_all_features": 0.2689,
    "direction_disagreements_union": [
      "worst area",
      "worst perimeter",
      "mean concave points",
      "area error"
    ]
  },
  "lime_stability": {
    "seeds": [
      0,
```

```
      1,
      2,
      3,
      4
    ],
    "mean_pairwise_jaccard_top_k": 0.3857,
    "min_pairwise_jaccard_top_k": 0.0,
    "features_with_sign_flip": 16
  }
}
  "direction_table": [ ... ], "lime_top10": [ ... ], "shap_top10": [ ... ]
```

CHAPTER 4: IMPLEMENTATION DETAILS, ALGORITHMS & TECHNOLOGICAL ARTIFACTS

4.1 Development Environment and Technology Stack

All work was done on a Windows 11 PC using only the CPU, in a project virtual environment (`.venv`). The exact package versions were recorded with `pip freeze` and pinned in `requirements.txt`. A clean installation into a new environment was verified by re-running the full test suite.

Table 4.1: Technology stack (exact versions used and tested)

Component	Version	Role in ClinXAI
Python	3.12.10	Programming language and runtime
NumPy / pandas	2.5.4 / 3.1.0	Arrays and tabular data handling
SciPy	1.18.1	Spearman rank correlation
scikit-learn	1.9.1	Dataset, Random Forest, Logistic Regression, metrics, CV
joblib	1.6.0	Model bundle serialisation
LIME	0.2.0.1	LimeTabularExplainer (local surrogate explanations)
SHAP	0.54.1	TreeExplainer, waterfall / beeswarm / force plots
Matplotlib	3.11.2	All static figures
Streamlit	1.66.1	Interactive web dashboard
pytest	9.1.1	Automated test suite
Playwright (verification only)	1.64.1	Real-browser click-through and screenshots (not an app dependency)
Operating system / hardware	Windows 11 Home	CPU only; no GPU or network service required

4.2 Dataset Loading and Target Mapping

The loader derives the malignant/benign mapping from sklearn's `target_names` instead of hard-coding it. It asserts the result against the raw labels in both directions and then validates the dataset:

Code excerpt 4.1 — src/data_loader.py, lines 26–49: dataset loading and malignant = 1 re-encoding

```
def load_dataset() -> tuple[pd.DataFrame, pd.Series, list[str]]:
    """Load Breast Cancer Wisconsin (Diagnostic) and re-encode so malignant = 1,
    benign = 0.

    sklearn ships the target as 0 = malignant, 1 = benign. The mapping is derived from
    ``target_names`` rather than hard-coded, then asserted.
    """
    raw = load_breast_cancer(as_frame=True)
    X = raw.data.copy()
    raw_target = raw.target.to_numpy()
    target_names = list(raw.target_names)
    raw_malignant = target_names.index("malignant")
    raw_benign = target_names.index("benign")

    y = pd.Series(
        np.where(raw_target == raw_malignant, MALIGNANT_LABEL, BENIGN_LABEL),
        index=X.index,
        name="diagnosis_malignant",
    )
    # Verify the re-encoding against sklearn's own labels.
    assert np.all((raw_target == raw_malignant) == (y.to_numpy() == MALIGNANT_LABEL))
    assert np.all((raw_target == raw_benign) == (y.to_numpy() == BENIGN_LABEL))

    validate_dataset(X, y)
    return X, y, list(X.columns)
```

4.3 Model Training Implementation

Algorithm 4.1 — pseudocode of the training and evaluation pipeline

```
Algorithm 4.1  Training and evaluation pipeline (train_model.py)
Input : sklearn breast-cancer data;  Output: model bundle, metrics, figures
1  X, y <- load_dataset()                # y = 1 for malignant
2  (Xtr, Xte, ytr, yte) <- stratified_split(X, y, test=0.20, seed=42)
3  rf <- RandomForest(T=300, gini, depth=None, seed=42).fit(Xtr, ytr)
4  assert rf.classes_ == [0, 1]          # column 1 of predict_proba = P(malignant)
5  lr <- Pipeline(StandardScaler, LogisticRegression).fit(Xtr, ytr)
6  for model in {rf, lr}:
7      p <- predict_proba(model, Xte)[:, 1];  yhat <- [p >= 0.5]
8      record accuracy, precision, recall, specificity, F1, ROC-AUC, confusion matrix
9  cv_auc <- 5-fold stratified CV ROC-AUC of a fresh RF on (Xtr, ytr) only
10 save bundle {rf, lr, feature order, split, metrics};  write JSON/CSV/PNG evidence
```

Code excerpt 4.2 — src/model.py, lines 34–38: Random Forest training with class-order assertion

```
def train_random_forest(X_train: pd.DataFrame, y_train: pd.Series) ->
    RandomForestClassifier:
    model = RandomForestClassifier(**RF_PARAMS)
    model.fit(X_train, y_train)
    assert list(model.classes_) == [0, 1], "Expected classes [0=benign, 1=malignant]"
    return model
```

Code excerpt 4.3 — src/model.py, lines 48–51: probability of the malignant class, looked up by label

```
def malignant_proba(model, X: pd.DataFrame) -> np.ndarray:
    """P(malignant) using the column that corresponds to label 1, not a positional
    guess."""
    col = list(model.classes_).index(MALIGNANT_LABEL)
    return model.predict_proba(X)[:, col]
```

`malignant_proba` looks up the column whose label is 1 instead of assuming a position. Together with the class-order assertion, this ensures that every reported probability really is P(malignant).

4.4 LIME Explainer Implementation

Two details make the LIME implementation correct and reproducible. First, LIME passes bare numpy arrays to the prediction function, so a wrapper restores the training column names and order before calling the forest. Second, a fresh explainer is created for every explanation with a fixed seed. Re-using one explainer would advance its random generator, so the same patient would get a different explanation on a second call. All 30 features are requested so that LIME and SHAP can be compared over the full feature set.

Code excerpt 4.4 — `src/explain_lime.py`, lines 24–34: explainer fitted on training data only

```
def build_lime_explainer(X_train: pd.DataFrame, seed: int = RANDOM_STATE) ->
    LimeTabularExplainer:
    """Fit LIME's sampling/discretisation statistics on TRAINING data only."""
    return LimeTabularExplainer(
        training_data=X_train.to_numpy(),
        feature_names=list(X_train.columns),
        class_names=CLASS_NAMES,
        mode="classification",
        discretize_continuous=True,
        discretizer="quartile",
        random_state=seed,
    )
```

Code excerpt 4.5 — `src/explain_lime.py`, lines 37–44: feature-order-safe prediction wrapper

```
def make_predict_fn(model, feature_names: list[str]):
    """LIME passes bare numpy arrays; rebuild a DataFrame so column order/name match
    training."""
    assert list(model.classes_) == [0, 1]

    def predict_fn(arr: np.ndarray) -> np.ndarray:
        return model.predict_proba(pd.DataFrame(arr, columns=feature_names))

    return predict_fn
```

Code excerpt 4.6 — `src/explain_lime.py`, lines 64–72: explaining label 1 (malignant) for one patient

```
    explainer = build_lime_explainer(X_train, seed)
    exp = explainer.explain_instance(
        x,
        make_predict_fn(model, feature_names),
        labels=(MALIGNANT_LABEL,),
        num_features=len(feature_names),
        num_samples=num_samples,
    )
    idx_weights = exp.as_map()[MALIGNANT_LABEL]
```

4.5 SHAP Explainer Implementation

The return format of `TreeExplainer.shap_values` has changed across SHAP versions: older releases return a list of per-class arrays, while version 0.54.1 returns one array of shape (n, 30, 2). `_select_malignant` normalises every format to the malignant-class matrix and its base value [25]. `additivity_error` then checks Equation (2.8) against the model's own probabilities.

Code excerpt 4.7 — `src/explain_shap.py`, lines 22–37: version-safe selection of the malignant class

```
def _select_malignant(values, expected_value) -> tuple[np.ndarray, float]:
    """Normalise SHAP's version-dependent return formats to (n, n_features) for class
    1."""
    ev = np.atleast_1d(np.asarray(expected_value, dtype=float))
    if isinstance(values, list):  # older SHAP: list of per-class arrays
        vals = np.asarray(values[MALIGNANT_LABEL])
        base = ev[MALIGNANT_LABEL]
    else:
        vals = np.asarray(values)
        if vals.ndim == 3:  # SHAP >= 0.45: (n_samples, n_features, n_classes)
            vals = vals[:, :, MALIGNANT_LABEL]
            base = ev[MALIGNANT_LABEL]
        elif vals.ndim == 2:  # single-output model
            base = ev[-1] if ev.size > 1 else ev[0]
        else:
            raise ValueError(f"Unexpected SHAP value shape {vals.shape}")
    return vals, float(base)
```

Code excerpt 4.8 — `src/explain_shap.py`, lines 40–49

```
def compute_shap(explainer: shap.TreeExplainer, X: pd.DataFrame) -> shap.Explanation:
    """Return a 2-D shap.Explanation (malignant class) for the rows of X."""
    raw = explainer.shap_values(X)
    vals, base = _select_malignant(raw, explainer.expected_value)
    return shap.Explanation(
        values=vals,
        base_values=np.full(vals.shape[0], base),
        data=X.to_numpy(),
        feature_names=list(X.columns),
    )
```

Code excerpt 4.9 — `src/explain_shap.py`, lines 52–56

```
def additivity_error(model, explanation: shap.Explanation, X: pd.DataFrame) -> float:
    """Max |base + sum(SHAP) - P(malignant)| over the rows of X."""
    reconstructed = explanation.base_values + explanation.values.sum(axis=1)
    proba = model.predict_proba(X)[:, MALIGNANT_LABEL]
    return float(np.max(np.abs(reconstructed - proba)))
```

4.6 LIME–SHAP Comparison Implementation

`compare_explanations` aligns both vectors by feature name, then computes the top-5 sets, Jaccard similarity, Spearman correlation of absolute attributions over all 30 features, and the direction table for the union of the two top-5 lists. Magnitudes are reported but never compared directly, because the two methods use different units.

Code excerpt 4.10 — `src/comparison.py`, lines 29–37: agreement measures (excerpt)

```
def compare_explanations(lime_weights: pd.Series, shap_values: pd.Series, k: int =
    TOP_K) -> dict:
    """Compare two attribution vectors indexed by the same feature names (same
    patient, class)."""
    lime_weights = lime_weights.reindex(shap_values.index).fillna(0.0)
    lime_top, shap_top = top_k(lime_weights, k), top_k(shap_values, k)
    shared = [f for f in lime_top if f in shap_top]
    union = list(dict.fromkeys(lime_top + shap_top))

    rho, _ = spearmanr(lime_weights.abs().to_numpy(), shap_values.abs().to_numpy())
```

Algorithm 4.2 — pseudocode of the case-study selection rules

```
Algorithm 4.2  Programmatic case-study selection (comparison.select_case_studies)
Input : per-patient summary of the 114 test patients;  ties broken by patient_id
1  case1 <- argmax P(malignant)                              # clearly malignant
2  case2 <- argmin P(malignant)                              # clearly benign
3  case3 <- argmin |P(malignant) - 0.5|                      # borderline
4  case4 <- argmin (Jaccard_top5, then Spearman) among remaining   # ranking
    divergence
5  case5 <- argmax n_direction_disagreements among remaining   # direction
    disagreement
6  if max disagreements = 0: report case5 as NOT FOUND (never fabricated)
```

4.7 Streamlit Interface Implementation

`app.py` builds the dashboard with a custom medical-style theme (`.streamlit/config.toml`), a sidebar for patient selection and LIME settings, and seven tabs (Figure 4.1). Expensive resources are cached as shown below. If the model file is missing, the app trains it automatically on first launch. If the case-study files are missing, the Case Studies tab shows instructions instead of failing.



Figure 4.1: ClinXAI dashboard window (real screenshot, Microsoft Edge, 1440 × 900): sidebar patient selector and LIME settings, banner, seven tabs and disclaimer

Code excerpt 4.11 — `app.py`, lines 61–85: cached resources and per-patient LIME cache

```
@st.cache_resource(show_spinner="Loading model and computing SHAP values …")
def load_resources():
    if not MODEL_PATH.exists():
        import train_model
        train_model.main()
    bundle = load_bundle()
    explainer = build_tree_explainer(bundle["model"])
    shap_test = compute_shap(explainer, bundle["X_test"])
    return {
        **bundle,
        "explainer": explainer,
        "shap_test": shap_test,
        "importance": global_importance(shap_test),
        "proba_test": malignant_proba(bundle["model"], bundle["X_test"]),
        "additivity_error": additivity_error(bundle["model"], shap_test,
    bundle["X_test"]),
    }


@st.cache_data(show_spinner="Running LIME (local perturbations) …", max_entries=64)
def cached_lime(values: tuple, seed: int, num_samples: int):
    R = load_resources()
    patient = pd.DataFrame([values], columns=R["feature_names"])
    return explain_patient_lime(R["model"], R["X_train"], patient, seed=seed,
    num_samples=num_samples)
```

4.8 Input Validation and Error Handling

Every patient vector passes through `validate_patient_input` before it reaches the model, whether it comes from the test set, from what-if edits or from a test case. The

function accepts a dictionary, a pandas Series or a sequence. It reorders the values into training order and rejects anything that is physically or structurally invalid:

Code excerpt 4.12 — `src/data_loader.py`, lines 82–117: defensive validation of biomarker input

```
def validate_patient_input(values, feature_names: list[str]) -> pd.DataFrame:
    """Validate a single patient's biomarker values and return a correctly ordered 1-
    row frame.

    Accepts a dict (feature -> value), a pandas Series, or a 1-D sequence in training
    order.
    Raises ValueError for missing/extra features, non-numeric, non-finite or negative
    values
    (all 30 features are non-negative physical measurements).
    """
    if isinstance(values, pd.DataFrame):
        if len(values) != 1:
            raise ValueError("Exactly one patient row is required")
        values = values.iloc[0]
    if isinstance(values, (dict, pd.Series)):
        keys = list(values.keys())
        missing = [f for f in feature_names if f not in keys]
        extra = [k for k in keys if k not in feature_names]
        if missing:
            raise ValueError(f"Missing features: {missing}")
        if extra:
            raise ValueError(f"Unknown features: {extra}")
        ordered = [values[f] for f in feature_names]
    else:
        ordered = list(values)
        if len(ordered) != len(feature_names):
            raise ValueError(f"Expected {len(feature_names)} values, got
    {len(ordered)}")

    try:
        arr = np.asarray(ordered, dtype=float)
    except (TypeError, ValueError) as exc:
        raise ValueError(f"All biomarker values must be numeric: {exc}") from exc
    if not np.isfinite(arr).all():
        bad = [f for f, v in zip(feature_names, arr) if not np.isfinite(v)]
        raise ValueError(f"Non-finite values for: {bad}")
    if (arr < 0).any():
        bad = [f for f, v in zip(feature_names, arr) if v < 0]
        raise ValueError(f"Negative values are not physically valid for: {bad}")
    return pd.DataFrame([arr], columns=feature_names)
```

Table 4.2: Defensive-engineering guardrails and their verified behaviour

Situation	Guardrail	Verified by
Missing / unknown feature, wrong length	ValueError naming the features	3 pytest cases
NaN, ±infinity, non-numeric text	ValueError (non-finite / not numeric)	3 pytest cases
Negative measurement	ValueError (physically invalid); UI fields have min_value = 0	pytest case
Value outside the training range	Warning: model is extrapolating; explanations less reliable	Browser screenshot (Figure 5.28)
Class order or feature order mismatch	Assertions at training and bundle load	pytest + validation
Different SHAP return format	_select_malignant handles list / 3-D / 2-D outputs	pytest case

Situation	Guardrail	Verified by
SHAP plot API unavailable	Fallback to summary_plot / manual waterfall	Code path in visualization.py
Interactive force plot cannot load	Static Matplotlib force plot is always shown first	Browser check
Model bundle missing	App trains the model on first launch	Code path in app.py
Case-study outputs missing	Informative message instead of an exception	Code path in app.py
No direction-disagreement patient exists	Case reported as not found; never fabricated	pytest case

4.9 Code Structure and Module Responsibilities

Table 4.3: Module responsibilities and size (lines of code, including comments and blank lines)

File	Responsibility	Lines
app.py	Streamlit dashboard: sidebar, seven tabs, caching, what-if, stability toggle	488
train_model.py	Training, evaluation, CV, model bundle, metrics, confusion matrix, ROC	89
evaluate.py	SHAP global/local, LIME for all test patients, comparison, case studies, figures	183
src/config.py	Paths, seeds, class convention, RF and LIME parameters	41
src/data_loader.py	Loading, re-encoding, validation, split, overview, patient-input validation	124
src/model.py	Model training, P(malignant), metrics, CV, bundle save/load	109
src/explain_lime.py	LIME explainer, prediction wrapper, LimeResult	94
src/explain_shap.py	TreeExplainer, class selection, additivity, tables, global importance	80
src/comparison.py	Top-k, Jaccard, Spearman, direction table, LIME stability, case selection	125
src/visualization.py	All Matplotlib figures and the SHAP force-plot HTML export	298
tests/test_pipeline.py	24 automated tests (pipeline and Streamlit smoke test)	282

In total the application, scripts and tests contain 1,916 lines. Appendix A lists every source and configuration file in full.

CHAPTER 5: EXPERIMENTAL EVALUATION, TESTING, TRADE-OFF ANALYSIS & EMPIRICAL RESULTS

5.1 Experimental Methodology

The evaluation answers three questions. (1) How accurately does the Random Forest detect malignancy on patients it has never seen? (2) What do LIME and SHAP say about individual predictions and about the model as a whole? (3) How far do the two explanation methods agree, and when do they diverge? The protocol is:

- **Hold-out evaluation.** A single stratified 80/20 split (random_state = 42). Models are fitted on the 455 training patients and evaluated once on the 114 test patients at a 0.5 threshold, with malignant as the positive class. A 5-fold stratified cross-validation of ROC-AUC on the training set alone estimates variability.
- **Baseline.** A standardised Logistic Regression is evaluated on the identical split.
- **Explanation evaluation.** LIME (seed 42, 5,000 samples) and TreeSHAP explain all 114 test patients. SHAP additivity is checked for each one, and the agreement measures of Section 2.8 are computed per patient (`outputs/case_studies/test_set_lime_vs_shap.csv`).
- **Case studies.** Five patients are selected by the fixed rules of Algorithm 4.2, and LIME's seed stability is measured for each with seeds 0–4.
- **Verification.** 24 automated tests, an independent 35-check validation of every saved output, and a 34-check click-through in a real browser (Section 5.11).

All numbers in this chapter come from the files generated by `train_model.py` and `evaluate.py`. The descriptive analysis in Section 5.9 recomputes summaries from the saved per-patient CSV and does not retrain or re-explain anything.

5.2 Dataset Characteristics and Train/Test Split

Table 5.1: Class composition of the full dataset and the stratified split

Subset	Patients	Malignant (1)	Benign (0)	Malignant share
Full dataset	569	212	357	37.3%
Training set	455	170	285	37.4%
Test set (held out)	114	42	72	36.8%

Stratification keeps the malignant share almost identical across subsets (37.3%, 37.4% and 36.8%). The test set holds 42 malignant tumours, so each missed malignancy reduces recall by 2.4 percentage points.

5.3 Accuracy, Precision, Recall, F1 and ROC-AUC

With TP, TN, FP and FN counted for the malignant class, the metrics are defined as:

```
Accuracy = (TP + TN) / (TP + TN + FP + FN)                    (5.1)
Precision = TP / (TP + FP)                                    (5.2)
Recall (sensitivity) = TP / (TP + FN)                         (5.3)
Specificity = TN / (TN + FP)                                  (5.4)
F1 = 2 · Precision · Recall / (Precision + Recall)            (5.5)
```

ROC-AUC is the area under the curve of true-positive rate against false-positive rate as the threshold sweeps from 1 to 0. It equals the probability that a randomly chosen malignant patient receives a higher score than a randomly chosen benign one. Table 5.2 reports the measured results.

Table 5.2: Held-out test-set performance (114 patients, positive class = malignant, threshold 0.5)

Model	Accuracy	Precision	Recall	Specificity	F1-score	ROC-AUC
Random Forest (300 trees)	97.37%	100.00%	92.86%	100.00%	96.30%	0.9944
Logistic Regression (baseline)	96.49%	97.50%	92.86%	98.61%	95.12%	0.9960

The Random Forest classified 111 of 114 test patients correctly (**accuracy 97.37%**). It produced no false alarms (**precision 100.00%**, specificity 100.00%) but missed 3 of the 42 malignant tumours (**recall 92.86%**), giving **F1 = 96.30%**. Its **ROC-AUC of 0.9944** shows near-perfect ranking of malignant above benign patients. The 5-fold cross-validated ROC-AUC on the training set was 0.9880 ± 0.0073, consistent with the test result. The single test split should not be over-interpreted: with 42 malignant patients, one more or one fewer miss changes recall by 2.4 points.

5.4 Confusion Matrix and ROC Curve

Figure 5.1 shows the confusion matrix: TN = 72, FP = 0, FN = 3, TP = 39. Every error is a false negative, a malignant tumour predicted as benign. Clinically this is the most serious type of error, so these patients are examined in Sections 5.8 and 5.9. Figure 5.2 compares the ROC curves of both models.

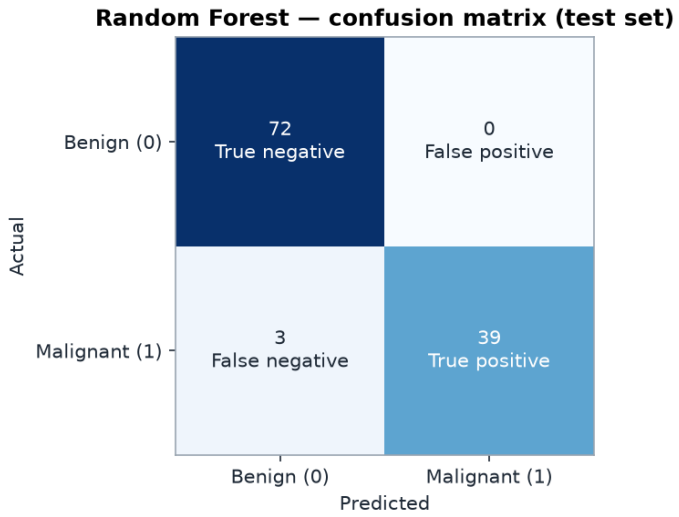


Figure 5.1: Random Forest confusion matrix on the 114-patient test set

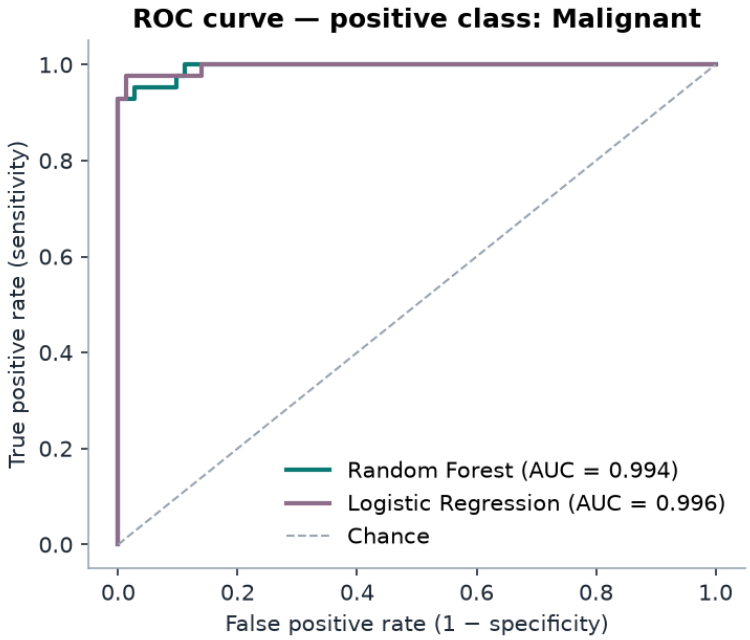


Figure 5.2: ROC curves of the Random Forest (AUC 0.994) and the Logistic Regression baseline (AUC 0.996)

Both curves rise almost vertically. The baseline's AUC is marginally higher (0.9960 vs 0.9944), while the forest has one fewer false positive at the 0.5 threshold. On 114 patients these differences are within sampling noise. Section 5.10 discusses what this means for the interpretability trade-off.

5.5 Global SHAP Feature Importance

Global importance is the mean absolute SHAP value of each feature over the 114 test patients, that is, the average change in P(malignant) attributed to the feature (Figure 5.3).

The beeswarm plot (Figure 5.4) adds direction: each dot is one patient, its horizontal position is the SHAP value, and its colour is the feature value.

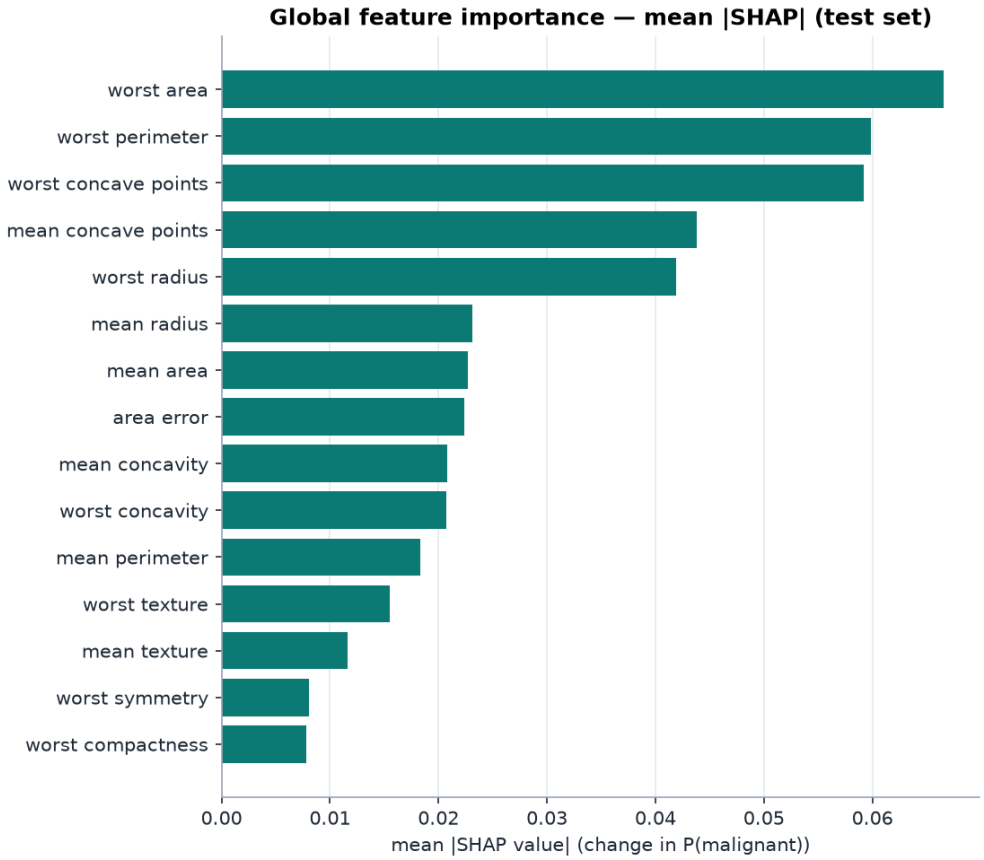


Figure 5.3: Global feature importance: mean |SHAP| on the test set (probability units)

Table 5.3: Ten most important features by mean |SHAP| (outputs/metrics/shap_global_importance.csv)

Rank	Feature	Mean |SHAP| (Δ P(malignant))
1	worst area	0.0666
2	worst perimeter	0.0598
3	worst concave points	0.0592
4	mean concave points	0.0438
5	worst radius	0.0419
6	mean radius	0.0232
7	mean area	0.0227
8	area error	0.0224
9	mean concavity	0.0208
10	worst concavity	0.0207

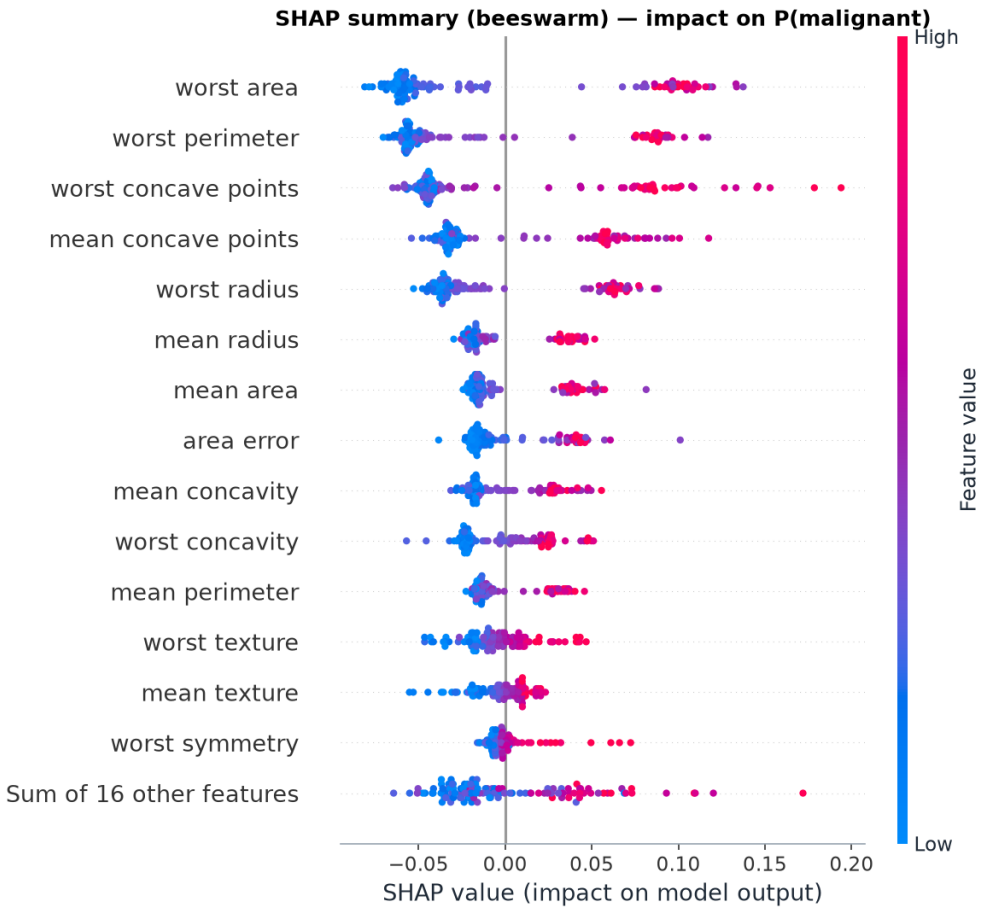


Figure 5.4: SHAP summary (beeswarm) plot: impact of each feature on P(malignant) for every test patient

Three groups dominate. They are the *worst* (largest) nuclear size measures (worst area, worst perimeter, worst radius), the concavity measures (worst and mean concave points, concavity), and mean size. The beeswarm shows a consistent direction: high values (red) push the probability up and low values (blue) push it down. This matches the class separation visible in the data itself (Figure 3.2). The model's global logic is therefore plausible. Plausible global logic, however, does not guarantee that each individual explanation is reliable, as the following sections show.

5.6 Individual Patient LIME Explanations

Figure 5.5 shows LIME for Patient #70, the most confident malignant prediction (P = 1.000). All ten displayed conditions are red, so every listed condition pushes toward malignant. The strongest are `worst area > 1122.50` (+0.1375) and `worst concave points > 0.16` (+0.1155). The surrogate fits well by LIME's standards ($R^2 = 0.63$). Its intercept is 0.268 and its prediction at the patient is 1.0006, slightly above 1. A linear surrogate is not bounded to the [0, 1] probability range.

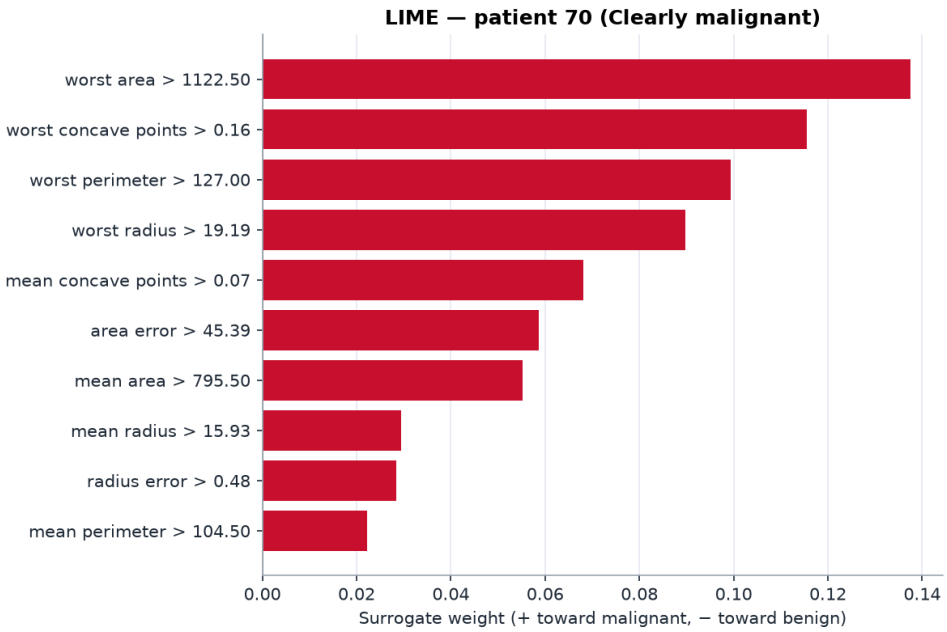


Figure 5.5: LIME explanation for Patient #70 (clearly malignant)

Figure 5.6 shows LIME for Patient #86, a malignant tumour the model predicted as benign (P = 0.297). The weights are an order of magnitude smaller, and the strongest conditions point *toward malignant* although the model's output is benign. The fit is very poor (R^2 = 0.03). The surrogate predicts 0.489 where the forest outputs 0.297, so the surrogate does not reproduce the decision it is meant to explain. One rule is displayed as `0.00 < fractal dimension error <= 0.00`. LIME rounds bin edges to two decimals, and this feature's values are all below 0.03, so the rule is real but unreadable.

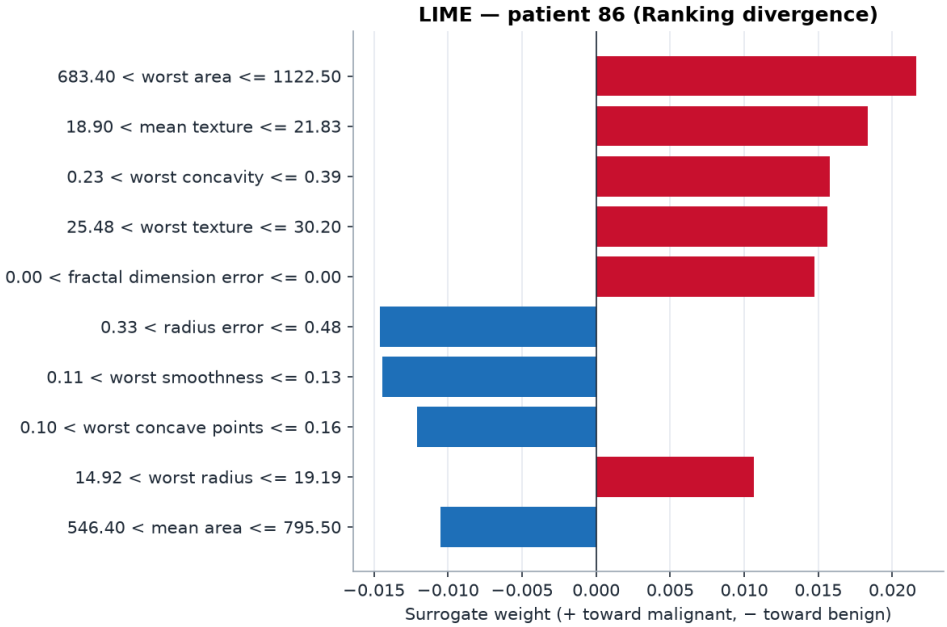


Figure 5.6: LIME explanation for Patient #86 (false negative)

5.7 Individual Patient SHAP Explanations

The waterfall plot starts at the base value E[f(X)] = 0.3747 and adds one SHAP value per feature until it reaches the model output. For Patient #70 (Figure 5.7) every displayed contribution is positive and their sum is +0.6253, so 0.3747 + 0.6253 = 1.000. For Patient #86 (Figure 5.8) the contributions conflict: mean concavity (+0.0496) and area error (+0.0270) push toward malignant, while worst perimeter (−0.0344), worst concave points (−0.0309) and mean concave points (−0.0308) push toward benign. The net effect is −0.0780, which gives 0.3747 − 0.0780 = 0.2967.

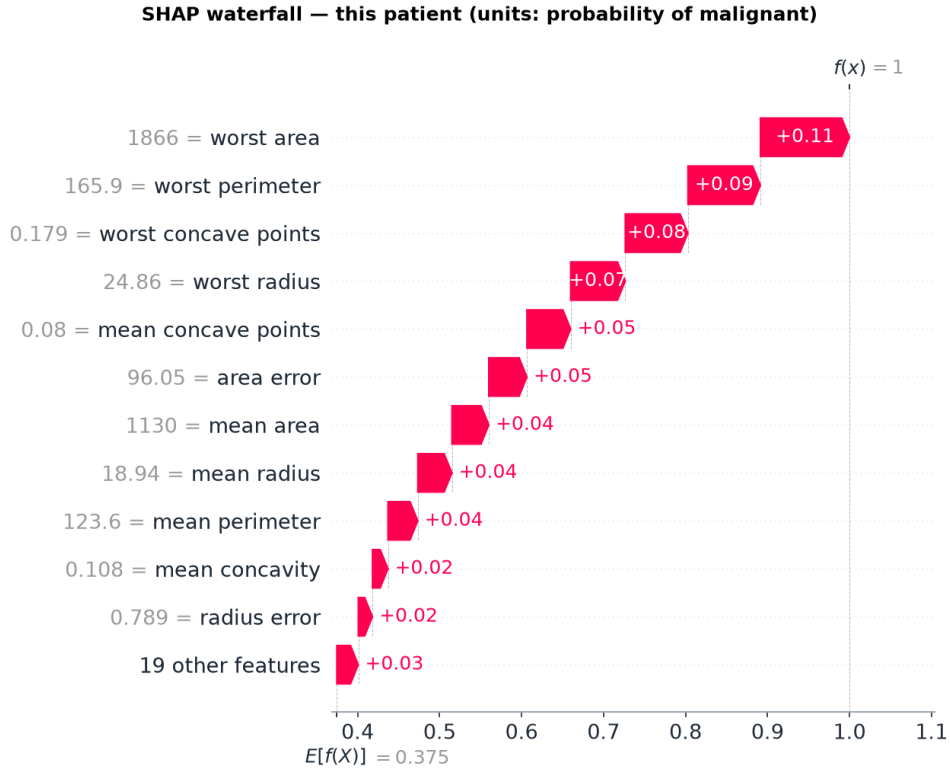


Figure 5.7: SHAP waterfall plot for Patient #70

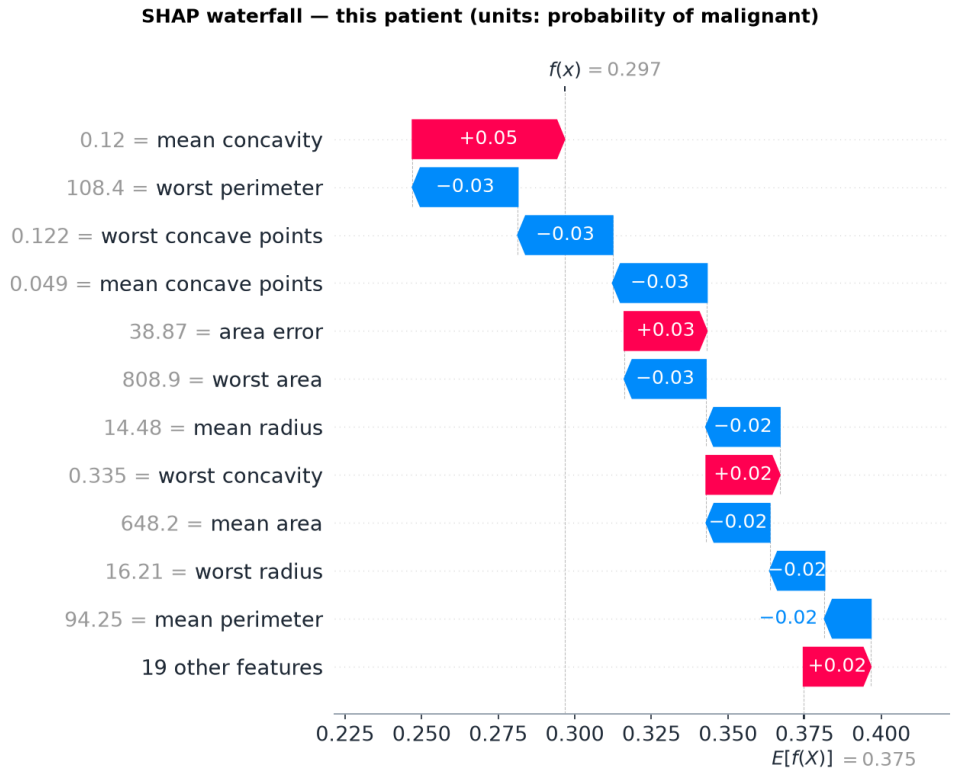


Figure 5.8: SHAP waterfall plot for Patient #86 (false negative)

Figure 5.9 shows the force plot for the borderline Patient #100 (P = 0.513). Red forces (worst area, worst radius, worst texture, area error) push the output above the base value, and blue forces (worst perimeter, mean and worst concave points) push it back. The two sides almost cancel. The interactive HTML versions are saved as `outputs/figures/shap_force_*.html`.

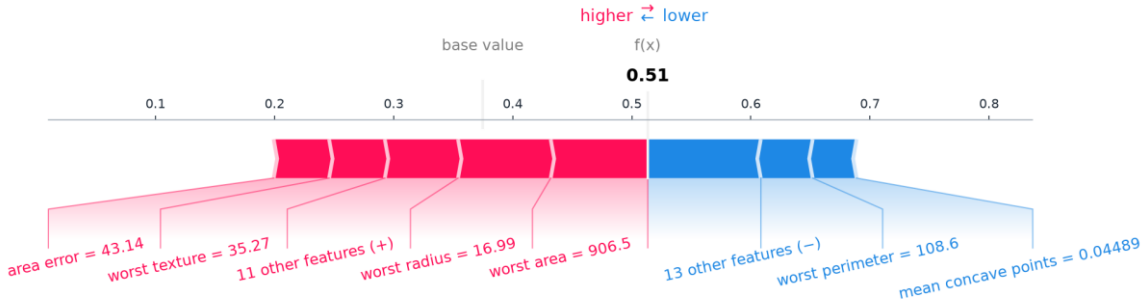


Figure 5.9: SHAP force plot for the borderline Patient #100 (minor features grouped as 'other features')

Additivity verification. Across all 114 test patients the largest difference between $\varphi_0 + \Sigma\varphi_j$ and the forest's `predict_proba` was **4.4e-16**, far below the 10^{-6} tolerance (`outputs/metrics/shap_additivity.json`). This confirms that the malignant class was

selected correctly and that base value, SHAP values and probabilities share the same units.

5.8 LIME–SHAP Case Study Comparison

Table 5.4 summarises the five programmatically selected case studies. All five requested case types were found in the held-out data, so none had to be reported as missing.

Table 5.4: Summary of the five case studies (same patient, same model, same class for both methods)

Case	Patient	P(mal.)	Actual / predicted	Top-5	Jaccard	ρ	Sign conflicts	LIME R^2
Clearly malignant	#70	1.000	Malignant / Malignant	5/5	1.00	0.90	0	0.63
Clearly benign	#74	0.000	Benign / Benign	5/5	1.00	0.83	0	0.17
Borderline	#100	0.513	Malignant / Malignant	2/5	0.25	0.60	2	0.05
Ranking divergence	#86	0.297	Malignant / Benign (false negative)	0/5	0.00	0.27	4	0.03
Direction disagreement	#73	0.160	Malignant / Benign (false negative)	1/5	0.11	0.39	4	0.06

Case 1 — Clearly malignant (Patient #70)

Table 5.5: Patient #70: LIME top-5 conditions and SHAP top-5 attributions

Rank	LIME condition	LIME weight	SHAP feature = value	SHAP value
1	worst area > 1122.50	+0.1375	worst area = 1866	+0.1087
2	worst concave points > 0.16	+0.1155	worst perimeter = 165.9	+0.0884
3	worst perimeter > 127.00	+0.0993	worst concave points = 0.1789	+0.0765
4	worst radius > 19.19	+0.0897	worst radius = 24.86	+0.0666
5	mean concave points > 0.07	+0.0681	mean concave points = 0.07951	+0.0534

Model output. P(malignant) = 1.000, correct; every tree votes malignant. **LIME.** The five strongest conditions are high worst area, worst concave points, worst perimeter, worst radius and mean concave points. **SHAP.** The same five features, with worst perimeter and worst concave points swapped in rank. **Agreement.** The top-5 sets are identical (Jaccard 1.00), $\rho = 0.90$, and there are no sign conflicts (Figure 5.10). **Why they agree.** The patient lies far inside the malignant region. Every feature value is extreme, so the forest's response is nearly flat around the patient and a linear surrogate describes it reasonably well ($R^2 = 0.63$). **Clinical limitation.** Agreement shows that both methods

describe the model consistently. It does not show that these features *caused* the malignancy, and the explanation adds little to an already unambiguous case.

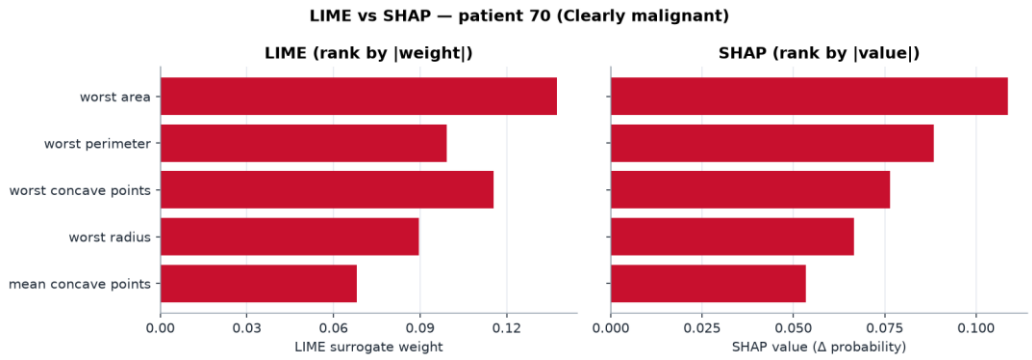


Figure 5.10: LIME vs SHAP for Patient #70 (full agreement)

Case 2 — Clearly benign (Patient #74)

Table 5.6: Patient #74: LIME top-5 conditions and SHAP top-5 attributions

Rank	LIME condition	LIME weight	SHAP feature = value	SHAP value
1	516.15 < worst area ≤ 683.40	−0.0747	worst area = 611.1	−0.0586
2	84.13 < worst perimeter ≤ 97.65	−0.0566	worst perimeter = 89.71	−0.0562
3	0.06 < worst concave points ≤ 0.10	−0.0516	worst concave points = 0.0866	−0.0413
4	13.02 < worst radius ≤ 14.92	−0.0464	worst radius = 14.11	−0.0353
5	0.02 < mean concave points ≤ 0.03	−0.0375	mean concave points = 0.02272	−0.0333

Model output. P(malignant) = 0.000, correct. **LIME and SHAP** select the same five features in exactly the same order (Jaccard 1.00, ρ = 0.83), all pushing toward benign. LIME's top-5 set is also perfectly stable across seeds (mean Jaccard 1.00). **The catch.** The surrogate's fit is low ($R^2 = 0.17$). It predicts 0.111 where the model outputs 0.000, an error of 0.11 on the probability scale. **Clinical limitation.** Agreement in *ranking* does not imply that the surrogate is *faithful*. A clinician reading LIME's weights as amounts of probability would be misled about their size even though the ordering is right.

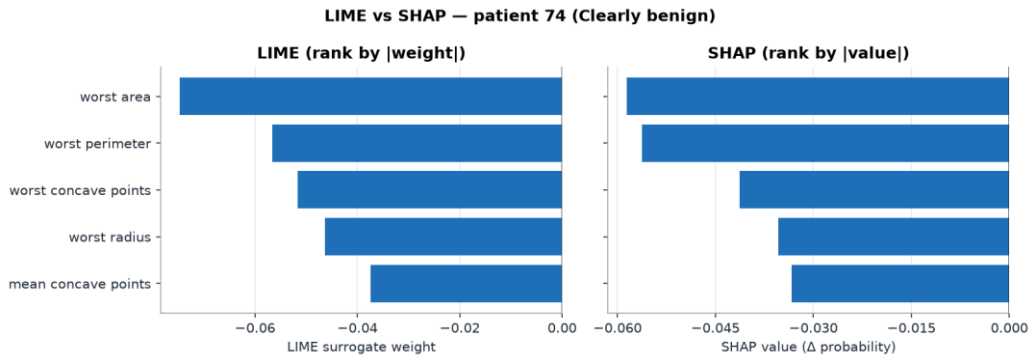


Figure 5.11: LIME vs SHAP for Patient #74 (full agreement)

Case 3 — Borderline prediction (Patient #100)

Table 5.7: Patient #100: LIME top-5 conditions and SHAP top-5 attributions

Rank	LIME condition	LIME weight	SHAP feature = value	SHAP value
1	worst texture > 30.20	+0.0458	worst area = 906.5	+0.0814
2	mean texture > 21.83	+0.0309	worst radius = 16.99	+0.0778
3	0.25 < worst symmetry ≤ 0.28	−0.0207	worst texture = 35.27	+0.0468
4	683.40 < worst area ≤ 1122.50	+0.0205	area error = 43.14	+0.0465
5	0.23 < worst concavity ≤ 0.39	+0.0183	worst perimeter = 108.6	−0.0436

Model output. P(malignant) = 0.513. This is the test patient closest to the threshold, and it is correctly but only barely classified malignant. **LIME** leads with texture (worst texture > 30.20, mean texture > 21.83). **SHAP** leads with size (worst area 906.5, worst radius 16.99). They share only worst texture and worst area (Jaccard 0.25, ρ = 0.60). Two features change sign (Figure 5.12). Area error is −0.0108 in LIME but +0.0465 in SHAP, and worst perimeter is +0.0082 in LIME but −0.0436 in SHAP. **Plausible reasons.** SHAP itself gives worst radius +0.078 but worst perimeter −0.044, although the two features are almost collinear (r = 0.994). The forest splits on one or the other, and the Shapley values follow those splits. LIME, which perturbs features independently, cannot reproduce that pattern. Near the 0.5 boundary the forest's response is non-linear, and the surrogate fits poorly (R^2 = 0.05). LIME is also unstable here: across five seeds its top-5 sets have a mean Jaccard of only 0.48. **Clinical limitation.** For a genuinely uncertain prediction, the two methods tell different stories, and neither should be offered as the reason for the diagnosis.

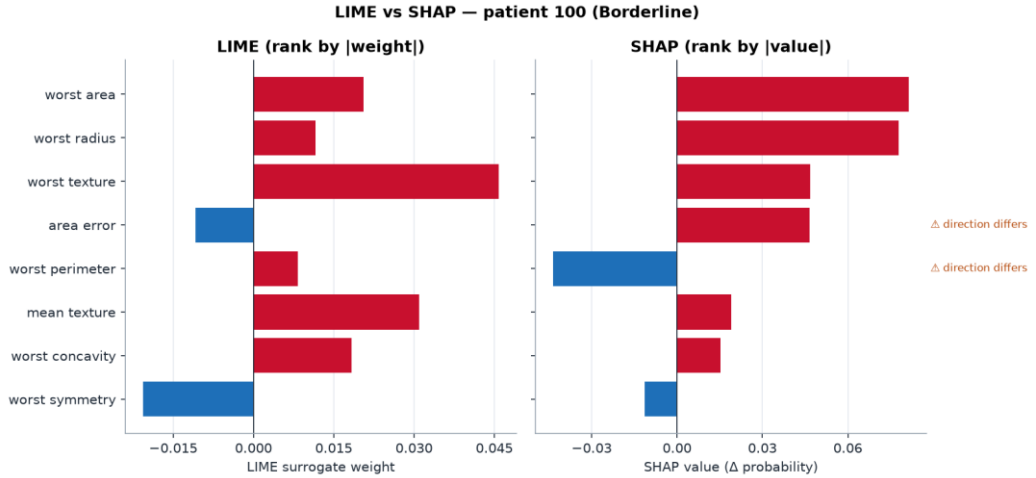


Figure 5.12: LIME vs SHAP for the borderline Patient #100 (two direction disagreements)

Case 4 — Ranking divergence, a missed malignancy (Patient #86)

Table 5.8: Patient #86: LIME top-5 conditions and SHAP top-5 attributions

Rank	LIME condition	LIME weight	SHAP feature = value	SHAP value
1	683.40 < worst area ≤ 1122.50	+0.0216	mean concavity = 0.1204	+0.0496
2	18.90 < mean texture ≤ 21.83	+0.0184	worst perimeter = 108.4	−0.0344
3	0.23 < worst concavity ≤ 0.39	+0.0158	worst concave points = 0.1225	−0.0309
4	25.48 < worst texture ≤ 30.20	+0.0156	mean concave points = 0.04938	−0.0308
5	0.00 < fractal dimension error ≤ 0.00	+0.0148	area error = 38.87	+0.0270

Model output. P(malignant) = 0.297, so the model predicts benign, but the tumour is **malignant**. This is a false negative. **LIME** ranks worst area (bin 683.40–1122.50), mean texture, worst concavity, worst texture and fractal dimension error highest, and all of them push toward malignant. **SHAP** ranks mean concavity, worst perimeter, worst concave points, mean concave points and area error highest, and its negative contributions win. The two top-5 lists share **no feature** (Jaccard 0.00, $\rho = 0.27$), and four features change sign: worst area, worst perimeter, mean concave points and area error (Figure 5.13).

Plausible reasons. (i) LIME's surrogate barely fits ($R^2 = 0.03$). It predicts 0.489 where the forest outputs 0.297, so its weights describe a model that is not the forest. (ii) LIME is unstable for this patient: the mean pairwise top-5 Jaccard across seeds is 0.39, the worst pair shares no feature at all (0.00), and 16 of 30 features change sign between seeds. (iii) The two methods use different reference points. The patient's worst area (808.9) lies inside the wide training-quartile bin 683.4–1122.5. LIME compares *being in this bin* with the other bins of the training distribution, and the lower bins are almost entirely benign, so the weight is positive. SHAP evaluates the exact value along the forest's own paths given the patient's other measurements, and relative to the base value it lowers the probability. Both statements can be true at once because they answer different questions.

Clinical limitation. This case shows the core risk. A clinician reading LIME would see malignant-leaning reasons attached to a benign label. A clinician reading SHAP would see a coherent, exactly additive rationale for a prediction that is wrong. SHAP faithfully explains *the model's* error; it does not reveal that the model is wrong. Neither explanation is evidence about the tumour.

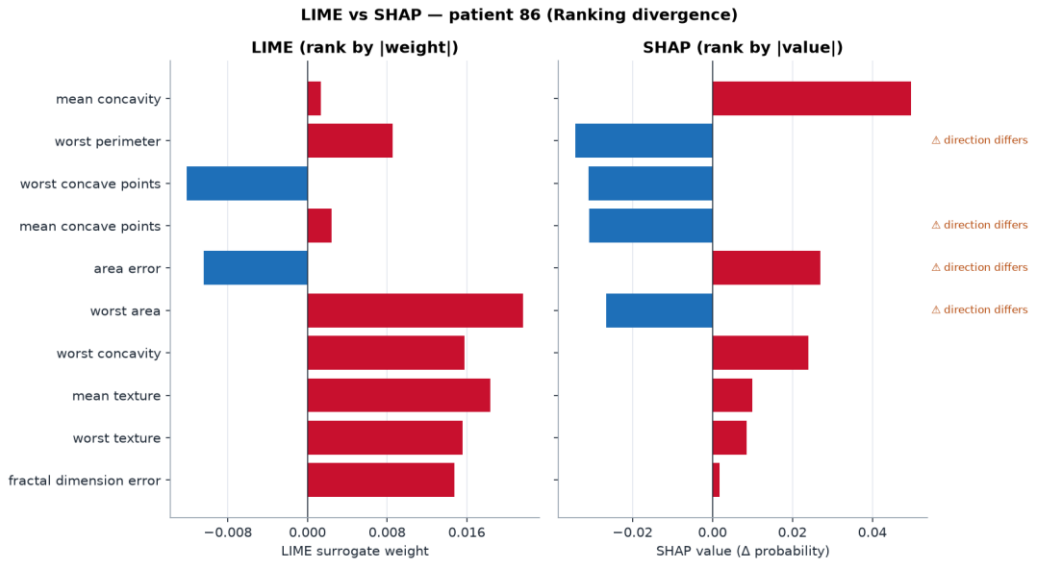


Figure 5.13: LIME vs SHAP for Patient #86 (false negative; no shared top-5 feature, four sign conflicts)

Case 5 — Direction disagreement, a missed malignancy (Patient #73)

Table 5.9: Patient #73: LIME top-5 conditions and SHAP top-5 attributions

Rank	LIME condition	LIME weight	SHAP feature = value	SHAP value
1	worst texture ≤ 21.49	−0.0450	worst perimeter = 110.3	−0.0377
2	mean texture ≤ 16.34	−0.0443	worst texture = 20.86	−0.0270
3	683.40 < worst area ≤ 1122.50	+0.0220	mean concavity = 0.07789	−0.0248
4	0.25 < worst symmetry ≤ 0.28	−0.0205	worst concave points = 0.1383	−0.0182
5	17.86 < area error ≤ 24.87	−0.0184	mean concave points = 0.05069	−0.0172

Model output. P(malignant) = 0.160, predicted benign, actually **malignant**: the second false negative examined in detail. **Agreement.** Both methods agree that low texture (worst texture 20.86, below the 21.49 bin edge) is a strong benign signal, and worst texture is the only shared top-5 feature (Jaccard 0.11, ρ = 0.39). **Disagreement.** Four features change sign (Figure 5.14): worst area (+0.0220 LIME vs −0.0109 SHAP), worst perimeter (+0.0081 vs −0.0377), mean concavity (+0.0029 vs −0.0248) and mean concave points (+0.0024 vs −0.0172). In every one of these conflicts LIME's weight is small and positive while SHAP's is clearly negative. SHAP's top-5 are all negative, so it explains why the forest leaned benign.

Plausible reasons. The fit is poor again (R^2 = 0.06), and the surrogate predicts 0.398 where the forest outputs 0.160. Worst area (812.4) falls in the same wide bin as Patient #86, with the same reference-point effect. The disputed features belong to strongly correlated groups: size (worst area, worst perimeter; r = 0.977) and concavity (mean and worst concave points; r = 0.912). Correlated features let the two methods spread the same

signal differently. **Clinical limitation.** The two missed malignancies are exactly where the explanations are least faithful and least consistent. Explanations from a single method would have hidden this.

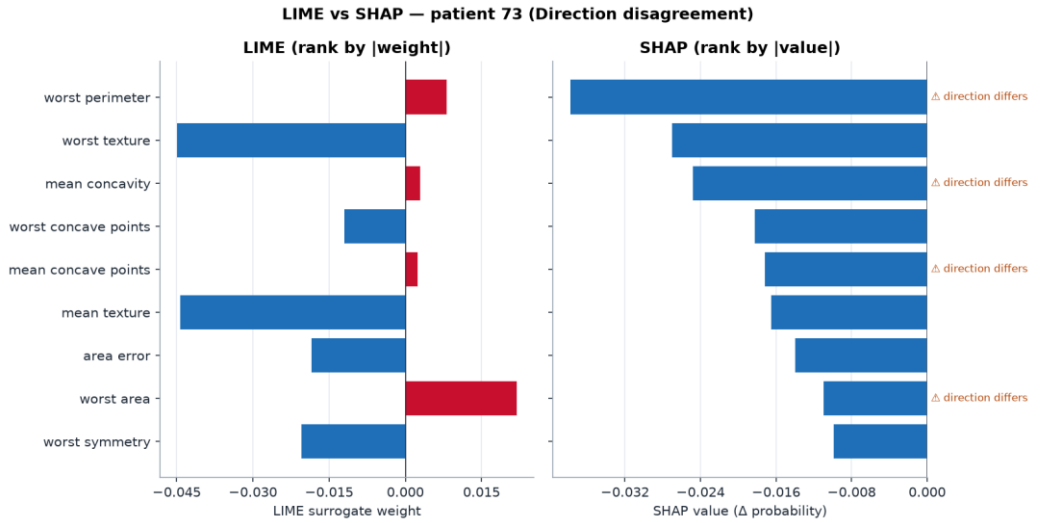


Figure 5.14: LIME vs SHAP for Patient #73 (false negative; four direction disagreements)

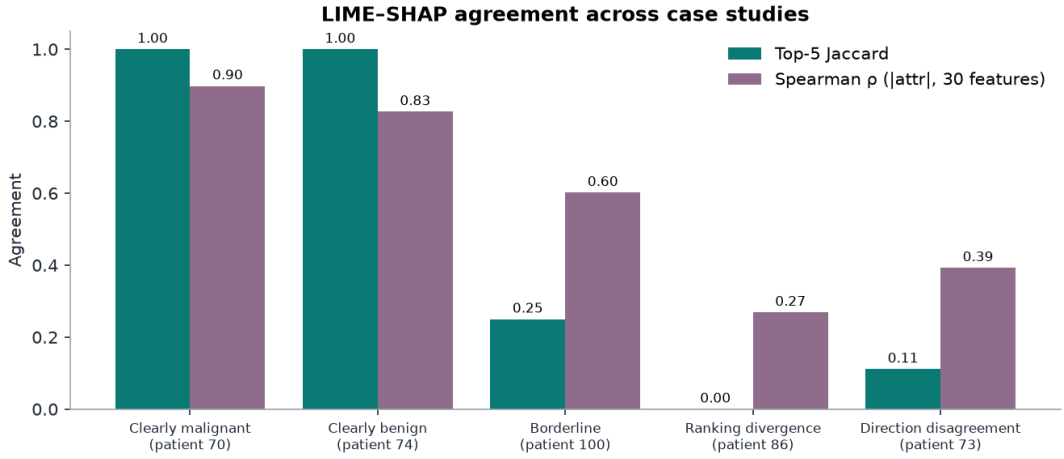


Figure 5.15: LIME–SHAP agreement (top-5 Jaccard and Spearman ρ) across the five case studies

5.9 Explanation Divergence Analysis

The case studies illustrate the pattern. The whole test set quantifies it. Table 5.10 gives the aggregate agreement over all 114 patients.

Table 5.10: Aggregate LIME–SHAP agreement over the 114 held-out patients

Measure	Value	Interpretation
Mean top-5 feature overlap	3.66 / 5	On average 3–4 shared top features
Mean top-5 Jaccard similarity	0.62	1 = identical sets, 0 = disjoint
Mean Spearman ρ of |attributions| (30 features)	0.70	Moderate-to-strong rank agreement
Patients with ≥ 1 direction disagreement	28.9% (33 of 114)	Sign conflict within the top-5 union
Mean LIME local-surrogate R^2	0.27	Low average local fidelity
Maximum SHAP additivity error	4.4e-16	Exact (numerical precision)

The distribution of top-5 overlap is uneven. Of 114 patients, 27 share all five features, 47 share four, 18 share three, 19 share two, 2 share one and 1 shares none (Table 5.11). Most patients therefore receive broadly consistent explanations, and a minority receive conflicting ones.

Table 5.11: Distribution of the number of shared top-5 features per test patient

Shared top-5 features	0	1	2	3	4	5
Number of patients	1	2	19	18	47	27

Disagreement tracks model uncertainty. Define confidence as |P(malignant) − 0.5|. Over the 114 patients, Spearman's correlation between confidence and top-5 Jaccard is 0.60, and between confidence and LIME's R^2 it is 0.49 (Figure 5.16). For the 90 confident patients (confidence ≥ 0.4), the mean Jaccard is 0.69 and the mean R^2 is 0.30. For the 24 less confident patients, they drop to 0.36 and 0.16.

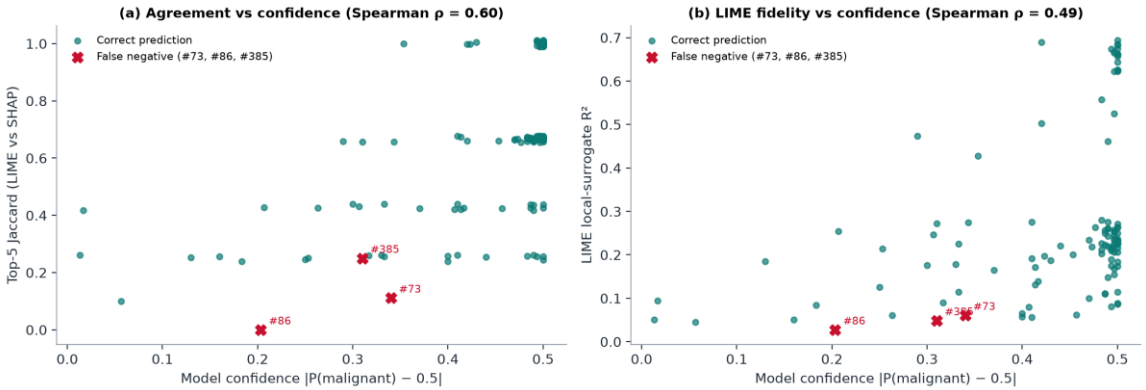


Figure 5.16: Explanation agreement (a) and LIME fidelity (b) against model confidence for all 114 test patients; false negatives marked (descriptive analysis of test_set_lime_vs_shap.csv)

All three false negatives fall in the low-agreement, low-fidelity region (Table 5.12). Each has a top-5 Jaccard ≤ 0.25, a LIME R^2 ≤ 0.061 and at least three direction disagreements. Only 12 of the 114 test patients have three or more disagreements, and 3 of them are false negatives. The third false negative, Patient #385, was not one of the five case studies. It shows the same signature, which suggests the pattern is systematic and not specific to the selected cases.

Table 5.12: All false negatives in the test set and their explanation-agreement profile

Patient	P(malignant)	Top-5 Jaccard	Spearman ρ	LIME R^2	Direction disagreements
#73	0.160	0.11	0.39	0.061	4
#86	0.297	0.00	0.27	0.027	4
#385	0.190	0.25	0.24	0.048	3

LIME stability. Table 5.13 reports how much LIME's own top-5 set changes when only the random seed changes (seeds 0–4, 5,000 samples). SHAP needs no such table, because TreeSHAP is deterministic for a fixed model.

Table 5.13: LIME seed stability for the five case studies (mean / minimum pairwise top-5 Jaccard)

Case	Patient	Mean Jaccard	Minimum Jaccard	Features changing sign (of 30)
Clearly malignant	#70	0.87	0.67	9
Clearly benign	#74	1.00	1.00	11
Borderline	#100	0.48	0.25	14
Ranking divergence	#86	0.39	0.00	16
Direction disagreement	#73	0.60	0.43	12

Even for the confident Patient #70, 9 low-weight features change sign between seeds. The top of the ranking stays stable, but the tail is noise. For Patient #86, two seeds give completely disjoint top-5 sets. When an explanation is not reproducible against itself, its disagreement with SHAP is unsurprising.

Why the methods diverge — synthesis. The evidence points to five mechanisms:

- **Different estimands and reference points.** LIME weights describe a bin indicator relative to other bins under the training distribution. SHAP values describe a contribution relative to the base value $E[f(X)] = 0.3747$. A mid-range value can raise risk relative to the first and lower risk relative to the second (worst area for Patients #86 and #73).
- **Linear surrogate vs non-linear boundary.** Near $P = 0.5$ the forest's response changes abruptly. A single linear surrogate cannot follow it, as the low R^2 values for uncertain patients show.
- **Correlated features.** 21 pairs of the 30 features have $|r| > 0.9$ in the training data, for example worst radius–worst perimeter ($r = 0.994$) and mean radius–mean perimeter ($r = 0.998$). Credit can be split arbitrarily among near-duplicates.
- **Independent perturbation.** LIME samples each feature independently, which creates unrealistic patients (for instance a large radius with a small perimeter) on which the forest's response is uninformative [22].
- **Sampling noise.** With 5,000 samples LIME's ranking is still seed-dependent for uncertain patients [18].

Implication. Low LIME local fidelity and LIME–SHAP disagreement are not cosmetic. They are concentrated on the predictions that most need scrutiny: borderline outputs and

missed cancers. In this project, disagreement between explainers is therefore a useful **warning signal** that a prediction deserves human review. It is not a reason to pick whichever explanation seems more convincing.

5.10 Model Performance and Interpretability Trade-offs

Table 5.14: Performance and interpretability trade-off between the two models

Aspect	Random Forest	Logistic Regression (baseline)
Accuracy / F1	97.37% / 96.30%	96.49% / 95.12%
Precision / Recall	100.00% / 92.86%	97.50% / 92.86%
ROC-AUC	0.9944	0.9960
Errors on 114 patients	FP = 0, FN = 3	FP = 1, FN = 3
Model size	10,976 tree nodes	30 coefficients + intercept
Interpretability	Post-hoc only (LIME / SHAP)	Intrinsic (standardised coefficients)
Explanation fidelity	SHAP exact; LIME approximate (mean R^2 0.27)	Exact by construction
Training time (both models + CV)	≈ 7 s for the full train_model.py run (CPU)	(included)

Several trade-offs follow from these results:

- **Accuracy vs interpretability.** On this dataset the forest's advantage is one fewer false positive. The transparent baseline has a slightly higher AUC, and both miss three malignant tumours. The accuracy gained by using a black box is negligible, which supports Rudin's argument [5]. The forest remains the subject of this study because the project's aim is to audit post-hoc explanations of a complex model.
- **Fidelity vs simplicity (LIME).** A five-feature linear story is easy to read, but its mean R^2 of 0.27 shows how much of the forest's local behaviour it leaves out.
- **Exactness vs generality.** TreeSHAP is exact and deterministic but works only for trees. LIME works for any model at the cost of sampling error.
- **Stability vs computation.** More LIME samples would reduce seed variance but multiply the number of model calls. `evaluate.py` already needs about 36 s on the CPU to run 139 LIME explanations (114 patients plus 25 stability runs), as well as SHAP and all figures.
- **Precision vs recall.** At the 0.5 threshold the forest has perfect precision but three false negatives, at P = 0.160, 0.190 and 0.297. A lower threshold would catch more cancers at the cost of false alarms. Such a threshold must be chosen on separate validation data, not on this test set, and calibration would be needed before probabilities are read as risks [26].

5.11 Operational Verification Screenshots

The screenshots below were captured from the running application (`streamlit run app.py`) in Microsoft Edge by an automated Playwright script, at 1440 px width and 2× pixel density. They show the real final build. Nothing was mocked or edited.

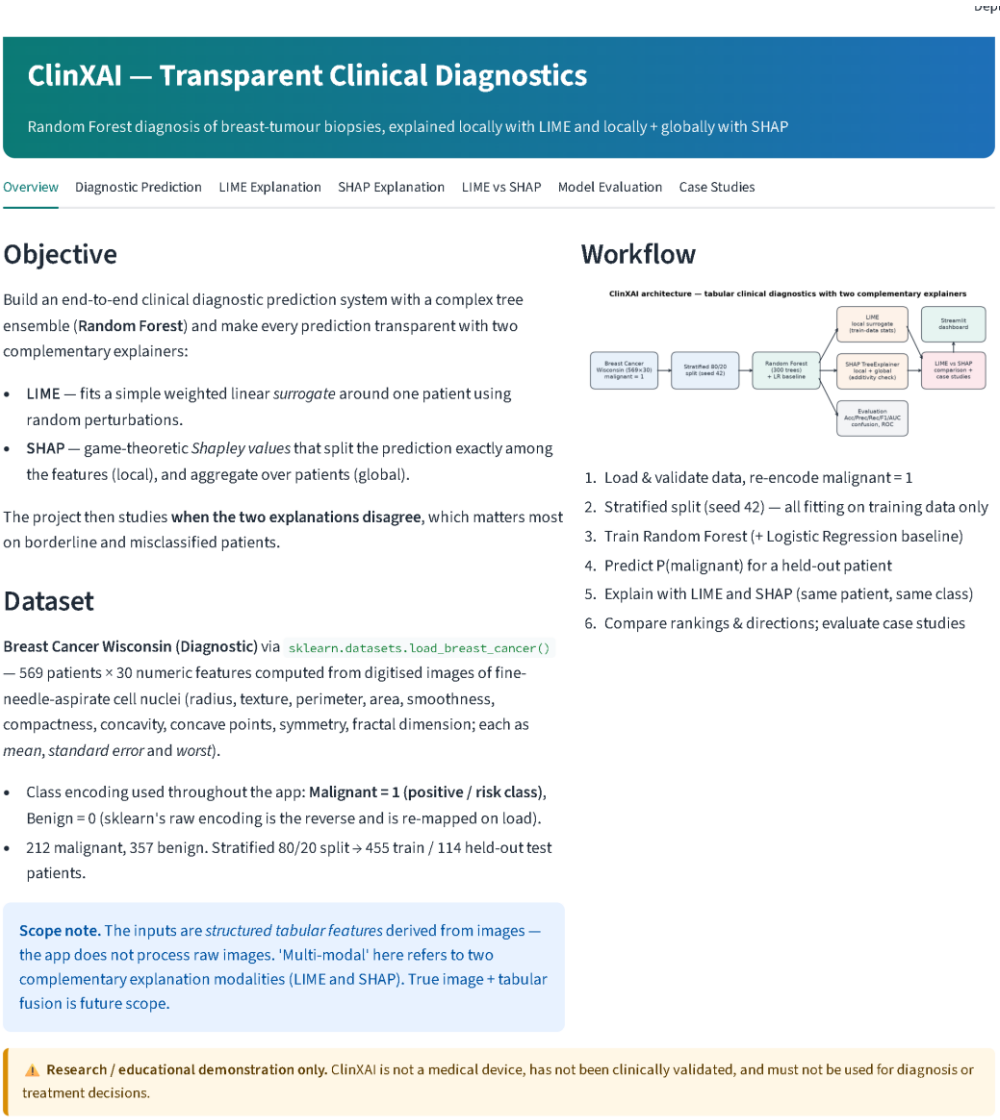


Figure 5.17: Overview tab: objective, dataset description, scope note, workflow diagram and educational-use disclaimer

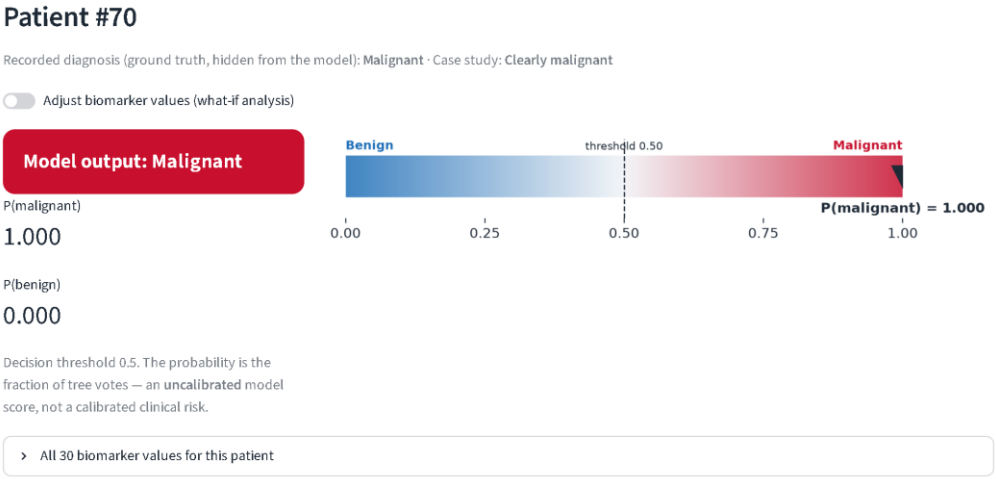


Figure 5.18: Diagnostic Prediction tab for Patient #70: output label, P(malignant) = 1.000, risk gauge and the uncalibrated-score note

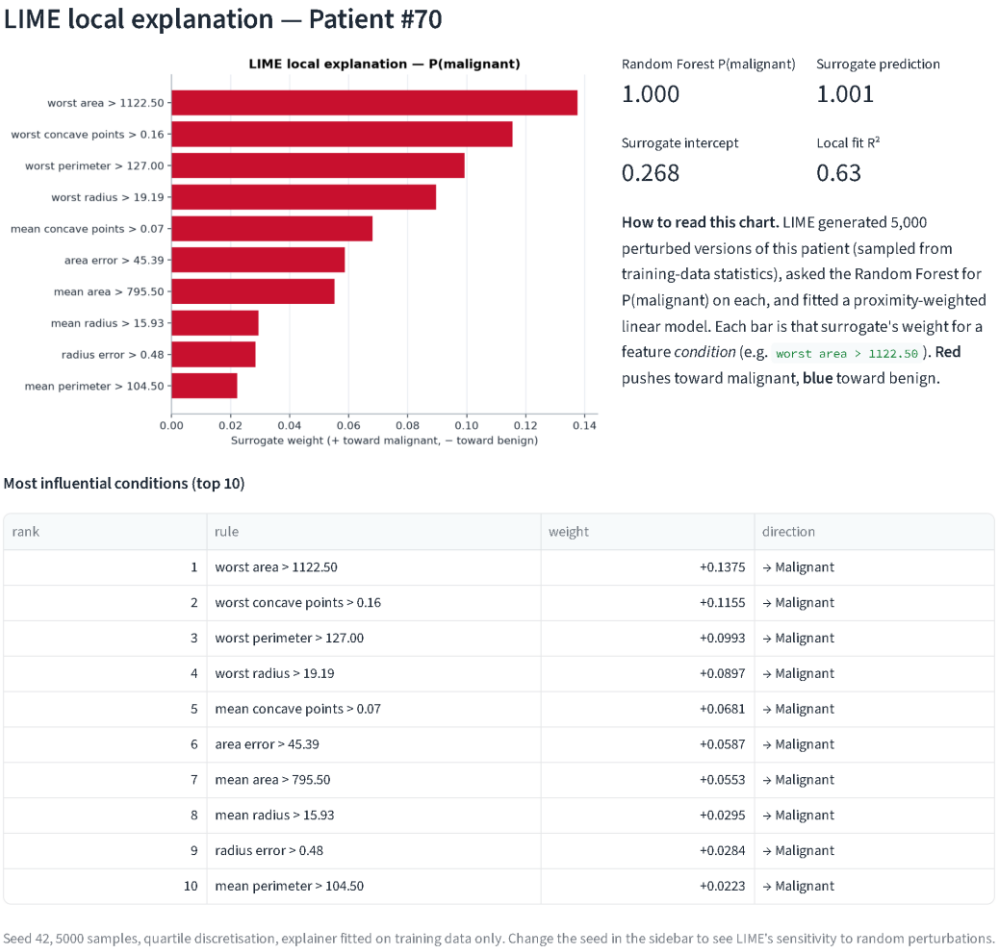


Figure 5.19: LIME Explanation tab for Patient #70: rule chart, surrogate metrics and plain-English reading guide

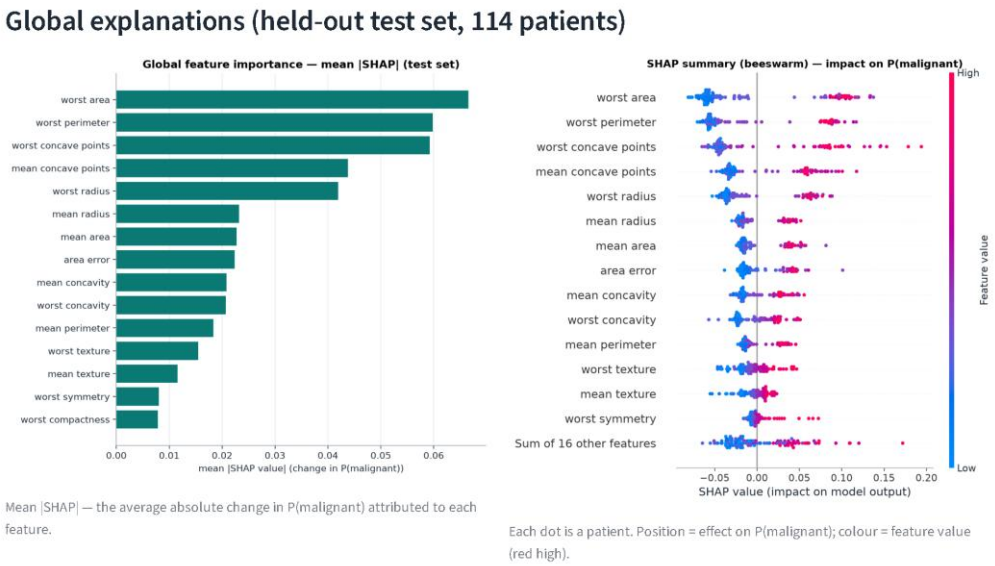
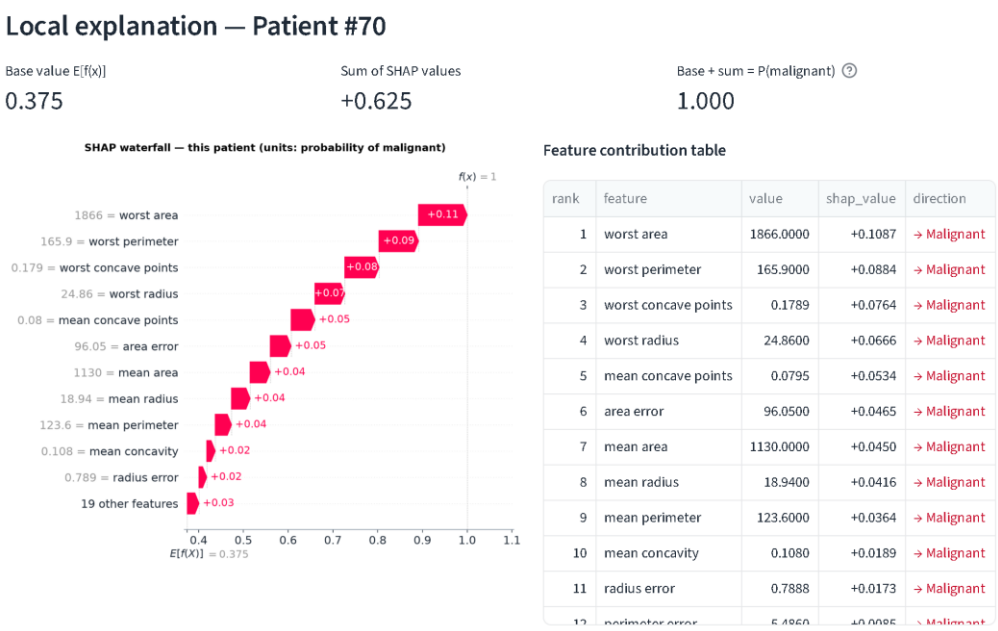


Figure 5.20: SHAP Explanation tab: global importance and beeswarm plots



rank	feature	value	shap_value	direction
1	worst area	1866.0000	+0.1087	→ Malignant
2	worst perimeter	165.9000	+0.0884	→ Malignant
3	worst concave points	0.1789	+0.0764	→ Malignant
4	worst radius	24.8600	+0.0666	→ Malignant
5	mean concave points	0.0795	+0.0534	→ Malignant
6	area error	96.0500	+0.0465	→ Malignant
7	mean area	1130.0000	+0.0450	→ Malignant
8	mean radius	18.9400	+0.0416	→ Malignant
9	mean perimeter	123.6000	+0.0364	→ Malignant
10	mean concavity	0.1080	+0.0189	→ Malignant
11	radius error	0.7888	+0.0173	→ Malignant
12	perimeter error	5.4860	+0.0086	→ Malignant

Figure 5.21: SHAP Explanation tab: additivity metrics, waterfall plot and contribution table for Patient #70

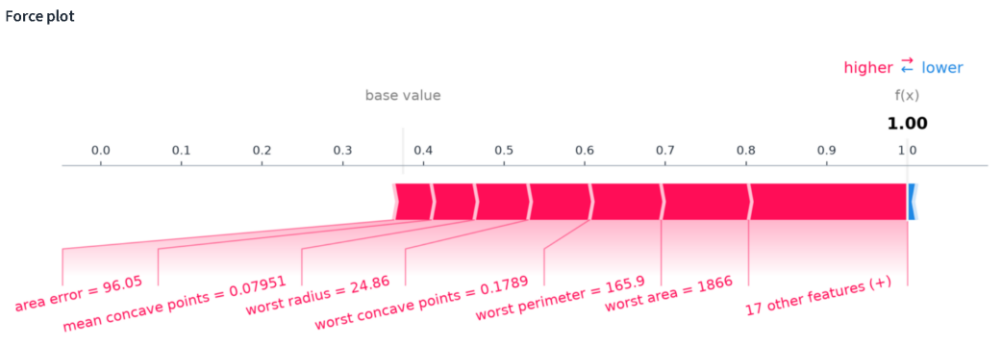


Figure 5.22: SHAP Explanation tab: static force plot for Patient #70

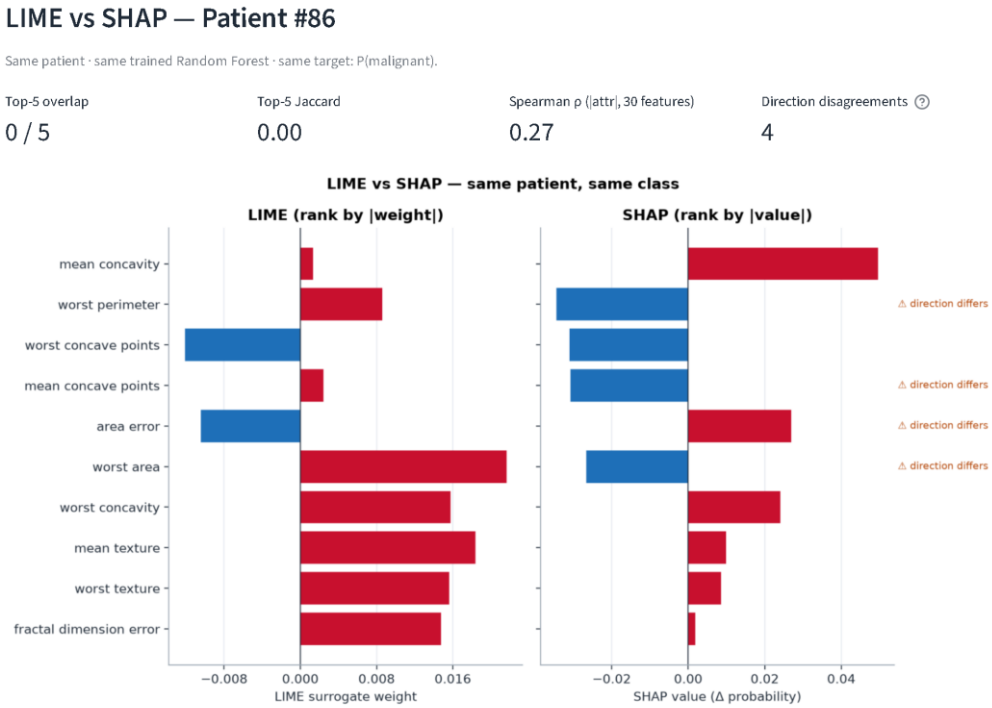


Figure 5.23: LIME vs SHAP tab for Patient #86: agreement metrics (0/5 overlap, 4 disagreements) and side-by-side chart

Ranking & direction table (union of both top-5 lists)

feature	lime_rank	shap_rank	lime_weight	shap_value	agreement
worst area	1	6	+0.0216	-0.0266	⚠ opposite
mean texture	2	14	+0.0184	+0.0099	✓ same direction
worst concavity	3	8	+0.0158	+0.0240	✓ same direction
worst texture	4	15	+0.0156	+0.0086	✓ same direction
fractal dimension error	5	25	+0.0148	+0.0018	✓ same direction
mean concavity	29	1	+0.0013	+0.0496	✓ same direction
worst perimeter	14	2	+0.0086	-0.0344	⚠ opposite
worst concave points	8	3	-0.0121	-0.0310	✓ same direction
mean concave points	24	4	+0.0024	-0.0308	⚠ opposite
area error	11	5	-0.0104	+0.0270	⚠ opposite

Opposite direction for: **worst area, worst perimeter, mean concave points, area error**

Measure LIME stability across 5 random seeds

Why can the explanations differ?

- **Different questions.** LIME fits a linear surrogate over *discretised* feature bins in a sampled neighbourhood; SHAP distributes the exact prediction using Shapley values along the trees' own splits. Their magnitudes are not interchangeable — only ranks and signs are compared here.
- **Correlated features.** Radius, perimeter and area (mean/worst) are almost collinear. Both methods may share credit among them differently, so a feature can rank high in one and low in the other.
- **Perturbation sampling.** LIME's samples are drawn independently per feature, creating unrealistic combinations (e.g. large radius with small area) that the forest never saw.
- **Weak local fit for this patient** — LIME $R^2 = 0.03$, so the surrogate is a poor approximation here.
- **Near the decision boundary** ($P = 0.30$): the forest's response is non-linear here, which a single linear surrogate cannot capture.

Jaccard and Spearman measure agreement between explanation methods — they are not clinical validation metrics.

Figure 5.24: LIME vs SHAP tab for Patient #86: ranking and direction table, warning and divergence reasons



Figure 5.25: Model Evaluation tab: metrics, confusion matrix, ROC curves and baseline comparison

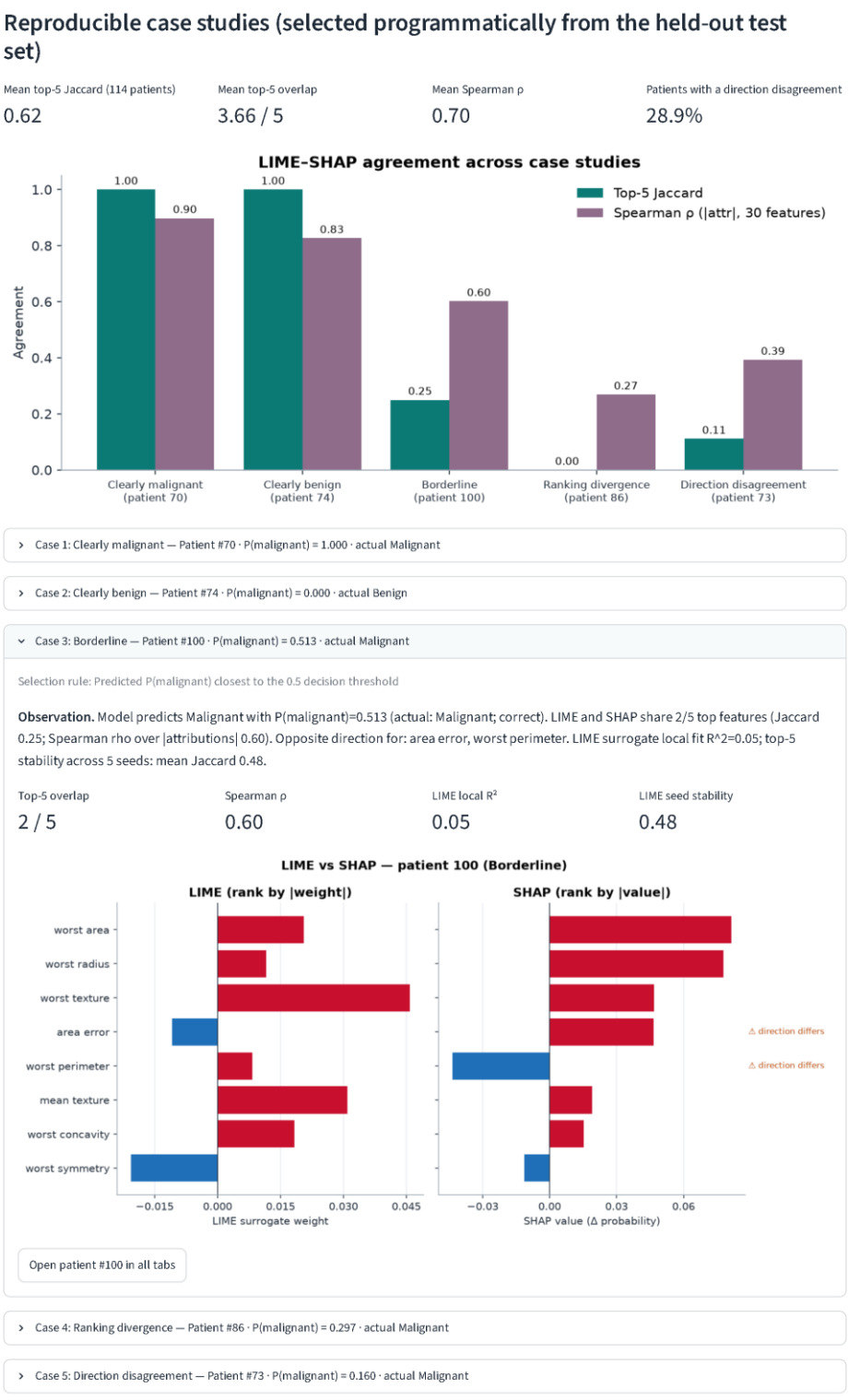


Figure 5.26: Case Studies tab: aggregate agreement, case overview chart and the expanded borderline case

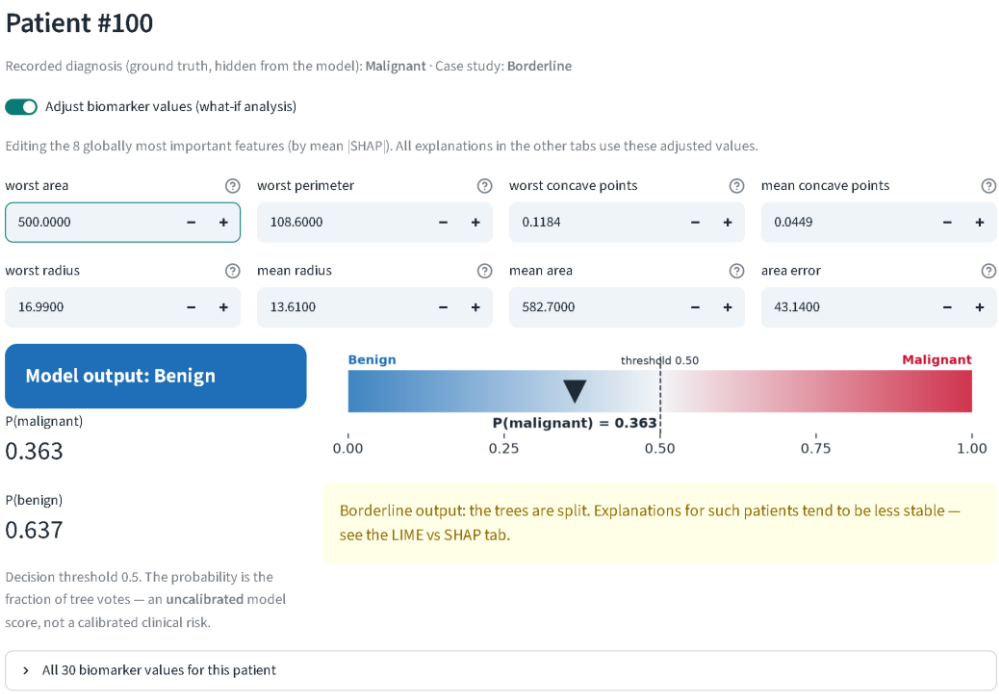


Figure 5.27: What-if analysis on Patient #100: lowering worst area from 906.5 to 500 flips the output to benign (P = 0.363)

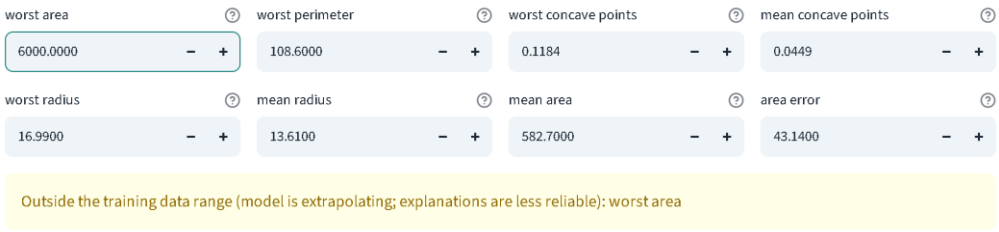


Figure 5.28: Error handling: a worst area of 6000 (training maximum 4254) triggers the extrapolation warning

Terminal verification. The final lines of the automated test run are reproduced below. The complete logs are in Appendix B.

Terminal output — `python -m pytest -v` (last lines)

```
tests/test_pipeline.py::test_invalid_numeric_input[<lambda>-numeric] PASSED [ 83%]
tests/test_pipeline.py::test_invalid_numeric_input[<lambda>-Missing] PASSED [ 87%]
tests/test_pipeline.py::test_invalid_numeric_input[<lambda>-Unknown] PASSED [ 91%]
tests/test_pipeline.py::test_invalid_input_length PASSED                 [ 95%]
tests/test_pipeline.py::test_streamlit_app_runs_and_updates_on_patient_change PASSED
    [100%]
============================== 24 passed in 17.82s ==============================
```

Table 5.15: Verification summary of the implemented system

Verification layer	Method	Result
Automated test suite	pytest, 24 tests (data, mapping, model, LIME, SHAP, additivity, plots, comparison, invalid input, app smoke test)	24 / 24 passed
Clean installation	New virtual environment from requirements.txt, then pytest	24 / 24 passed
Saved-output validation	Independent recomputation of metrics, SHAP, LIME and case studies	35 / 35 passed
Real-browser click-through	Playwright + Microsoft Edge: all tabs, patient switch, what-if, seed, stability, force plot, case jump	34 / 34 passed
Runtime warnings	train_model.py and evaluate.py with warnings enabled	0 warnings

CHAPTER 6: SOCIETAL IMPACT, ETHICAL GOVERNANCE, SAFETY & SECURITY AUDITS

6.1 Importance of Transparency in Clinical AI

In clinical decision support, transparency is a safety mechanism, not a courtesy. Explanations let a clinician check a prediction against their own knowledge, notice when a model relies on an implausible signal, and explain a decision to the patient [6]. The WHO lists ensuring transparency, explainability and intelligibility among its six principles for AI in health [11]. ClinXAI adds a second lesson from Chapter 5: transparency must cover the *explanations themselves*. The dashboard therefore shows LIME and SHAP side by side for the same patient and class, reports LIME's own fidelity (R^2) and stability, and flags sign conflicts, so that disagreement is visible instead of hidden.

6.2 Human Oversight and Clinical Decision Support

ClinXAI is designed as a **decision-support aid for education**, never as an autonomous diagnostic system. Its human-oversight features are:

- A persistent disclaimer on every view: research and educational demonstration only, not a medical device, no diagnosis or treatment decisions.
- The probability is labelled as an uncalibrated model score, so it is not read as a clinical risk.
- A borderline warning when $|P - 0.5| < 0.15$, and an extrapolation warning when inputs leave the training range.
- Explanation-quality indicators: a low-R^2 warning for LIME, the stability toggle and the direction-disagreement count.
- No recommendation is ever generated. The output is a score plus explanations for a human to evaluate.

A responsible clinical workflow built on this design would keep the clinician as the decision-maker. It would route any prediction that is borderline, out of range, or shows LIME–SHAP disagreement to mandatory human review, and it would record the final human decision separately from the model output.

6.3 Patient Privacy and Responsible Data Usage

The project uses only the public, de-identified Breast Cancer Wisconsin (Diagnostic) dataset [3]. It contains no names, identifiers or demographic attributes. ClinXAI collects no user data, makes no network calls to external AI services, and keeps all computation and files on the local machine. Under India's Digital Personal Data Protection Act, 2023 [27], processing identifiable patient data in a future clinical version would require, among other obligations, a lawful basis such as informed consent, purpose limitation and reasonable security safeguards. None of these is needed for, or claimed by, the present educational prototype.

6.4 Model Bias, Dataset Limitations and Generalisation

- **Single-centre, historical data.** All 569 samples come from one institution's FNA imaging pipeline in the early 1990s. The data cannot represent other populations, scanners, staining protocols or sample-preparation practices.
- **No demographic attributes.** Age, ethnicity and similar variables are absent, so subgroup performance and fairness *cannot be audited* with this data. The project does not claim to be fair across groups.
- **Class imbalance.** With 37% malignant cases, accuracy alone would flatter the model. This is why recall, precision, F1 and AUC are reported.
- **Distribution shift.** Features measured on new equipment could shift systematically. The model would then extrapolate, which the dashboard warns about only for values outside the training range, not for subtler shifts within it.
- **Small test set.** 114 patients, including 42 malignant, give wide uncertainty on every metric.

6.5 Safety Considerations and False Negatives

The model's only errors on the test set were **3 false negatives**: malignant tumours predicted benign (Patients #73, #86 and #385). In screening and diagnosis a false negative can delay treatment, so it is the most harmful error type. Section 5.9 showed that these patients also receive the least faithful and least consistent explanations. A clinician therefore could not rely on the explanation to catch the error. Table 6.1 audits the system's main failure modes and robustness issues honestly, including those that were not mitigated.

Table 6.1: Safety, robustness and security audit of the prototype

Failure mode / threat	Evidence in this project	Mitigation present	Residual risk / not done
Missed malignancy (FN)	3 FNs; P = 0.160–0.297	Borderline warning; disagreement signals	Threshold not tuned; no calibration
Unfaithful local explanation	Mean LIME R^2 0.27; ≤ 0.06 on FNs	R^2 shown; warning if < 0.3	No automatic suppression of weak explanations
Explanation instability	LIME seed Jaccard as low as 0.00 (#86)	Fixed seed; stability toggle	More samples or averaging not implemented
Invalid / corrupted input	NaN, ∞, text, negatives, missing fields	validate_patient_input (tested)	—
Out-of-distribution input	Value above training maximum	Extrapolation warning (browser-verified)	No detection of in-range shift
Adversarial explanation manipulation [22]	Not tested	None	Possible for perturbation-based explainers
Tampered model file	joblib uses Python pickle	Bundle loaded only from the local project folder	No checksum or signature verification
Unauthorised access	Local single-user app	Runs on localhost only	No authentication or audit log

The pickle point deserves emphasis. Loading a joblib file executes code during unpickling, so a replaced `models/clinxai_rf_bundle.joblib` could run arbitrary code. In this prototype the file is created and read locally by the same user. A deployed version should verify a cryptographic hash before loading, or use a non-executable model format.

6.6 Relevant Ethical and Regulatory Considerations

ClinXAI is an educational prototype that is not placed on any market and is not used clinically. It is therefore **not claimed to comply with, be certified under, or be clinically validated against** any of the frameworks below. Table 6.2 maps their relevant principles to what the prototype demonstrates, to show the gap a real clinical product would have to close.

Table 6.2: Illustrative alignment with ethical and regulatory frameworks (not a compliance claim)

Framework	Relevant principle	What ClinXAI demonstrates	Gap for real clinical use
EU AI Act, Reg. (EU) 2024/1689 [10]	Medical-device AI is high-risk; transparency to deployers (Art. 13), human oversight (Art. 14), accuracy and robustness (Art. 15)	Explanations, disclaimers, warnings, reported accuracy and robustness limits	Risk-management system, technical documentation, conformity assessment, post-market monitoring
WHO guidance, 2021 [11]	Autonomy; safety; transparency and explainability; accountability; inclusiveness; sustainability	Human decision retained; transparent outputs; limits stated	Clinical evaluation; equity assessment

Framework	Relevant principle	What ClinXAI demonstrates	Gap for real clinical use
ICMR AI ethics guidelines, 2023 [28]	Principles including autonomy, data quality, equity, non-discrimination and validity (clinical and field validation before use)	Public de-identified data; documented validation of software behaviour	Ethics-committee approval; clinical and field validation
DPDP Act, 2023 [27]	Lawful, consented, purpose-limited processing of personal data	No personal data processed	Consent, notices, security safeguards for real patient data
ISO/IEC 23894:2023 [29]	AI risk identification, analysis and treatment	Informal risk audit (Table 6.1)	Formal, documented risk-management process
IEEE Ethically Aligned Design [30]	Transparency, accountability, awareness of misuse	Open methods, reproducible evidence, misuse warnings	Independent review

6.7 Alignment with UN Sustainable Development Goal 3

SDG 3, *Good Health and Well-being* [31], includes target 3.4: by 2030, reduce premature mortality from non-communicable diseases, such as cancer, by one third through prevention and treatment. Trustworthy diagnostic support could contribute to earlier and more consistent detection, particularly where specialist cytologists are scarce. ClinXAI's contribution is methodological. It shows how to make a diagnostic model's reasoning inspectable, and it warns that explanation methods themselves can mislead on the hardest cases. The prototype is also light on resources: training takes about 7 seconds and the full evaluation about 36 seconds on a CPU, with no GPU, so the approach is reproducible in low-resource teaching settings. Any real health impact would depend on clinical validation, which has not been done.

CHAPTER 7: CONCLUSION, ENGINEERING LIMITATIONS & FUTURE ENHANCEMENTS

7.1 Summary of Work Completed

ClinXAI delivers a complete, reproducible explainable-diagnostics pipeline for the Breast Cancer Wisconsin (Diagnostic) dataset. A 300-tree Random Forest predicts the probability of malignancy. LIME and TreeSHAP explain every prediction for the same malignant class, and SHAP also provides global importance. A dedicated comparison module quantifies where the explainers agree. A seven-tab Streamlit dashboard makes all of this interactive, including what-if analysis and five programmatically selected case studies. The system is covered by 24 automated tests, an independent validation of every saved output, and a real-browser click-through. All report figures and tables are generated from the actual implementation.

7.2 Achievement of Project Objectives

Table 7.1: Achievement of the project objectives defined in Section 1.3

Objective	Status	Evidence
O1 — Random Forest classifier and baseline	Achieved	Accuracy 97.37%, precision 100%, recall 92.86%, F1 96.30%, ROC-AUC 0.9944; LR baseline on the same split (Table 5.2)
O2 — LIME and TreeSHAP explanations	Achieved	Local explanations for all 114 test patients; global bar and beeswarm; SHAP additivity error 4.4×10^{-16}
O3 — Quantified LIME–SHAP agreement	Achieved	Mean overlap 3.66/5, Jaccard 0.62, Spearman 0.70, direction disagreement in 28.9% of patients; 5 case studies; stability analysis
O4 — Interactive, validated dashboard	Achieved (local)	Seven tabs; 24/24 tests; 34/34 browser checks; runs locally, not publicly deployed

The most important finding is not the model's accuracy, which is high and comparable to a linear baseline. It is that the two most popular post-hoc explanation methods agree on confident predictions and diverge on uncertain ones and on missed cancers. For Patients #86 and #73, LIME's surrogate explained almost none of the forest's local behaviour (R^2 0.03 and 0.06), and its top features overlapped with SHAP's by zero and one feature respectively. Explanation disagreement is therefore a practical warning signal for human review.

7.3 Technical Limitations

- **Local explanation instability.** LIME's top-5 set changes with the random seed (mean pairwise Jaccard 0.39 for Patient #86, with one pair fully disjoint). The fixed seed gives reproducibility but not robustness.
- **Low LIME fidelity.** The mean surrogate R^2 is 0.27 and falls to about 0.03–0.06 on the false negatives. Quartile discretisation and two-decimal rule display (for example `0.00 < fractal dimension error <= 0.00`) further reduce readability.
- **SHAP computation and background assumptions.** The path-dependent TreeSHAP variant uses the trees' training cover instead of an explicit background dataset. Interventional SHAP with a chosen background, or KernelSHAP, could distribute credit among the highly correlated features differently. Those variants were not evaluated.
- **Small dataset.** With 569 patients and a single 114-patient test split, every metric has wide sampling uncertainty. Only 3 false negatives exist to study.
- **Possible distribution shift.** The data comes from one historical source. Performance and explanations may not transfer to other populations or imaging pipelines, and in-range shifts are not detected.
- **No external clinical validation.** No clinician evaluated the explanations, and the model has not been tested on external data. Probabilities are uncalibrated.
- **Tabular only.** The system uses features derived from images. It does not process images or any second clinical data modality.
- **Engineering limits.** The app is single-user and local, without authentication. The first launch takes about 6–8 s. Browser verification covered Microsoft Edge at desktop width only.

7.4 Future Enhancements

- **Genuine multimodal diagnostics.** Combine a convolutional network on the raw cytology images with the tabular nuclear features. Explain the image branch with saliency methods and the tabular branch with SHAP, then compare the explanations across modalities.
- **Explanation quality controls.** Suppress or flag LIME explanations below a fidelity threshold, average LIME over several seeds or larger samples, and group correlated features before attribution.

- **Alternative SHAP formulations.** Compare path-dependent TreeSHAP with interventional TreeSHAP and KernelSHAP using an explicit background dataset.
- **Calibration and threshold selection.** Calibrate probabilities [26] and choose a recall-oriented threshold on separate validation data, reporting the false-positive cost.
- **External validation and distribution-shift monitoring** on independent datasets.
- **Medical expert validation before any deployment.** Run a structured study with pathologists and oncologists to test whether the explanations are understandable, correct and useful. Without this, the system must remain an educational tool.
- **Deployment hardening.** Add model-file integrity checks, authentication, audit logging and a documented risk-management process.

REFERENCES

[1] World Health Organization, "Breast cancer," WHO fact sheet. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/breast-cancer

[2] W. N. Street, W. H. Wolberg, and O. L. Mangasarian, "Nuclear feature extraction for breast tumor diagnosis," in *Proc. SPIE 1905, Biomedical Image Processing and Biomedical Visualization*, San Jose, CA, USA, 1993, pp. 861–870, doi: 10.1117/12.148698.

[3] W. Wolberg, O. Mangasarian, N. Street, and W. Street, "Breast Cancer Wisconsin (Diagnostic)," UCI Machine Learning Repository, 1993, doi: 10.24432/C5DW2B.

[4] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[5] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Machine Intelligence*, vol. 1, no. 5, pp. 206–215, 2019, doi: 10.1038/s42256-019-0048-x.

[6] A. Holzinger, C. Biemann, C. S. Pattichis, and D. B. Kell, "What do we need to build explainable AI systems for the medical domain?," 2017, arXiv:1712.09923.

[7] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, San Francisco, CA, USA, 2016, pp. 1135–1144, doi: 10.1145/2939672.2939778.

[8] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, Long Beach, CA, USA, 2017, pp. 4765–4774.

[9] S. Krishna, T. Han, A. Gu, J. Pombra, S. Jabbari, S. Wu, and H. Lakkaraju, "The disagreement problem in explainable machine learning: A practitioner's perspective," 2022, arXiv:2202.01602.

[10] European Parliament and Council of the European Union, "Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)," *Official Journal of the European Union*, L series, 12 Jul. 2024.

[11] World Health Organization, *Ethics and Governance of Artificial Intelligence for Health: WHO Guidance*. Geneva, Switzerland: WHO, 2021, ISBN: 978-92-4-002920-0.

[12] A. Adadi and M. Berrada, "Peeking inside the black-box: A survey on explainable artificial intelligence (XAI)," *IEEE Access*, vol. 6, pp. 52138–52160, 2018, doi: 10.1109/ACCESS.2018.2870052.

[13] A. Barredo Arrieta *et al.*, "Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI," *Information Fusion*, vol. 58, pp. 82–115, 2020, doi: 10.1016/j.inffus.2019.12.012.

[14] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," 2017, arXiv:1702.08608.

[15] Z. C. Lipton, "The mythos of model interpretability," *Commun. ACM*, vol. 61, no. 10, pp. 36–43, 2018, doi: 10.1145/3233231.

[16] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[17] M. T. Ribeiro, "LIME: Explaining the predictions of any machine learning classifier," GitHub repository. [Online]. Available: https://github.com/marcotcr/lime

[18] D. Alvarez-Melis and T. S. Jaakkola, "On the robustness of interpretability methods," presented at the ICML Workshop on Human Interpretability in Machine Learning, Stockholm, Sweden, 2018, arXiv:1806.08049.

[19] L. S. Shapley, "A value for n-person games," in *Contributions to the Theory of Games*, vol. II, H. W. Kuhn and A. W. Tucker, Eds. Princeton, NJ, USA: Princeton Univ. Press, 1953, pp. 307–317.

[20] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020, doi: 10.1038/s42256-019-0138-9.

[21] M. T. Ribeiro, S. Singh, and C. Guestrin, "Anchors: High-precision model-agnostic explanations," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 32, no. 1, New Orleans, LA, USA, 2018, pp. 1527–1535, doi: 10.1609/aaai.v32i1.11491.

[22] D. Slack, S. Hilgard, E. Jia, S. Singh, and H. Lakkaraju, "Fooling LIME and SHAP: Adversarial attacks on post hoc explanation methods," in *Proc. AAAI/ACM Conf. AI, Ethics, and Society (AIES)*, New York, NY, USA, 2020, pp. 180–186, doi: 10.1145/3375627.3375830.

[23] C. Molnar, *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*, 2nd ed., 2022. [Online]. Available: https://christophm.github.io/interpretable-ml-book/

[24] Snowflake Inc., "Streamlit documentation." [Online]. Available: https://docs.streamlit.io

[25] S. M. Lundberg *et al.*, "SHAP (SHapley Additive exPlanations) documentation." [Online]. Available: https://shap.readthedocs.io

[26] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proc. 22nd Int. Conf. Machine Learning (ICML)*, Bonn, Germany, 2005, pp. 625–632, doi: 10.1145/1102351.1102430.

[27] Government of India, "The Digital Personal Data Protection Act, 2023," Act No. 22 of 2023, Ministry of Law and Justice, New Delhi, India, Aug. 2023.

[28] Indian Council of Medical Research, *Ethical Guidelines for Application of Artificial Intelligence in Biomedical Research and Healthcare*. New Delhi, India: ICMR, 2023.

[29] ISO/IEC 23894:2023, *Information technology — Artificial intelligence — Guidance on risk management*, International Organization for Standardization, Geneva, Switzerland, 2023.

[30] The IEEE Global Initiative on Ethics of Autonomous and Intelligent Systems, *Ethically Aligned Design: A Vision for Prioritizing Human Well-being with Autonomous and Intelligent Systems*, 1st ed. IEEE, 2019.

[31] United Nations General Assembly, "Transforming our world: The 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, Oct. 2015.

APPENDIX A: COMPLETE SOURCE CODE LISTINGS / CORE CONFIGURATION FILES

This appendix reproduces every source and configuration file of the ClinXAI application exactly as submitted, read directly from the project folder when the report was generated. Code is set in Consolas 9 pt with single line spacing on a light-grey background. Lines longer than the page width wrap with a hanging indent.

A.1 requirements.txt — Pinned dependencies (11 lines)

```
# Tested with Python 3.12.10 on Windows 11 (exact versions verified by the test suite)
numpy==2.5.4
pandas==3.1.0
scipy==1.18.1
scikit-learn==1.9.1
joblib==1.6.0
matplotlib==3.11.2
lime==0.2.0.1
shap==0.54.1
streamlit==1.66.1
pytest==9.1.1
```

A.2 .streamlit/config.toml — Streamlit theme and server configuration (13 lines)

```
[theme]
base = "light"
primaryColor = "#0B7A75"
backgroundColor = "#FFFFFF"
secondaryBackgroundColor = "#EEF4F7"
textColor = "#1F2A37"
font = "sans serif"

[server]
headless = true

[browser]
gatherUsageStats = false
```

A.3 src/__init__.py — Package marker (3 lines)

```
"""ClinXAI: Transparent clinical diagnostics using a Random Forest explained with LIME and
    SHAP."""

__version__ = "1.0.0"
```

A.4 src/config.py — Central configuration (41 lines)

```
"""Central configuration: paths, seeds and class conventions used across ClinXAI."""
from pathlib import Path

ROOT_DIR = Path(__file__).resolve().parent.parent
MODELS_DIR = ROOT_DIR / "models"
```

```
OUTPUTS_DIR = ROOT_DIR / "outputs"
FIGURES_DIR = OUTPUTS_DIR / "figures"
METRICS_DIR = OUTPUTS_DIR / "metrics"
CASE_STUDIES_DIR = OUTPUTS_DIR / "case_studies"

MODEL_PATH = MODELS_DIR / "clinxai_rf_bundle.joblib"

RANDOM_STATE = 42
TEST_SIZE = 0.20
DECISION_THRESHOLD = 0.5

# Application-wide class convention (sklearn's raw dataset uses malignant=0, benign=1).
# ClinXAI re-encodes the target so that the positive clinical-risk class is malignant.
POSITIVE_CLASS = 1
CLASS_NAMES = ["Benign", "Malignant"]  # index == encoded label
MALIGNANT_LABEL = 1
BENIGN_LABEL = 0

RF_PARAMS = {
    "n_estimators": 300,
    "max_depth": None,
    "min_samples_leaf": 1,
    "class_weight": None,
    "random_state": RANDOM_STATE,
    "n_jobs": -1,
}

LIME_NUM_SAMPLES = 5000
LIME_NUM_FEATURES = 10
LIME_STABILITY_SEEDS = (0, 1, 2, 3, 4)
TOP_K = 5


def ensure_dirs() -> None:
    for d in (MODELS_DIR, FIGURES_DIR, METRICS_DIR, CASE_STUDIES_DIR):
        d.mkdir(parents=True, exist_ok=True)
```

A.5 src/data_loader.py — Data loading, mapping, validation and split (124 lines)

```
"""Dataset loading, label re-encoding, validation, splitting and patient-input
    validation."""
from __future__ import annotations

from dataclasses import dataclass

import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

from src.config import BENIGN_LABEL, MALIGNANT_LABEL, RANDOM_STATE, TEST_SIZE

EXPECTED_N_SAMPLES = 569
EXPECTED_N_FEATURES = 30


@dataclass
class DataSplit:
    X_train: pd.DataFrame
    X_test: pd.DataFrame
    y_train: pd.Series
    y_test: pd.Series
    feature_names: list[str]
```

```
def load_dataset() -> tuple[pd.DataFrame, pd.Series, list[str]]:
    """Load Breast Cancer Wisconsin (Diagnostic) and re-encode so malignant = 1, benign =
    0.

    sklearn ships the target as 0 = malignant, 1 = benign. The mapping is derived from
    ``target_names`` rather than hard-coded, then asserted.
    """
    raw = load_breast_cancer(as_frame=True)
    X = raw.data.copy()
    raw_target = raw.target.to_numpy()
    target_names = list(raw.target_names)
    raw_malignant = target_names.index("malignant")
    raw_benign = target_names.index("benign")

    y = pd.Series(
        np.where(raw_target == raw_malignant, MALIGNANT_LABEL, BENIGN_LABEL),
        index=X.index,
        name="diagnosis_malignant",
    )
    # Verify the re-encoding against sklearn's own labels.
    assert np.all((raw_target == raw_malignant) == (y.to_numpy() == MALIGNANT_LABEL))
    assert np.all((raw_target == raw_benign) == (y.to_numpy() == BENIGN_LABEL))

    validate_dataset(X, y)
    return X, y, list(X.columns)


def validate_dataset(X: pd.DataFrame, y: pd.Series) -> None:
    if X.shape != (EXPECTED_N_SAMPLES, EXPECTED_N_FEATURES):
        raise ValueError(f"Unexpected dataset shape {X.shape}")
    if X.isna().any().any():
        raise ValueError("Dataset contains missing values")
    if not np.isfinite(X.to_numpy()).all():
        raise ValueError("Dataset contains non-finite values")
    if set(np.unique(y)) != {BENIGN_LABEL, MALIGNANT_LABEL}:
        raise ValueError("Target must be binary {0, 1}")
    if len(set(X.columns)) != X.shape[1]:
        raise ValueError("Duplicate feature names")


def split_data(X: pd.DataFrame, y: pd.Series) -> DataSplit:
    """Stratified 80/20 split. All fitting (model, LIME statistics) uses the training part
    only."""
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=TEST_SIZE, stratify=y, random_state=RANDOM_STATE
    )
    return DataSplit(X_train, X_test, y_train, y_test, list(X.columns))


def dataset_overview(X: pd.DataFrame, y: pd.Series) -> pd.DataFrame:
    """Per-feature summary table for the report."""
    desc = X.describe().T[["mean", "std", "min", "max"]]
    desc["mean_malignant"] = X[y == MALIGNANT_LABEL].mean()
    desc["mean_benign"] = X[y == BENIGN_LABEL].mean()
    desc.index.name = "feature"
    return desc.round(4)


def validate_patient_input(values, feature_names: list[str]) -> pd.DataFrame:
    """Validate a single patient's biomarker values and return a correctly ordered 1-row
    frame.

    Accepts a dict (feature -> value), a pandas Series, or a 1-D sequence in training
    order.
    Raises ValueError for missing/extra features, non-numeric, non-finite or negative
    values
    (all 30 features are non-negative physical measurements).
    """
```

```
    if isinstance(values, pd.DataFrame):
        if len(values) != 1:
            raise ValueError("Exactly one patient row is required")
        values = values.iloc[0]
    if isinstance(values, (dict, pd.Series)):
        keys = list(values.keys())
        missing = [f for f in feature_names if f not in keys]
        extra = [k for k in keys if k not in feature_names]
        if missing:
            raise ValueError(f"Missing features: {missing}")
        if extra:
            raise ValueError(f"Unknown features: {extra}")
        ordered = [values[f] for f in feature_names]
    else:
        ordered = list(values)
        if len(ordered) != len(feature_names):
            raise ValueError(f"Expected {len(feature_names)} values, got {len(ordered)}")

    try:
        arr = np.asarray(ordered, dtype=float)
    except (TypeError, ValueError) as exc:
        raise ValueError(f"All biomarker values must be numeric: {exc}") from exc
    if not np.isfinite(arr).all():
        bad = [f for f, v in zip(feature_names, arr) if not np.isfinite(v)]
        raise ValueError(f"Non-finite values for: {bad}")
    if (arr < 0).any():
        bad = [f for f, v in zip(feature_names, arr) if v < 0]
        raise ValueError(f"Negative values are not physically valid for: {bad}")
    return pd.DataFrame([arr], columns=feature_names)


def out_of_range_features(row: pd.DataFrame, X_train: pd.DataFrame) -> list[str]:
    """Features whose value lies outside the training range (extrapolation warning)."""
    lo, hi = X_train.min(), X_train.max()
    r = row.iloc[0]
    return [f for f in X_train.columns if r[f] < lo[f] or r[f] > hi[f]]
```

A.6 src/model.py — Model training, evaluation and persistence (109 lines)

```
"""Model training, evaluation, persistence and prediction helpers."""
from __future__ import annotations

import joblib
import numpy as np
import pandas as pd
import sklearn
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
    roc_curve,
)
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

from src.config import (
    CLASS_NAMES,
    DECISION_THRESHOLD,
    MALIGNANT_LABEL,
```

```
    MODEL_PATH,
    RANDOM_STATE,
    RF_PARAMS,
)
from src.data_loader import DataSplit


def train_random_forest(X_train: pd.DataFrame, y_train: pd.Series) ->
    RandomForestClassifier:
    model = RandomForestClassifier(**RF_PARAMS)
    model.fit(X_train, y_train)
    assert list(model.classes_) == [0, 1], "Expected classes [0=benign, 1=malignant]"
    return model


def train_logistic_baseline(X_train: pd.DataFrame, y_train: pd.Series):
    """Interpretable baseline. Scaling lives inside the pipeline, so it is fit on training
    data only."""
    pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000,
    random_state=RANDOM_STATE))
    pipe.fit(X_train, y_train)
    return pipe


def malignant_proba(model, X: pd.DataFrame) -> np.ndarray:
    """P(malignant) using the column that corresponds to label 1, not a positional
    guess."""
    col = list(model.classes_).index(MALIGNANT_LABEL)
    return model.predict_proba(X)[:, col]


def evaluate_classifier(model, X_test: pd.DataFrame, y_test: pd.Series) -> dict:
    proba = malignant_proba(model, X_test)
    pred = (proba >= DECISION_THRESHOLD).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, pred, labels=[0, 1]).ravel()
    return {
        "positive_class": f"{CLASS_NAMES[MALIGNANT_LABEL]} (label {MALIGNANT_LABEL})",
        "decision_threshold": DECISION_THRESHOLD,
        "accuracy": float(accuracy_score(y_test, pred)),
        "precision": float(precision_score(y_test, pred, pos_label=MALIGNANT_LABEL)),
        "recall_sensitivity": float(recall_score(y_test, pred, pos_label=MALIGNANT_LABEL)),
        "specificity": float(tn / (tn + fp)),
        "f1": float(f1_score(y_test, pred, pos_label=MALIGNANT_LABEL)),
        "roc_auc": float(roc_auc_score(y_test, proba)),
        "confusion_matrix": {"tn": int(tn), "fp": int(fp), "fn": int(fn), "tp": int(tp)},
    }


def cross_validated_auc(model_factory, X_train: pd.DataFrame, y_train: pd.Series) -> dict:
    """5-fold stratified CV on the training set only (no test data involved)."""
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
    scores = cross_val_score(model_factory(), X_train, y_train, cv=cv, scoring="roc_auc")
    return {"cv_roc_auc_mean": float(scores.mean()), "cv_roc_auc_std": float(scores.std())}


def roc_points(model, X_test, y_test):
    fpr, tpr, _ = roc_curve(y_test, malignant_proba(model, X_test))
    return fpr, tpr


def save_bundle(model, baseline, split: DataSplit, metrics: dict, path=MODEL_PATH) -> None:
    """Persist the model together with everything needed to reproduce explanations."""
    bundle = {
        "model": model,
        "baseline": baseline,
        "feature_names": split.feature_names,
        "class_names": CLASS_NAMES,
        "positive_class": MALIGNANT_LABEL,
        "X_train": split.X_train,
```

```
        "y_train": split.y_train,
        "X_test": split.X_test,
        "y_test": split.y_test,
        "metrics": metrics,
        "versions": {"sklearn": sklearn.__version__},
    }
    path.parent.mkdir(parents=True, exist_ok=True)
    joblib.dump(bundle, path)


def load_bundle(path=MODEL_PATH) -> dict:
    if not path.exists():
        raise FileNotFoundError(f"Model bundle not found at {path}. Run `python
    train_model.py` first.")
    bundle = joblib.load(path)
    model = bundle["model"]
    if list(model.feature_names_in_) != bundle["feature_names"]:
        raise ValueError("Feature ordering mismatch between model and saved metadata")
    return bundle
```

A.7 src/explain_lime.py — LIME explanations (94 lines)

```
"""LIME local explanations for the malignant-class probability."""
from __future__ import annotations

from dataclasses import dataclass

import numpy as np
import pandas as pd
from lime.lime_tabular import LimeTabularExplainer

from src.config import CLASS_NAMES, LIME_NUM_SAMPLES, MALIGNANT_LABEL, RANDOM_STATE


@dataclass
class LimeResult:
    table: pd.DataFrame          # feature, rule, weight, abs_weight, rank (sorted by
    |weight|)
    weights: pd.Series           # weight per feature, indexed by feature name (training
    order)
    intercept: float             # surrogate intercept for the malignant class
    local_pred: float            # surrogate's prediction at the instance
    score: float                 # R^2 of the weighted surrogate fit (local fidelity)
    model_proba: float           # Random Forest P(malignant) for the instance
    seed: int


def build_lime_explainer(X_train: pd.DataFrame, seed: int = RANDOM_STATE) ->
    LimeTabularExplainer:
    """Fit LIME's sampling/discretisation statistics on TRAINING data only."""
    return LimeTabularExplainer(
        training_data=X_train.to_numpy(),
        feature_names=list(X_train.columns),
        class_names=CLASS_NAMES,
        mode="classification",
        discretize_continuous=True,
        discretizer="quartile",
        random_state=seed,
    )


def make_predict_fn(model, feature_names: list[str]):
    """LIME passes bare numpy arrays; rebuild a DataFrame so column order/name match
    training."""
    assert list(model.classes_) == [0, 1]
```

```
    def predict_fn(arr: np.ndarray) -> np.ndarray:
        return model.predict_proba(pd.DataFrame(arr, columns=feature_names))

    return predict_fn


def explain_patient_lime(
    model,
    X_train: pd.DataFrame,
    patient: pd.DataFrame | pd.Series,
    seed: int = RANDOM_STATE,
    num_samples: int = LIME_NUM_SAMPLES,
) -> LimeResult:
    """Explain P(malignant) for one patient.

    A fresh explainer is created per call with a fixed seed so the explanation is exactly
    reproducible (re-using one explainer would advance its RNG between calls).
    All features are requested so LIME and SHAP can be compared over the full feature set.
    """
    feature_names = list(X_train.columns)
    row = patient.iloc[0] if isinstance(patient, pd.DataFrame) else patient
    x = row[feature_names].to_numpy(dtype=float)

    explainer = build_lime_explainer(X_train, seed)
    exp = explainer.explain_instance(
        x,
        make_predict_fn(model, feature_names),
        labels=(MALIGNANT_LABEL,),
        num_features=len(feature_names),
        num_samples=num_samples,
    )
    idx_weights = exp.as_map()[MALIGNANT_LABEL]
    rules = exp.as_list(label=MALIGNANT_LABEL)  # same ordering as as_map
    records = [
        {"feature": feature_names[i], "rule": rule, "weight": float(w)}
        for (i, w), (rule, _) in zip(idx_weights, rules)
    ]
    table = pd.DataFrame(records)
    table["abs_weight"] = table["weight"].abs()
    table = table.sort_values("abs_weight", ascending=False).reset_index(drop=True)
    table["rank"] = np.arange(1, len(table) + 1)
    table["direction"] = np.where(table["weight"] > 0, "→ Malignant", "→ Benign")

    weights = table.set_index("feature")["weight"].reindex(feature_names).fillna(0.0)
    model_proba = float(model.predict_proba(pd.DataFrame([x], columns=feature_names))[0,
    MALIGNANT_LABEL])
    return LimeResult(
        table=table,
        weights=weights,
        intercept=float(exp.intercept[MALIGNANT_LABEL]),
        local_pred=float(np.ravel(exp.local_pred)[0]),
        score=float(exp.score),
        model_proba=model_proba,
        seed=seed,
    )
```

A.8 src/explain_shap.py — SHAP explanations (80 lines)

```
"""SHAP TreeExplainer attributions for the malignant-class probability."""
from __future__ import annotations

import numpy as np
import pandas as pd
import shap

from src.config import MALIGNANT_LABEL
```

```
def build_tree_explainer(model) -> shap.TreeExplainer:
    """TreeExplainer in its default 'tree_path_dependent' mode.

    No background dataset is needed: the expected value is taken from the training-sample
    coverage stored in each tree. For a scikit-learn Random Forest the trees' raw output is
    the class-probability average, so SHAP values, the base value and predict_proba all
    share
    probability units and satisfy base + sum(SHAP) == P(malignant).
    """
    return shap.TreeExplainer(model)


def _select_malignant(values, expected_value) -> tuple[np.ndarray, float]:
    """Normalise SHAP's version-dependent return formats to (n, n_features) for class 1."""
    ev = np.atleast_1d(np.asarray(expected_value, dtype=float))
    if isinstance(values, list):  # older SHAP: list of per-class arrays
        vals = np.asarray(values[MALIGNANT_LABEL])
        base = ev[MALIGNANT_LABEL]
    else:
        vals = np.asarray(values)
        if vals.ndim == 3:  # SHAP >= 0.45: (n_samples, n_features, n_classes)
            vals = vals[:, :, MALIGNANT_LABEL]
            base = ev[MALIGNANT_LABEL]
        elif vals.ndim == 2:  # single-output model
            base = ev[-1] if ev.size > 1 else ev[0]
        else:
            raise ValueError(f"Unexpected SHAP value shape {vals.shape}")
    return vals, float(base)


def compute_shap(explainer: shap.TreeExplainer, X: pd.DataFrame) -> shap.Explanation:
    """Return a 2-D shap.Explanation (malignant class) for the rows of X."""
    raw = explainer.shap_values(X)
    vals, base = _select_malignant(raw, explainer.expected_value)
    return shap.Explanation(
        values=vals,
        base_values=np.full(vals.shape[0], base),
        data=X.to_numpy(),
        feature_names=list(X.columns),
    )


def additivity_error(model, explanation: shap.Explanation, X: pd.DataFrame) -> float:
    """Max |base + sum(SHAP) - P(malignant)| over the rows of X."""
    reconstructed = explanation.base_values + explanation.values.sum(axis=1)
    proba = model.predict_proba(X)[:, MALIGNANT_LABEL]
    return float(np.max(np.abs(reconstructed - proba)))


def shap_table(explanation_row: shap.Explanation) -> pd.DataFrame:
    """Per-feature contribution table for a single patient, sorted by |SHAP|."""
    df = pd.DataFrame(
        {
            "feature": explanation_row.feature_names,
            "value": explanation_row.data,
            "shap_value": explanation_row.values,
        }
    )
    df["abs_shap"] = df["shap_value"].abs()
    df = df.sort_values("abs_shap", ascending=False).reset_index(drop=True)
    df["rank"] = np.arange(1, len(df) + 1)
    df["direction"] = np.where(df["shap_value"] > 0, "→ Malignant", "→ Benign")
    return df


def global_importance(explanation: shap.Explanation) -> pd.Series:
    """Mean |SHAP| per feature over the explained set (global importance)."""
    return (
```

```
        pd.Series(np.abs(explanation.values).mean(axis=0), index=explanation.feature_names)
        .sort_values(ascending=False)
    )
```

A.9 src/comparison.py — LIME–SHAP comparison and case selection (125 lines)

```
"""LIME vs SHAP comparison measures and programmatic case-study selection.

These are measures of agreement *between explanation methods*; they say nothing about
    clinical
validity. LIME weights are coefficients of a local weighted linear surrogate over
    discretised
feature bins, while SHAP values are additive Shapley attributions in probability units, so
    only
ranks and signs are compared - never raw magnitudes.
"""
from __future__ import annotations

from itertools import combinations

import numpy as np
import pandas as pd
from scipy.stats import spearmanr

from src.config import LIME_STABILITY_SEEDS, TOP_K
from src.explain_lime import explain_patient_lime


def top_k(weights: pd.Series, k: int = TOP_K) -> list[str]:
    return list(weights.abs().sort_values(ascending=False, kind="mergesort").index[:k])


def jaccard(a, b) -> float:
    a, b = set(a), set(b)
    return len(a & b) / len(a | b) if a | b else 1.0


def compare_explanations(lime_weights: pd.Series, shap_values: pd.Series, k: int = TOP_K) -
    > dict:
    """Compare two attribution vectors indexed by the same feature names (same patient,
    class)."""
    lime_weights = lime_weights.reindex(shap_values.index).fillna(0.0)
    lime_top, shap_top = top_k(lime_weights, k), top_k(shap_values, k)
    shared = [f for f in lime_top if f in shap_top]
    union = list(dict.fromkeys(lime_top + shap_top))

    rho, _ = spearmanr(lime_weights.abs().to_numpy(), shap_values.abs().to_numpy())

    direction_rows = []
    for f in union:
        lw, sv = float(lime_weights[f]), float(shap_values[f])
        direction_rows.append(
            {
                "feature": f,
                "lime_weight": lw,
                "shap_value": sv,
                "lime_rank": int(lime_weights.abs().rank(ascending=False,
    method="first")[f]),
                "shap_rank": int(shap_values.abs().rank(ascending=False,
    method="first")[f]),
                "in_lime_top_k": f in lime_top,
                "in_shap_top_k": f in shap_top,
                "same_direction": bool(np.sign(lw) == np.sign(sv)),
            }
```

```
        )
    direction_df = pd.DataFrame(direction_rows)
    disagreements = direction_df.loc[~direction_df["same_direction"], "feature"].tolist()
    shared_df = direction_df[direction_df["feature"].isin(shared)]
    return {
        "k": k,
        "lime_top_k": lime_top,
        "shap_top_k": shap_top,
        "shared_top_k": shared,
        "top_k_overlap": len(shared),
        "jaccard_top_k": jaccard(lime_top, shap_top),
        "spearman_abs_all_features": float(rho) if np.isfinite(rho) else float("nan"),
        "direction_agreement_shared": float(shared_df["same_direction"].mean()) if
    len(shared_df) else float("nan"),
        "direction_disagreements_union": disagreements,
        "direction_table": direction_df,
    }


def lime_stability(model, X_train: pd.DataFrame, patient, seeds=LIME_STABILITY_SEEDS, k:
    int = TOP_K,
                   num_samples: int | None = None) -> dict:
    """Re-run LIME with different random seeds and measure how much the top-k set
    changes."""
    kwargs = {"num_samples": num_samples} if num_samples else {}
    runs = [explain_patient_lime(model, X_train, patient, seed=s, **kwargs) for s in seeds]
    tops = [top_k(r.weights, k) for r in runs]
    pair_j = [jaccard(a, b) for a, b in combinations(tops, 2)]
    W = pd.DataFrame({s: r.weights for s, r in zip(seeds, runs)})
    sign_flip = (np.sign(W).nunique(axis=1) > 1)
    return {
        "seeds": list(seeds),
        "mean_pairwise_jaccard_top_k": float(np.mean(pair_j)),
        "min_pairwise_jaccard_top_k": float(np.min(pair_j)),
        "features_with_sign_flip": int(sign_flip.sum()),
        "weight_mean": W.mean(axis=1),
        "weight_std": W.std(axis=1),
    }


def select_case_studies(summary: pd.DataFrame) -> list[dict]:
    """Pick cases from the per-patient test summary.

    `summary` needs columns: patient_id, p_malignant, jaccard_top_k,
    spearman_abs_all_features,
    n_direction_disagreements. Ties are broken by patient_id for reproducibility.
    """
    s = summary.sort_values("patient_id").reset_index(drop=True)
    cases, used = [], set()

    def add(case_type, row, criterion):
        used.add(int(row["patient_id"]))
        cases.append({"case_type": case_type, "patient_id": int(row["patient_id"]),
    "criterion": criterion,
                      "found": True})

    def avail():
        return s[~s["patient_id"].isin(used)]

    add("Clearly malignant", avail().sort_values("p_malignant", ascending=False,
    kind="mergesort").iloc[0],
        "Highest predicted P(malignant) on the held-out test set")
    add("Clearly benign", avail().sort_values("p_malignant", ascending=True,
    kind="mergesort").iloc[0],
        "Lowest predicted P(malignant) on the held-out test set")
    border = avail().assign(dist=lambda d: (d["p_malignant"] - 0.5).abs())
    add("Borderline", border.sort_values("dist", kind="mergesort").iloc[0],
        "Predicted P(malignant) closest to the 0.5 decision threshold")
```

```
    div = avail().sort_values(["jaccard_top_k", "spearman_abs_all_features"],
    kind="mergesort").iloc[0]
    add("Ranking divergence", div, "Lowest top-5 Jaccard overlap between LIME and SHAP
    (ties: lowest Spearman)")

    dirs = avail().sort_values(["n_direction_disagreements", "jaccard_top_k"],
    ascending=[False, True],
                               kind="mergesort")
    if len(dirs) and dirs.iloc[0]["n_direction_disagreements"] > 0:
        add("Direction disagreement", dirs.iloc[0],
            "Most features (within the union of both top-5 lists) with opposite LIME/SHAP
    sign")
    else:
        cases.append({"case_type": "Direction disagreement", "patient_id": None, "found":
    False,
                      "criterion": "No test patient had a LIME/SHAP sign disagreement
    within the top-5 union"})
    return cases
```

A.10 src/visualization.py — Figures (298 lines)

```
"""Matplotlib figures for ClinXAI. Every function returns a Figure and never calls
    plt.show()."""
from __future__ import annotations

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt  # noqa: E402
import numpy as np  # noqa: E402
import pandas as pd  # noqa: E402
import shap  # noqa: E402
from matplotlib.patches import FancyBboxPatch  # noqa: E402

# Shared palette: red = pushes toward malignant, blue = pushes toward benign (matches
    SHAP).
MALIGNANT_COLOR = "#C8102E"
BENIGN_COLOR = "#1E6FB8"
ACCENT = "#0B7A75"
INK = "#1F2A37"
GRID = "#E5E9F0"

plt.rcParams.update(
    {
        "font.family": "DejaVu Sans",
        "font.size": 10,
        "axes.edgecolor": "#9AA5B1",
        "axes.labelcolor": INK,
        "axes.titleweight": "bold",
        "axes.titlesize": 12,
        "xtick.color": INK,
        "ytick.color": INK,
        "figure.dpi": 110,
        "savefig.bbox": "tight",
        "savefig.dpi": 150,
    }
)


def _style(ax):
    ax.spines[["top", "right"]].set_visible(False)
    ax.grid(axis="x", color=GRID, linewidth=0.8)
    ax.set_axisbelow(True)
```

```
def save(fig, path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    fig.savefig(path, bbox_inches="tight", facecolor="white")


# ---------------------------------------------------------------- evaluation
def plot_confusion_matrix(cm: dict, title: str = "Confusion matrix (test set)"):
    mat = np.array([[cm["tn"], cm["fp"]], [cm["fn"], cm["tp"]]])
    fig, ax = plt.subplots(figsize=(4.6, 4))
    ax.imshow(mat, cmap="Blues")
    labels = [["True negative", "False positive"], ["False negative", "True positive"]]
    for i in range(2):
        for j in range(2):
            color = "white" if mat[i, j] > mat.max() / 2 else INK
            ax.text(j, i, f"{mat[i, j]}\n{labels[i][j]}", ha="center", va="center",
    color=color, fontsize=10)
    ax.set_xticks([0, 1], ["Benign (0)", "Malignant (1)"])
    ax.set_yticks([0, 1], ["Benign (0)", "Malignant (1)"])
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")
    ax.set_title(title)
    fig.tight_layout()
    return fig


def plot_roc(curves: dict, title: str = "ROC curve — positive class: Malignant"):
    """curves: name -> (fpr, tpr, auc)."""
    fig, ax = plt.subplots(figsize=(5, 4.3))
    colors = [ACCENT, "#8E6C8A", "#D98E04"]
    for (name, (fpr, tpr, auc)), c in zip(curves.items(), colors):
        ax.plot(fpr, tpr, color=c, lw=2, label=f"{name} (AUC = {auc:.3f})")
    ax.plot([0, 1], [0, 1], ls="--", color="#9AA5B1", lw=1, label="Chance")
    ax.set_xlabel("False positive rate (1 – specificity)")
    ax.set_ylabel("True positive rate (sensitivity)")
    ax.set_title(title)
    ax.legend(loc="lower right", frameon=False)
    ax.spines[["top", "right"]].set_visible(False)
    fig.tight_layout()
    return fig


def plot_risk_gauge(p: float, threshold: float = 0.5):
    """Horizontal probability bar with the decision threshold marked."""
    fig, ax = plt.subplots(figsize=(7, 1.35))
    grad = np.linspace(0, 1, 256).reshape(1, -1)
    cmap = matplotlib.colors.LinearSegmentedColormap.from_list("risk", [BENIGN_COLOR,
    "#F2F4F7", MALIGNANT_COLOR])
    ax.imshow(grad, extent=[0, 1, 0, 1], aspect="auto", cmap=cmap, alpha=0.85)
    ax.axvline(threshold, color=INK, ls="--", lw=1)
    ax.text(threshold, 1.08, f"threshold {threshold:.2f}", ha="center", va="bottom",
    fontsize=8, color=INK)
    ax.plot([p], [0.5], marker="v", markersize=16, color=INK, transform=ax.transData)
    ax.text(p, -0.12, f"P(malignant) = {p:.3f}", ha="center", va="top", fontsize=10,
    fontweight="bold", color=INK)
    ax.set_xlim(0, 1)
    ax.set_ylim(-0.5, 1.4)
    ax.set_yticks([])
    ax.set_xticks([0, 0.25, 0.5, 0.75, 1.0])
    ax.text(0.0, 1.08, "Benign", ha="left", va="bottom", fontsize=9, color=BENIGN_COLOR,
    fontweight="bold")
    ax.text(1.0, 1.08, "Malignant", ha="right", va="bottom", fontsize=9,
    color=MALIGNANT_COLOR, fontweight="bold")
    for s in ax.spines.values():
        s.set_visible(False)
    fig.tight_layout()
    return fig


# ---------------------------------------------------------------- LIME
```

```
def plot_lime(lime_table: pd.DataFrame, n: int = 10, title: str = "LIME local explanation —
    P(malignant)"):
    d = lime_table.head(n).iloc[::-1]
    fig, ax = plt.subplots(figsize=(8, 0.42 * len(d) + 1.2))
    colors = [MALIGNANT_COLOR if w > 0 else BENIGN_COLOR for w in d["weight"]]
    ax.barh(d["rule"], d["weight"], color=colors)
    ax.axvline(0, color=INK, lw=0.8)
    ax.set_xlabel("Surrogate weight (+ toward malignant, − toward benign)")
    ax.set_title(title)
    _style(ax)
    fig.tight_layout()
    return fig


# ---------------------------------------------------------------- SHAP
def plot_shap_global_bar(importance: pd.Series, n: int = 15,
                         title: str = "Global feature importance — mean |SHAP| (test
    set)"):
    d = importance.head(n).iloc[::-1]
    fig, ax = plt.subplots(figsize=(7.5, 0.36 * len(d) + 1.2))
    ax.barh(d.index, d.values, color=ACCENT)
    ax.set_xlabel("mean |SHAP value| (change in P(malignant))")
    ax.set_title(title)
    _style(ax)
    fig.tight_layout()
    return fig


def plot_shap_beeswarm(explanation: shap.Explanation, max_display: int = 15):
    plt.close("all")
    try:
        shap.plots.beeswarm(explanation, max_display=max_display, show=False)
    except Exception:  # fallback for SHAP versions without the Explanation API
        shap.summary_plot(explanation.values, explanation.data,
    feature_names=explanation.feature_names,
                          max_display=max_display, show=False)
    fig = plt.gcf()
    fig.set_size_inches(8, 0.38 * max_display + 1.6)
    plt.title("SHAP summary (beeswarm) — impact on P(malignant)", fontweight="bold")
    fig.tight_layout()
    return fig


def plot_shap_waterfall(explanation_row: shap.Explanation, max_display: int = 12):
    plt.close("all")
    try:
        shap.plots.waterfall(explanation_row, max_display=max_display, show=False)
        fig = plt.gcf()
    except Exception:  # fallback: manual waterfall
        fig = _manual_waterfall(explanation_row, max_display)
    fig.set_size_inches(8.5, 0.42 * max_display + 1.8)
    fig.suptitle("SHAP waterfall — this patient (units: probability of malignant)",
    fontweight="bold", y=1.0)
    fig.tight_layout()
    return fig


def _manual_waterfall(row: shap.Explanation, max_display: int):
    vals = pd.Series(row.values, index=row.feature_names)
    order = vals.abs().sort_values(ascending=False).index[:max_display][::-1]
    fig, ax = plt.subplots()
    start = float(row.base_values)
    cum = start + vals.drop(order).sum()
    for i, f in enumerate(order):
        v = vals[f]
        ax.barh(i, v, left=cum, color=MALIGNANT_COLOR if v > 0 else BENIGN_COLOR)
        cum += v
    ax.set_yticks(range(len(order)), order)
    ax.axvline(start, color="#9AA5B1", ls="--")
```

```
    _style(ax)
    return fig


def plot_shap_force_matplotlib(explanation_row: shap.Explanation, top: int = 6):
    """Static force plot. The `top` strongest features are shown individually; the
    remainder are
    summed into 'other features (+/−)' groups, which keeps base + sum(SHAP) = f(x)
    exact."""
    plt.close("all")
    vals = np.asarray(explanation_row.values, dtype=float)
    data = np.asarray(explanation_row.data, dtype=float)
    names = list(explanation_row.feature_names)
    keep = list(np.argsort(-np.abs(vals))[:top])
    rest = [i for i in range(len(vals)) if i not in keep]
    pos_rest = [i for i in rest if vals[i] > 0]
    neg_rest = [i for i in rest if vals[i] <= 0]

    f_vals = [vals[i] for i in keep]
    f_data = [f"{data[i]:.4g}" if abs(data[i]) < 1000 else f"{data[i]:.0f}" for i in keep]
    f_names = [names[i] for i in keep]
    for group, sign in ((pos_rest, "+"), (neg_rest, "−")):
        if group:
            f_vals.append(vals[group].sum())
            f_data.append("")
            f_names.append(f"{len(group)} other features ({sign})")
    try:
        shap.plots.force(float(explanation_row.base_values), np.array(f_vals),
    np.array(f_data, dtype=object),
                         feature_names=f_names, matplotlib=True, show=False,
    text_rotation=12,
                         contribution_threshold=0.03)
        fig = plt.gcf()
        fig.set_size_inches(14, 3.6)
    except Exception:
        fig = _manual_waterfall(explanation_row, top)
    return fig


def shap_force_html(explanation_row: shap.Explanation) -> str:
    """Interactive force plot as a standalone HTML string (bundles SHAP's JS)."""
    plot = shap.plots.force(
        float(explanation_row.base_values),
        np.asarray(explanation_row.values, dtype=float),
        np.round(np.asarray(explanation_row.data, dtype=float), 3),
        feature_names=list(explanation_row.feature_names),
        out_names="P(malignant)",
    )
    return f"<html><head><meta charset='utf-
    8'>{shap.getjs()}</head><body>{plot.html()}</body></html>"


# ---------------------------------------------------------------- LIME vs SHAP
def plot_lime_vs_shap(direction_table: pd.DataFrame, title: str = "LIME vs SHAP — same
    patient, same class"):
    """Side-by-side bars over the union of both top-k lists. Independent x-axes because the
    two methods' magnitudes have different meanings."""
    d = direction_table.sort_values("shap_rank", ascending=False)
    fig, axes = plt.subplots(1, 2, figsize=(11, 0.45 * len(d) + 1.6), sharey=True)
    for ax, col, name in [(axes[0], "lime_weight", "LIME surrogate weight"),
                          (axes[1], "shap_value", "SHAP value (Δ probability)")]:
        colors = [MALIGNANT_COLOR if v > 0 else BENIGN_COLOR for v in d[col]]
        ax.barh(d["feature"], d[col], color=colors)
        ax.axvline(0, color=INK, lw=0.8)
        ax.set_xlabel(name)
        ax.xaxis.set_major_locator(matplotlib.ticker.MaxNLocator(5))
        _style(ax)
    for i, (_, r) in enumerate(d.iterrows()):
        if not r["same_direction"]:
```

```
            axes[1].text(1.01, i, "⚠ direction differs",
    transform=axes[1].get_yaxis_transform(),
                         va="center", fontsize=8, color="#B54708")
    axes[0].set_title("LIME (rank by |weight|)")
    axes[1].set_title("SHAP (rank by |value|)")
    fig.suptitle(title, fontweight="bold")
    fig.tight_layout()
    return fig


def plot_case_study_overview(cases: pd.DataFrame):
    """Top-5 Jaccard and Spearman per case study."""
    d = cases[cases["found"]].copy()
    labels = [f"{t}\n(patient {int(p)})" for t, p in zip(d["case_type"], d["patient_id"])]
    x = np.arange(len(d))
    fig, ax = plt.subplots(figsize=(9, 4))
    for offset, col, color, label in [(-0.2, "jaccard_top_k", ACCENT, "Top-5 Jaccard"),
                                      (0.2, "spearman_abs_all_features", "#8E6C8A",
    "Spearman ρ (|attr|, 30 features)")]:
        bars = ax.bar(x + offset, d[col], 0.4, color=color, label=label)
        ax.bar_label(bars, fmt="%.2f", fontsize=8, padding=2)
    ax.set_xticks(x, labels, fontsize=8)
    ax.set_ylim(min(0, d["spearman_abs_all_features"].min() - 0.1), 1.05)
    ax.axhline(0, color=INK, lw=0.6)
    ax.set_ylabel("Agreement")
    ax.set_title("LIME–SHAP agreement across case studies")
    ax.legend(frameon=False)
    ax.spines[["top", "right"]].set_visible(False)
    fig.tight_layout()
    return fig


# ---------------------------------------------------------------- architecture
def plot_architecture():
    fig, ax = plt.subplots(figsize=(12, 4.2))
    ax.set_xlim(0, 12)
    ax.set_ylim(0, 4.2)
    ax.axis("off")

    def box(x, y, w, h, text, fc):
        ax.add_patch(FancyBboxPatch((x, y), w, h,
    boxstyle="round,pad=0.05,rounding_size=0.15",
                                    fc=fc, ec=INK, lw=1))
        ax.text(x + w / 2, y + h / 2, text, ha="center", va="center", fontsize=9,
    color=INK, wrap=True)

    def arrow(x1, y1, x2, y2):
        ax.annotate("", xy=(x2, y2), xytext=(x1, y1), arrowprops=dict(arrowstyle="->",
    color=INK, lw=1.3))

    box(0.2, 1.6, 1.9, 1.0, "Breast Cancer\nWisconsin (569×30)\nmalignant = 1", "#E8F1FA")
    box(2.6, 1.6, 1.9, 1.0, "Stratified 80/20\nsplit (seed 42)", "#E8F1FA")
    box(5.0, 1.6, 1.9, 1.0, "Random Forest\n(300 trees)\n+ LR baseline", "#E6F4F1")
    box(7.5, 3.0, 2.0, 0.95, "LIME\nlocal surrogate\n(train-data stats)", "#FDF2E9")
    box(7.5, 1.6, 2.0, 0.95, "SHAP TreeExplainer\nlocal + global\n(additivity check)",
    "#FDF2E9")
    box(7.5, 0.2, 2.0, 0.95, "Evaluation\nAcc/Prec/Rec/F1/AUC\nconfusion, ROC", "#F3F4F6")
    box(10.0, 1.6, 1.8, 1.0, "LIME vs SHAP\ncomparison +\ncase studies", "#FCE8EC")
    box(10.0, 3.0, 1.8, 0.95, "Streamlit\ndashboard", "#E6F4F1")
    arrow(2.1, 2.1, 2.6, 2.1)
    arrow(4.5, 2.1, 5.0, 2.1)
    arrow(6.9, 2.3, 7.5, 3.4)
    arrow(6.9, 2.1, 7.5, 2.1)
    arrow(6.9, 1.9, 7.5, 0.7)
    arrow(9.5, 3.45, 10.0, 2.4)
    arrow(9.5, 2.1, 10.0, 2.1)
    arrow(10.9, 2.6, 10.9, 3.0)
    ax.set_title("ClinXAI architecture — tabular clinical diagnostics with two
    complementary explainers",
```

```
                fontweight="bold")
    return fig
```

A.11 train_model.py — Training script (89 lines)

```
"""Train the ClinXAI Random Forest (and a Logistic Regression baseline), evaluate on the
    held-out
test set, and save the model bundle plus report-ready metrics and figures.

Usage:  python train_model.py
"""
import json

import matplotlib.pyplot as plt
import pandas as pd
from sklearn.ensemble import RandomForestClassifier

from src.config import FIGURES_DIR, METRICS_DIR, MODEL_PATH, RF_PARAMS, ensure_dirs
from src.data_loader import dataset_overview, load_dataset, split_data
from src.model import (
    cross_validated_auc,
    evaluate_classifier,
    roc_points,
    save_bundle,
    train_logistic_baseline,
    train_random_forest,
)
from src.visualization import plot_confusion_matrix, plot_roc, save


def main() -> dict:
    ensure_dirs()
    X, y, feature_names = load_dataset()
    split = split_data(X, y)
    print(f"Dataset: {X.shape[0]} samples x {X.shape[1]} features | malignant (1) =
    {int(y.sum())}, "
          f"benign (0) = {int((y == 0).sum())}")
    print(f"Train: {len(split.X_train)} | Test: {len(split.X_test)} (stratified,
    random_state=42)")

    rf = train_random_forest(split.X_train, split.y_train)
    lr = train_logistic_baseline(split.X_train, split.y_train)

    rf_metrics = evaluate_classifier(rf, split.X_test, split.y_test)
    lr_metrics = evaluate_classifier(lr, split.X_test, split.y_test)
    rf_metrics.update(cross_validated_auc(lambda: RandomForestClassifier(**RF_PARAMS),
                                          split.X_train, split.y_train))

    metrics = {
        "dataset": {
            "name": "Breast Cancer Wisconsin (Diagnostic) —
    sklearn.datasets.load_breast_cancer",
            "n_samples": int(X.shape[0]),
            "n_features": int(X.shape[1]),
            "label_encoding": {"malignant": 1, "benign": 0},
            "n_malignant": int(y.sum()),
            "n_benign": int((y == 0).sum()),
            "n_train": len(split.X_train),
            "n_test": len(split.X_test),
            "test_malignant": int(split.y_test.sum()),
            "test_benign": int((split.y_test == 0).sum()),
            "split": "stratified 80/20, random_state=42",
        },
        "random_forest": {"params": {k: v for k, v in RF_PARAMS.items()}, **rf_metrics},
        "logistic_regression_baseline": lr_metrics,
    }
```

```
    save_bundle(rf, lr, split, metrics)
    with open(METRICS_DIR / "metrics.json", "w", encoding="utf-8") as f:
        json.dump(metrics, f, indent=2)

    rows = []
    for name, m in [("Random Forest", rf_metrics), ("Logistic Regression (baseline)",
    lr_metrics)]:
        rows.append({"model": name, **{k: m[k] for k in
                     ["accuracy", "precision", "recall_sensitivity", "specificity", "f1",
    "roc_auc"]},
                     **m["confusion_matrix"]})
    pd.DataFrame(rows).round(4).to_csv(METRICS_DIR / "metrics.csv", index=False)
    dataset_overview(X, y).to_csv(METRICS_DIR / "dataset_overview.csv")

    save(plot_confusion_matrix(rf_metrics["confusion_matrix"], "Random Forest — confusion
    matrix (test set)"),
         FIGURES_DIR / "confusion_matrix.png")
    curves = {
        "Random Forest": (*roc_points(rf, split.X_test, split.y_test),
    rf_metrics["roc_auc"]),
        "Logistic Regression": (*roc_points(lr, split.X_test, split.y_test),
    lr_metrics["roc_auc"]),
    }
    save(plot_roc(curves), FIGURES_DIR / "roc_curve.png")
    plt.close("all")

    print("\nTest-set metrics (positive class = Malignant):")
    print(pd.DataFrame(rows).set_index("model").round(4).to_string())
    print(f"\nRF 5-fold CV ROC-AUC on training set: {rf_metrics['cv_roc_auc_mean']:.4f} "
          f"+/- {rf_metrics['cv_roc_auc_std']:.4f}")
    print(f"\nSaved model bundle -> {MODEL_PATH}")
    return metrics


if __name__ == "__main__":
    main()
```

A.12 evaluate.py — Explanation evaluation script (183 lines)

```
"""Generate explanation evidence: SHAP global/local plots, LIME plots, LIME-vs-SHAP
    comparison over
the whole held-out test set, and programmatically selected case studies.

Usage:  python evaluate.py        (requires python train_model.py first)
"""
import json

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from src.config import (
    CASE_STUDIES_DIR,
    CLASS_NAMES,
    DECISION_THRESHOLD,
    FIGURES_DIR,
    METRICS_DIR,
    TOP_K,
    ensure_dirs,
)
from src.comparison import compare_explanations, lime_stability, select_case_studies
from src.explain_lime import explain_patient_lime
from src.explain_shap import additivity_error, build_tree_explainer, compute_shap,
    global_importance, shap_table
from src.model import load_bundle, malignant_proba
from src import visualization as viz
```

```
ADDITIVITY_TOL = 1e-6


def describe_case(case: dict, p: float, true_label: int, cmp: dict, lime_res, stab: dict) -
    > str:
    pred = CLASS_NAMES[int(p >= DECISION_THRESHOLD)]
    correct = "correct" if int(p >= DECISION_THRESHOLD) == true_label else "INCORRECT"
    parts = [
        f"Model predicts {pred} with P(malignant)={p:.3f} (actual:
    {CLASS_NAMES[true_label]}; {correct}).",
        f"LIME and SHAP share {cmp['top_k_overlap']}/{TOP_K} top features "
        f"(Jaccard {cmp['jaccard_top_k']:.2f}; Spearman rho over |attributions|
    {cmp['spearman_abs_all_features']:.2f}).",
    ]
    if cmp["direction_disagreements_union"]:
        parts.append("Opposite direction for: " + ",
    ".join(cmp["direction_disagreements_union"]) + ".")
    else:
        parts.append("No direction disagreement within the top-5 union.")
    parts.append(f"LIME surrogate local fit R^2={lime_res.score:.2f}; top-5 stability
    across "
                 f"{len(stab['seeds'])} seeds: mean Jaccard
    {stab['mean_pairwise_jaccard_top_k']:.2f}.")
    return " ".join(parts)


def main() -> None:
    ensure_dirs()
    bundle = load_bundle()
    model, X_train, X_test, y_test = bundle["model"], bundle["X_train"], bundle["X_test"],
    bundle["y_test"]

    # ------------------------------------------------------------ SHAP (global)
    explainer = build_tree_explainer(model)
    shap_test = compute_shap(explainer, X_test)
    err = additivity_error(model, shap_test, X_test)
    assert err < ADDITIVITY_TOL, f"SHAP additivity violated: {err}"
    print(f"SHAP additivity check on {len(X_test)} test patients: max |error| = {err:.2e}")
    with open(METRICS_DIR / "shap_additivity.json", "w", encoding="utf-8") as f:
        json.dump({"max_abs_error": err, "tolerance": ADDITIVITY_TOL, "n_patients":
    len(X_test),
                   "base_value_p_malignant": float(shap_test.base_values[0]),
                   "explainer":
    "shap.TreeExplainer(feature_perturbation='tree_path_dependent')",
                   "output": "P(malignant) — probability units"}, f, indent=2)

    importance = global_importance(shap_test)
    importance.rename("mean_abs_shap").rename_axis("feature").to_csv(METRICS_DIR /
    "shap_global_importance.csv")
    viz.save(viz.plot_shap_global_bar(importance), FIGURES_DIR /
    "shap_global_importance.png")
    viz.save(viz.plot_shap_beeswarm(shap_test), FIGURES_DIR / "shap_summary_beeswarm.png")
    plt.close("all")

    # ------------------------------------------------------------ LIME vs SHAP on every
    test patient
    proba = malignant_proba(model, X_test)
    rows, cache = [], {}
    print(f"Running LIME on {len(X_test)} test patients ...")
    for i, pid in enumerate(X_test.index):
        lime_res = explain_patient_lime(model, X_train, X_test.loc[[pid]])
        shap_vals = pd.Series(shap_test.values[i], index=shap_test.feature_names)
        cmp = compare_explanations(lime_res.weights, shap_vals)
        cache[pid] = (i, lime_res, cmp)
        rows.append({
            "patient_id": int(pid),
            "true_label": int(y_test.loc[pid]),
            "p_malignant": float(proba[i]),
            "predicted_label": int(proba[i] >= DECISION_THRESHOLD),
```

```
            "lime_top5": "; ".join(cmp["lime_top_k"]),
            "shap_top5": "; ".join(cmp["shap_top_k"]),
            "top_k_overlap": cmp["top_k_overlap"],
            "jaccard_top_k": cmp["jaccard_top_k"],
            "spearman_abs_all_features": cmp["spearman_abs_all_features"],
            "n_direction_disagreements": len(cmp["direction_disagreements_union"]),
            "lime_local_r2": lime_res.score,
            "lime_local_pred": lime_res.local_pred,
        })
    summary = pd.DataFrame(rows)
    summary.round(4).to_csv(CASE_STUDIES_DIR / "test_set_lime_vs_shap.csv", index=False)

    aggregate = {
        "n_patients": len(summary),
        "mean_top5_jaccard": float(summary["jaccard_top_k"].mean()),
        "mean_top5_overlap": float(summary["top_k_overlap"].mean()),
        "mean_spearman_abs": float(summary["spearman_abs_all_features"].mean()),
        "pct_patients_with_direction_disagreement":
    float((summary["n_direction_disagreements"] > 0).mean() * 100),
        "mean_lime_local_r2": float(summary["lime_local_r2"].mean()),
    }

    # ------------------------------------------------------------ case studies
    cases = select_case_studies(summary)
    case_rows, case_details = [], []
    for n, case in enumerate(cases, start=1):
        if not case["found"]:
            print(f"Case {n} ({case['case_type']}): NOT FOUND - {case['criterion']}")
            case_rows.append({"case": n, **case})
            case_details.append({"case": n, **case})
            continue
        pid = case["patient_id"]
        i, lime_res, cmp = cache[pid]
        patient = X_test.loc[[pid]]
        stab = lime_stability(model, X_train, patient)
        p = float(proba[i])
        observation = describe_case(case, p, int(y_test.loc[pid]), cmp, lime_res, stab)
        print(f"Case {n} ({case['case_type']}), patient {pid}: {observation}")

        tag = f"case{n}_patient{pid}"
        row_expl = shap_test[i]
        viz.save(viz.plot_lime(lime_res.table, title=f"LIME — patient {pid}
    ({case['case_type']})"),
                 FIGURES_DIR / f"lime_{tag}.png")
        viz.save(viz.plot_shap_waterfall(row_expl), FIGURES_DIR /
    f"shap_waterfall_{tag}.png")
        viz.save(viz.plot_shap_force_matplotlib(row_expl), FIGURES_DIR /
    f"shap_force_{tag}.png")
        (FIGURES_DIR / f"shap_force_{tag}.html").write_text(viz.shap_force_html(row_expl),
    encoding="utf-8")
        viz.save(viz.plot_lime_vs_shap(cmp["direction_table"],
                                       title=f"LIME vs SHAP — patient {pid}
    ({case['case_type']})"),
                 FIGURES_DIR / f"lime_vs_shap_{tag}.png")
        plt.close("all")

        case_rows.append({
            "case": n, **case,
            "true_label": CLASS_NAMES[int(y_test.loc[pid])],
            "p_malignant": round(p, 4),
            "predicted": CLASS_NAMES[int(p >= DECISION_THRESHOLD)],
            "lime_top5": "; ".join(cmp["lime_top_k"]),
            "shap_top5": "; ".join(cmp["shap_top_k"]),
            "top_k_overlap": cmp["top_k_overlap"],
            "jaccard_top_k": round(cmp["jaccard_top_k"], 4),
            "spearman_abs_all_features": round(cmp["spearman_abs_all_features"], 4),
            "direction_disagreements": "; ".join(cmp["direction_disagreements_union"]),
            "lime_local_r2": round(lime_res.score, 4),
            "lime_stability_mean_jaccard": round(stab["mean_pairwise_jaccard_top_k"], 4),
```

```
            "lime_features_with_sign_flip_across_seeds": stab["features_with_sign_flip"],
            "observation": observation,
        })
        case_details.append({
            "case": n, **case,
            "true_label": CLASS_NAMES[int(y_test.loc[pid])],
            "p_malignant": p,
            "shap_base_value": float(row_expl.base_values),
            "shap_sum": float(row_expl.values.sum()),
            "lime_intercept": lime_res.intercept,
            "lime_local_pred": lime_res.local_pred,
            "lime_local_r2": lime_res.score,
            "comparison": {k: v for k, v in cmp.items() if k != "direction_table"},
            "direction_table": cmp["direction_table"].round(5).to_dict(orient="records"),
            "lime_top10": lime_res.table.head(10)[["feature", "rule",
    "weight"]].round(5).to_dict(orient="records"),
            "shap_top10": shap_table(row_expl).head(10)[["feature", "value",
    "shap_value"]].round(5)
            .to_dict(orient="records"),
            "lime_stability": {k: v for k, v in stab.items() if k not in ("weight_mean",
    "weight_std")},
            "observation": observation,
        })

    case_df = pd.DataFrame(case_rows)
    case_df.to_csv(CASE_STUDIES_DIR / "case_studies.csv", index=False)
    with open(CASE_STUDIES_DIR / "case_studies.json", "w", encoding="utf-8") as f:
        json.dump({"aggregate_test_set": aggregate, "cases": case_details}, f, indent=2,
    default=float)
    viz.save(viz.plot_case_study_overview(case_df), FIGURES_DIR /
    "case_study_agreement.png")
    viz.save(viz.plot_architecture(), FIGURES_DIR / "architecture.png")
    plt.close("all")

    print("\nAggregate LIME vs SHAP agreement over the test set:")
    for k, v in aggregate.items():
        print(f"  {k}: {v:.4f}" if isinstance(v, float) else f"  {k}: {v}")
    print(f"\nOutputs written to {FIGURES_DIR.parent}")


if __name__ == "__main__":
    main()
```

A.13 app.py — Streamlit dashboard (488 lines)

```
"""ClinXAI — Transparent Clinical Diagnostics (Streamlit dashboard).

Run:  streamlit run app.py
Research / educational demonstration only. Not a medical device; no clinical
    recommendations.
"""
import json
import warnings

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import streamlit as st
import streamlit.components.v1 as components

from src.comparison import compare_explanations, lime_stability
from src.config import (
    CASE_STUDIES_DIR,
    CLASS_NAMES,
    DECISION_THRESHOLD,
    FIGURES_DIR,
    LIME_NUM_SAMPLES,
```

```
    METRICS_DIR,
    MODEL_PATH,
    RANDOM_STATE,
    TOP_K,
)
from src.data_loader import out_of_range_features, validate_patient_input
from src.explain_lime import explain_patient_lime
from src.explain_shap import additivity_error, build_tree_explainer, compute_shap,
    global_importance, shap_table
from src.model import load_bundle, malignant_proba, roc_points
from src import visualization as viz

warnings.filterwarnings("ignore")
st.set_page_config(page_title="ClinXAI — Transparent Clinical Diagnostics",
    page_icon="🩺", layout="wide")

st.markdown(
    """
    <style>
      .block-container {padding-top: 1.6rem; max-width: 1350px;}
      .cx-banner {background: linear-gradient(90deg,#0B7A75 0%,#1E6FB8 100%); color:#fff;
                  padding: 18px 24px; border-radius: 12px; margin-bottom: 10px;}
      .cx-banner h1 {color:#fff; margin:0; font-size: 1.9rem;}
      .cx-banner p {margin: 4px 0 0 0; opacity: .92;}
      .cx-disclaimer {background:#FFF7E6; border-left: 5px solid #D98E04; padding: 10px
    14px;
                      border-radius: 6px; font-size: .92rem; color:#5B3B00;}
      .cx-pred {border-radius: 12px; padding: 16px 20px; color: #fff; font-size: 1.25rem;
    font-weight: 700;}
      .cx-mal {background:#C8102E;} .cx-ben {background:#1E6FB8;}
      div[data-testid="stMetricValue"] {font-size: 1.6rem;}
    </style>
    """,
    unsafe_allow_html=True,
)

DISCLAIMER = (
    "⚠️ <b>Research / educational demonstration only.</b> ClinXAI is not a medical device,
    has not been "
    "clinically validated, and must not be used for diagnosis or treatment decisions."
)


# ---------------------------------------------------------------------- cached resources
@st.cache_resource(show_spinner="Loading model and computing SHAP values …")
def load_resources():
    if not MODEL_PATH.exists():
        import train_model
        train_model.main()
    bundle = load_bundle()
    explainer = build_tree_explainer(bundle["model"])
    shap_test = compute_shap(explainer, bundle["X_test"])
    return {
        **bundle,
        "explainer": explainer,
        "shap_test": shap_test,
        "importance": global_importance(shap_test),
        "proba_test": malignant_proba(bundle["model"], bundle["X_test"]),
        "additivity_error": additivity_error(bundle["model"], shap_test, bundle["X_test"]),
    }


@st.cache_data(show_spinner="Running LIME (local perturbations) …", max_entries=64)
def cached_lime(values: tuple, seed: int, num_samples: int):
    R = load_resources()
    patient = pd.DataFrame([values], columns=R["feature_names"])
    return explain_patient_lime(R["model"], R["X_train"], patient, seed=seed,
    num_samples=num_samples)
```

```
@st.cache_data(max_entries=64)
def cached_shap(values: tuple):
    R = load_resources()
    patient = pd.DataFrame([values], columns=R["feature_names"])
    return compute_shap(R["explainer"], patient)[0]


@st.cache_data(show_spinner="Re-running LIME with 5 different seeds …", max_entries=16)
def cached_stability(values: tuple, num_samples: int):
    R = load_resources()
    patient = pd.DataFrame([values], columns=R["feature_names"])
    return lime_stability(R["model"], R["X_train"], patient, num_samples=num_samples)


@st.cache_data
def load_case_studies():
    path_csv, path_json = CASE_STUDIES_DIR / "case_studies.csv", CASE_STUDIES_DIR /
    "case_studies.json"
    if not path_csv.exists() or not path_json.exists():
        return None, None
    with open(path_json, encoding="utf-8") as f:
        return pd.read_csv(path_csv), json.load(f)


def show(fig):
    st.pyplot(fig, clear_figure=True, width="stretch")
    plt.close(fig)


R = load_resources()
model, X_train, X_test, y_test = R["model"], R["X_train"], R["X_test"], R["y_test"]
features = R["feature_names"]
case_df, case_json = load_case_studies()
case_lookup = {}
if case_df is not None:
    for _, r in case_df[case_df["found"]].iterrows():
        case_lookup[int(r["patient_id"])] = r["case_type"]

# ---------------------------------------------------------------------- sidebar
st.sidebar.markdown("### 🩺 ClinXAI")
st.sidebar.caption("Select a held-out test patient. Every tab updates for the selected
    patient.")
patient_ids = [int(i) for i in X_test.index]
if "patient_id" not in st.session_state:
    st.session_state["patient_id"] = next(iter(case_lookup), patient_ids[0])


def patient_label(pid: int) -> str:
    i = patient_ids.index(pid)
    tag = f"  ★ {case_lookup[pid]}" if pid in case_lookup else ""
    return f"Patient #{pid} — actual {CLASS_NAMES[int(y_test.loc[pid])]},
    P={R['proba_test'][i]:.2f}{tag}"


pid = st.sidebar.selectbox("Test patient", patient_ids, format_func=patient_label,
    key="patient_id")
st.sidebar.markdown("---")
st.sidebar.markdown("**LIME settings**")
lime_seed = int(st.sidebar.number_input("Random seed", min_value=0, max_value=10_000,
    value=RANDOM_STATE, step=1))
lime_samples = int(st.sidebar.select_slider("Perturbation samples", [1000, 2000, 5000,
    10000],
                                            value=LIME_NUM_SAMPLES))
st.sidebar.caption("Seed 42 with 5000 samples reproduces the saved case-study outputs
    exactly.")
st.sidebar.markdown("---")
st.sidebar.markdown(f"<div class='cx-disclaimer'>{DISCLAIMER}</div>",
    unsafe_allow_html=True)
```

```
# ---------------------------------------------------------------------- header
st.markdown(
    "<div class='cx-banner'><h1>ClinXAI — Transparent Clinical Diagnostics</h1>"
    "<p>Random Forest diagnosis of breast-tumour biopsies, explained locally with LIME and
    locally + globally "
    "with SHAP</p></div>",
    unsafe_allow_html=True,
)

tabs = st.tabs(["Overview", "Diagnostic Prediction", "LIME Explanation", "SHAP
    Explanation",
                "LIME vs SHAP", "Model Evaluation", "Case Studies"])

# ---------------------------------------------------------------------- A. Overview
with tabs[0]:
    c1, c2 = st.columns([1.35, 1])
    with c1:
        st.subheader("Objective")
        st.markdown(
            "Build an end-to-end clinical diagnostic prediction system with a complex tree
    ensemble "
            "(**Random Forest**) and make every prediction transparent with two
    complementary explainers:\n"
            "- **LIME** — fits a simple weighted linear *surrogate* around one patient
    using random perturbations.\n"
            "- **SHAP** — game-theoretic *Shapley values* that split the prediction exactly
    among the features "
            "(local), and aggregate over patients (global).\n\n"
            "The project then studies **when the two explanations disagree**, which matters
    most on borderline "
            "and misclassified patients."
        )
        st.subheader("Dataset")
        meta = R["metrics"]["dataset"]
        st.markdown(
            f"**Breast Cancer Wisconsin (Diagnostic)** via
    `sklearn.datasets.load_breast_cancer()` — "
            f"{meta['n_samples']} patients × {meta['n_features']} numeric features computed
    from digitised images "
            "of fine-needle-aspirate cell nuclei (radius, texture, perimeter, area,
    smoothness, compactness, "
            "concavity, concave points, symmetry, fractal dimension; each as *mean*,
    *standard error* and *worst*).\n\n"
            f"- Class encoding used throughout the app: **Malignant = 1 (positive / risk
    class)**, Benign = 0 "
            f"(sklearn's raw encoding is the reverse and is re-mapped on load).\n"
            f"- {meta['n_malignant']} malignant, {meta['n_benign']} benign. "
            f"Stratified 80/20 split → {meta['n_train']} train / {meta['n_test']} held-out
    test patients."
        )
        st.info(
            "**Scope note.** The inputs are *structured tabular features* derived from
    images — the app does not "
            "process raw images. 'Multi-modal' here refers to two complementary explanation
    modalities "
            "(LIME and SHAP). True image + tabular fusion is future scope."
        )
    with c2:
        st.subheader("Workflow")
        arch = FIGURES_DIR / "architecture.png"
        if arch.exists():
            st.image(str(arch), width="stretch")
        st.markdown(
            "1. Load & validate data, re-encode malignant = 1\n"
            "2. Stratified split (seed 42) — all fitting on training data only\n"
            "3. Train Random Forest (+ Logistic Regression baseline)\n"
            "4. Predict P(malignant) for a held-out patient\n"
            "5. Explain with LIME and SHAP (same patient, same class)\n"
```

```
            "6. Compare rankings & directions; evaluate case studies"
        )
    st.markdown(f"<div class='cx-disclaimer'>{DISCLAIMER}</div>", unsafe_allow_html=True)

# ------------------------------------------------------------------ B. Diagnostic
    prediction
with tabs[1]:
    base_row = X_test.loc[pid]
    st.subheader(f"Patient #{pid}")
    st.caption(f"Recorded diagnosis (ground truth, hidden from the model):
    **{CLASS_NAMES[int(y_test.loc[pid])]}**"
               + (f" · Case study: **{case_lookup[pid]}**" if pid in case_lookup else ""))

    adjust = st.toggle("Adjust biomarker values (what-if analysis)", value=False,
    key=f"adj_on_{pid}")
    values = base_row.copy()
    if adjust:
        top_feats = list(R["importance"].index[:8])
        st.caption("Editing the 8 globally most important features (by mean |SHAP|). All
    explanations in the "
                   "other tabs use these adjusted values.")
        cols = st.columns(4)
        for j, f in enumerate(top_feats):
            lo, hi = float(X_train[f].min()), float(X_train[f].max())
            values[f] = cols[j % 4].number_input(
                f, min_value=0.0, value=float(base_row[f]), format="%.4f",
    key=f"adj_{pid}_{f}",
                help=f"Training range: {lo:.4g} – {hi:.4g}",
            )

    try:
        patient = validate_patient_input(values.to_dict(), features)
    except ValueError as exc:
        st.error(f"Invalid biomarker input: {exc}")
        st.stop()
    oor = out_of_range_features(patient, X_train)
    if oor:
        st.warning("Outside the training data range (model is extrapolating; explanations
    are less reliable): "
                   + ", ".join(oor))
    values_t = tuple(float(v) for v in patient.iloc[0].to_numpy())
    p = float(malignant_proba(model, patient)[0])
    pred = int(p >= DECISION_THRESHOLD)

    c1, c2 = st.columns([1, 2.2])
    with c1:
        css = "cx-mal" if pred else "cx-ben"
        st.markdown(f"<div class='cx-pred {css}'>Model output: {CLASS_NAMES[pred]}</div>",
    unsafe_allow_html=True)
        st.metric("P(malignant)", f"{p:.3f}")
        st.metric("P(benign)", f"{1 - p:.3f}")
        st.caption(f"Decision threshold {DECISION_THRESHOLD}. The probability is the
    fraction of tree votes — an "
                   "**uncalibrated** model score, not a calibrated clinical risk.")
    with c2:
        show(viz.plot_risk_gauge(p, DECISION_THRESHOLD))
        conf = abs(p - 0.5)
        if conf < 0.15:
            st.warning("Borderline output: the trees are split. Explanations for such
    patients tend to be less "
                       "stable — see the LIME vs SHAP tab.")
    with st.expander("All 30 biomarker values for this patient"):
        tbl = pd.DataFrame({"value": patient.iloc[0], "train min": X_train.min(), "train
    median": X_train.median(),
                            "train max": X_train.max()})
        st.dataframe(tbl.style.format("{:.4f}"), width="stretch", height=420)

lime_res = cached_lime(values_t, lime_seed, lime_samples)
shap_row = cached_shap(values_t)
```

```
shap_series = pd.Series(shap_row.values, index=features)
cmp = compare_explanations(lime_res.weights, shap_series, TOP_K)
adj_note = " (adjusted values)" if adjust else ""

# ---------------------------------------------------------------- C. LIME
with tabs[2]:
    st.subheader(f"LIME local explanation — Patient #{pid}{adj_note}")
    c1, c2 = st.columns([1.6, 1])
    with c1:
        show(viz.plot_lime(lime_res.table, n=10))
    with c2:
        m1, m2 = st.columns(2)
        m1.metric("Random Forest P(malignant)", f"{lime_res.model_proba:.3f}")
        m2.metric("Surrogate prediction", f"{lime_res.local_pred:.3f}")
        m1.metric("Surrogate intercept", f"{lime_res.intercept:.3f}")
        m2.metric("Local fit R²", f"{lime_res.score:.2f}")
        st.markdown(
            "**How to read this chart.** LIME generated "
            f"{lime_samples:,} perturbed versions of this patient (sampled from training-
    data statistics), asked "
            "the Random Forest for P(malignant) on each, and fitted a proximity-weighted
    linear model. Each bar "
            "is that surrogate's weight for a feature *condition* (e.g. `worst area >
    1122.50`). "
            "**Red** pushes toward malignant, **blue** toward benign."
        )
        if lime_res.score < 0.3:
            st.warning(f"Local fit R² = {lime_res.score:.2f}: the linear surrogate explains
    little of the model's "
                       "local behaviour, so treat these weights with caution.")
    st.markdown("**Most influential conditions (top 10)**")
    st.dataframe(lime_res.table.head(10)[["rank", "rule", "weight",
    "direction"]].style.format({"weight": "{:+.4f}"}),
                 width="stretch", hide_index=True)
    st.caption(f"Seed {lime_seed}, {lime_samples} samples, quartile discretisation,
    explainer fitted on training "
               "data only. Change the seed in the sidebar to see LIME's sensitivity to
    random perturbations.")

# ---------------------------------------------------------------- D. SHAP
with tabs[3]:
    st.subheader("Global explanations (held-out test set, 114 patients)")
    g1, g2 = st.columns(2)
    with g1:
        show(viz.plot_shap_global_bar(R["importance"]))
        st.caption("Mean |SHAP| — the average absolute change in P(malignant) attributed to
    each feature.")
    with g2:
        show(viz.plot_shap_beeswarm(R["shap_test"]))
        st.caption("Each dot is a patient. Position = effect on P(malignant); colour =
    feature value (red high).")

    st.markdown("---")
    st.subheader(f"Local explanation — Patient #{pid}{adj_note}")
    base_v = float(shap_row.base_values)
    s1, s2, s3 = st.columns(3)
    s1.metric("Base value E[f(x)]", f"{base_v:.3f}")
    s2.metric("Sum of SHAP values", f"{shap_row.values.sum():+.3f}")
    s3.metric("Base + sum = P(malignant)", f"{base_v + shap_row.values.sum():.3f}",
              help=f"Model predict_proba: {p:.3f}")
    l1, l2 = st.columns([1.15, 1])
    with l1:
        show(viz.plot_shap_waterfall(shap_row))
    with l2:
        tbl = shap_table(shap_row)
        st.markdown("**Feature contribution table**")
        st.dataframe(tbl[["rank", "feature", "value", "shap_value", "direction"]]
                     .style.format({"value": "{:.4f}", "shap_value": "{:+.4f}"})
```

```
                    .map(lambda v: "color:#C8102E" if v == "→ Malignant" else
    "color:#1E6FB8", subset=["direction"]),
                    width="stretch", hide_index=True, height=440)
    st.markdown("**Force plot**")
    show(viz.plot_shap_force_matplotlib(shap_row))
    with st.expander("Interactive force plot (SHAP JavaScript)"):
        try:
            force_html = viz.shap_force_html(shap_row)  # generated locally by SHAP, not
    user content
            if hasattr(st, "iframe"):
                st.iframe(force_html, height=180)
            else:  # older Streamlit
                components.html(force_html, height=180, scrolling=True)
        except Exception as exc:  # JS bundle unavailable in this SHAP version
            st.info(f"Interactive force plot unavailable ({exc}); the static version above
    is equivalent.")
    st.caption("TreeExplainer (tree-path-dependent) on the Random Forest's probability
    output. SHAP values, the base "
               f"value and predict_proba share probability units; max additivity error on
    the test set = "
               f"{R['additivity_error']:.1e}.")

# ---------------------------------------------------------------------- E. LIME vs SHAP
with tabs[4]:
    st.subheader(f"LIME vs SHAP — Patient #{pid}{adj_note}")
    st.caption("Same patient · same trained Random Forest · same target: P(malignant).")
    m = st.columns(4)
    m[0].metric(f"Top-{TOP_K} overlap", f"{cmp['top_k_overlap']} / {TOP_K}")
    m[1].metric(f"Top-{TOP_K} Jaccard", f"{cmp['jaccard_top_k']:.2f}")
    m[2].metric("Spearman ρ (|attr|, 30 features)",
    f"{cmp['spearman_abs_all_features']:.2f}")
    dis = cmp["direction_disagreements_union"]
    m[3].metric("Direction disagreements", f"{len(dis)}", help="Features in the union of
    both top-5 lists whose "
                "LIME weight and SHAP value have opposite sign.")
    show(viz.plot_lime_vs_shap(cmp["direction_table"]))

    c1, c2 = st.columns(2)
    with c1:
        st.markdown(f"**LIME top-{TOP_K}**")
        st.dataframe(lime_res.table.head(TOP_K)[["rank", "feature", "rule", "weight"]]
                     .style.format({"weight": "{:+.4f}"}), width="stretch",
    hide_index=True)
    with c2:
        st.markdown(f"**SHAP top-{TOP_K}**")
        st.dataframe(shap_table(shap_row).head(TOP_K)[["rank", "feature", "value",
    "shap_value"]]
                     .style.format({"value": "{:.4f}", "shap_value": "{:+.4f}"}),
    width="stretch", hide_index=True)

    st.markdown("**Ranking & direction table** (union of both top-5 lists)")
    dt = cmp["direction_table"].copy()
    dt["agreement"] = np.where(dt["same_direction"], "✓ same direction", "⚠ opposite")
    st.dataframe(dt[["feature", "lime_rank", "shap_rank", "lime_weight", "shap_value",
    "agreement"]]
                 .style.format({"lime_weight": "{:+.4f}", "shap_value": "{:+.4f}"}),
    width="stretch", hide_index=True)

    if dis:
        st.warning("Opposite direction for: **" + ", ".join(dis) + "**")
    else:
        st.success("LIME and SHAP agree on the direction of every feature in the top-5
    union.")

    if st.toggle("Measure LIME stability across 5 random seeds", value=False, key="stab"):
        stab = cached_stability(values_t, lime_samples)
        s = st.columns(3)
        s[0].metric("Mean pairwise top-5 Jaccard",
    f"{stab['mean_pairwise_jaccard_top_k']:.2f}")
```

```
        s[1].metric("Worst-case top-5 Jaccard",
    f"{stab['min_pairwise_jaccard_top_k']:.2f}")
        s[2].metric("Features whose sign flips", f"{stab['features_with_sign_flip']} / 30")
        st.caption("SHAP TreeExplainer is deterministic for a fixed model, so its stability
    is 1.00 by construction.")

    st.markdown("#### Why can the explanations differ?")
    reasons = [
        "**Different questions.** LIME fits a linear surrogate over *discretised* feature
    bins in a sampled "
        "neighbourhood; SHAP distributes the exact prediction using Shapley values along
    the trees' own splits. "
        "Their magnitudes are not interchangeable — only ranks and signs are compared
    here.",
        "**Correlated features.** Radius, perimeter and area (mean/worst) are almost
    collinear. Both methods may "
        "share credit among them differently, so a feature can rank high in one and low in
    the other.",
        "**Perturbation sampling.** LIME's samples are drawn independently per feature,
    creating unrealistic "
        "combinations (e.g. large radius with small area) that the forest never saw.",
    ]
    if lime_res.score < 0.3:
        reasons.append(f"**Weak local fit for this patient** — LIME R² =
    {lime_res.score:.2f}, so the surrogate "
                       "is a poor approximation here.")
    if abs(p - 0.5) < 0.25:
        reasons.append(f"**Near the decision boundary** (P = {p:.2f}): the forest's
    response is non-linear here, "
                       "which a single linear surrogate cannot capture.")
    st.markdown("\n".join(f"- {r}" for r in reasons))
    st.caption("Jaccard and Spearman measure agreement between explanation methods — they
    are not clinical "
               "validation metrics.")

# ---------------------------------------------------------------------- F. Model evaluation
with tabs[5]:
    rf_m, lr_m = R["metrics"]["random_forest"],
    R["metrics"]["logistic_regression_baseline"]
    st.subheader("Random Forest — held-out test set")
    st.caption(f"Positive class: **{rf_m['positive_class']}** · threshold
    {rf_m['decision_threshold']} · "
               f"{R['metrics']['dataset']['split']}")
    m = st.columns(6)
    for col, (name, key) in zip(m, [("Accuracy", "accuracy"), ("Precision", "precision"),
                                    ("Recall (sensitivity)", "recall_sensitivity"),
    ("Specificity", "specificity"),
                                    ("F1-score", "f1"), ("ROC-AUC", "roc_auc")]):
        col.metric(name, f"{rf_m[key]:.4f}")
    c1, c2 = st.columns(2)
    with c1:
        show(viz.plot_confusion_matrix(rf_m["confusion_matrix"], "Random Forest — confusion
    matrix (test set)"))
    with c2:
        curves = {
            "Random Forest": (*roc_points(model, X_test, y_test), rf_m["roc_auc"]),
            "Logistic Regression": (*roc_points(R["baseline"], X_test, y_test),
    lr_m["roc_auc"]),
        }
        show(viz.plot_roc(curves))
    st.markdown("**Comparison with interpretable baseline**")
    cols = ["accuracy", "precision", "recall_sensitivity", "specificity", "f1", "roc_auc"]
    st.dataframe(pd.DataFrame({"Random Forest": [rf_m[c] for c in cols],
                               "Logistic Regression (baseline)": [lr_m[c] for c in cols]},
    index=cols)
                 .style.format("{:.4f}"), width="stretch")
    d = R["metrics"]["dataset"]
    st.markdown(
        f"- Training set: **{d['n_train']}** patients · Test set: **{d['n_test']}**
    patients "
```

```
        f"({d['test_malignant']} malignant / {d['test_benign']} benign)\n"
        f"- Random Forest: {rf_m['params']['n_estimators']} trees, random_state
    {rf_m['params']['random_state']}\n"
        f"- 5-fold stratified CV ROC-AUC on the training set:
    **{rf_m['cv_roc_auc_mean']:.4f} ± "
        f"{rf_m['cv_roc_auc_std']:.4f}**\n"
        f"- False negatives (malignant predicted benign):
    **{rf_m['confusion_matrix']['fn']}** — the clinically "
        "most costly error type; examined in the Case Studies tab."
    )
    st.caption("A single 114-patient test split is small; metrics would vary with a
    different split.")

# ---------------------------------------------------------------------- G. Case studies
with tabs[6]:
    st.subheader("Reproducible case studies (selected programmatically from the held-out
    test set)")
    if case_df is None:
        st.info("Case-study outputs not found. Run `python evaluate.py` to generate them.")
    else:
        agg = case_json["aggregate_test_set"]
        a = st.columns(4)
        a[0].metric("Mean top-5 Jaccard (114 patients)", f"{agg['mean_top5_jaccard']:.2f}")
        a[1].metric("Mean top-5 overlap", f"{agg['mean_top5_overlap']:.2f} / 5")
        a[2].metric("Mean Spearman ρ", f"{agg['mean_spearman_abs']:.2f}")
        a[3].metric("Patients with a direction disagreement",
    f"{agg['pct_patients_with_direction_disagreement']:.1f}%")
        ov = FIGURES_DIR / "case_study_agreement.png"
        if ov.exists():
            st.image(str(ov), width="stretch")

        def jump(target):
            st.session_state["patient_id"] = target

        for case in case_json["cases"]:
            title = f"Case {case['case']}: {case['case_type']}"
            if not case["found"]:
                with st.expander(title + " — not found"):
                    st.info(case["criterion"])
                continue
            cpid = case["patient_id"]
            with st.expander(f"{title} — Patient #{cpid} · P(malignant) =
    {case['p_malignant']:.3f} · "
                             f"actual {case['true_label']}", expanded=case["case"] == 3):
                st.caption(f"Selection rule: {case['criterion']}")
                st.markdown(f"**Observation.** {case['observation']}")
                c = case["comparison"]
                k = st.columns(4)
                k[0].metric("Top-5 overlap", f"{c['top_k_overlap']} / 5")
                k[1].metric("Spearman ρ", f"{c['spearman_abs_all_features']:.2f}")
                k[2].metric("LIME local R²", f"{case['lime_local_r2']:.2f}")
                k[3].metric("LIME seed stability",
    f"{case['lime_stability']['mean_pairwise_jaccard_top_k']:.2f}")
                tag = f"case{case['case']}_patient{cpid}"
                img = FIGURES_DIR / f"lime_vs_shap_{tag}.png"
                if img.exists():
                    st.image(str(img), width="stretch")
                st.button(f"Open patient #{cpid} in all tabs", key=f"jump_{cpid}",
    on_click=jump, args=(cpid,))

        st.markdown("#### Engineering limitations")
        st.markdown(
            "- **Uncalibrated probabilities** — Random Forest vote fractions; no
    calibration was fitted.\n"
            "- **Small, single-centre dataset** (569 patients); a single 80/20 split; no
    external validation.\n"
            "- **LIME instability** — results depend on the random seed and number of
    samples, and its local fit "
            "R² is often low (mean over test set reported above).\n"
```

```
            "- **LIME vs SHAP semantics differ** — surrogate coefficients vs Shapley
    attributions; only ranks and "
            "signs are compared, never magnitudes.\n"
            "- **Correlated features** (radius/perimeter/area) split credit unpredictably
    in both methods.\n"
            "- **Tree-path-dependent SHAP** reflects the model's learned structure, not
    causal effects.\n"
            "- **Tabular only** — features derived from images; no raw-image modality."
        )
    st.markdown(f"<div class='cx-disclaimer'>{DISCLAIMER}</div>", unsafe_allow_html=True)
```

A.14 tests/test_pipeline.py — Automated test suite (282 lines)

```
"""End-to-end verification tests for ClinXAI.  Run:  python -m pytest -q"""
import sys
import warnings
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import pytest
from matplotlib.figure import Figure
from sklearn.datasets import load_breast_cancer

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))

from src import visualization as viz  # noqa: E402
from src.comparison import compare_explanations, jaccard, lime_stability,
    select_case_studies, top_k  # noqa: E402
from src.config import MALIGNANT_LABEL  # noqa: E402
from src.data_loader import load_dataset, split_data, validate_patient_input  # noqa: E402
from src.explain_lime import explain_patient_lime  # noqa: E402
from src.explain_shap import (  # noqa: E402
    _select_malignant,
    additivity_error,
    build_tree_explainer,
    compute_shap,
    global_importance,
    shap_table,
)
from src.model import (  # noqa: E402
    evaluate_classifier,
    load_bundle,
    malignant_proba,
    save_bundle,
    train_logistic_baseline,
    train_random_forest,
)

warnings.filterwarnings("ignore")


@pytest.fixture(scope="session")
def data():
    X, y, names = load_dataset()
    return X, y, names, split_data(X, y)


@pytest.fixture(scope="session")
def model(data):
    split = data[3]
    return train_random_forest(split.X_train, split.y_train)
```

```
@pytest.fixture(scope="session")
def shap_test(data, model):
    return compute_shap(build_tree_explainer(model), data[3].X_test)


# ---------------------------------------------------------------- data
def test_dataset_loading(data):
    X, y, names, _ = data
    assert X.shape == (569, 30)
    assert len(names) == 30
    assert not X.isna().any().any()
    assert set(y.unique()) == {0, 1}


def test_malignant_benign_mapping(data):
    X, y, _, _ = data
    raw = load_breast_cancer()
    assert list(raw.target_names) == ["malignant", "benign"]  # sklearn: 0 = malignant
    assert np.array_equal(y.to_numpy() == 1, raw.target == 0)
    assert int(y.sum()) == 212 and int((y == 0).sum()) == 357
    # Clinical sanity: malignant nuclei are larger on average.
    assert X.loc[y == 1, "mean radius"].mean() > X.loc[y == 0, "mean radius"].mean()


def test_feature_ordering(data, model):
    X, _, names, split = data
    assert names == list(load_breast_cancer().feature_names)
    assert list(split.X_train.columns) == names and list(split.X_test.columns) == names
    assert list(model.feature_names_in_) == names
    # A shuffled dict is re-ordered into training order.
    row = split.X_test.iloc[0]
    shuffled = dict(reversed(list(row.items())))
    assert list(validate_patient_input(shuffled, names).columns) == names
    np.testing.assert_allclose(validate_patient_input(shuffled, names).iloc[0].to_numpy(),
    row.to_numpy())


def test_split_stratified_without_leakage(data):
    _, y, _, split = data
    assert len(split.X_train) == 455 and len(split.X_test) == 114
    assert set(split.X_train.index).isdisjoint(split.X_test.index)
    assert abs(split.y_train.mean() - split.y_test.mean()) < 0.02
    assert abs(split.y_test.mean() - y.mean()) < 0.02


# ---------------------------------------------------------------- model
def test_model_prediction_and_probability_range(data, model):
    split = data[3]
    assert list(model.classes_) == [0, 1]
    proba = model.predict_proba(split.X_test)
    assert proba.shape == (114, 2)
    assert np.all((proba >= 0) & (proba <= 1))
    np.testing.assert_allclose(proba.sum(axis=1), 1.0)
    p_mal = malignant_proba(model, split.X_test)
    np.testing.assert_allclose(p_mal, proba[:, 1])
    # predict() and the 0.5 threshold on P(malignant) agree (exact ties at 0.5 excluded)
    untied = p_mal != 0.5
    assert np.array_equal(model.predict(split.X_test)[untied], (p_mal[untied] >
    0.5).astype(int))


def test_metrics_use_malignant_as_positive(data, model):
    split = data[3]
    m = evaluate_classifier(model, split.X_test, split.y_test)
    cm = m["confusion_matrix"]
    assert cm["tp"] + cm["fn"] == int(split.y_test.sum())  # actual malignant count
    assert m["recall_sensitivity"] == pytest.approx(cm["tp"] / (cm["tp"] + cm["fn"]))
    for k in ["accuracy", "precision", "recall_sensitivity", "f1", "roc_auc"]:
        assert 0.0 <= m[k] <= 1.0
```

```
def test_bundle_roundtrip(data, model, tmp_path):
    split = data[3]
    baseline = train_logistic_baseline(split.X_train, split.y_train)
    path = tmp_path / "bundle.joblib"
    save_bundle(model, baseline, split, {"ok": True}, path=path)
    b = load_bundle(path)
    assert b["feature_names"] == split.feature_names and b["positive_class"] ==
    MALIGNANT_LABEL
    np.testing.assert_allclose(b["model"].predict_proba(split.X_test),
    model.predict_proba(split.X_test))


# ---------------------------------------------------------------- LIME
def test_lime_explanation(data, model):
    split = data[3]
    patient = split.X_test.iloc[[0]]
    res = explain_patient_lime(model, split.X_train, patient, seed=42, num_samples=1000)
    assert len(res.table) == 30 and list(res.weights.index) == split.feature_names
    assert res.model_proba == pytest.approx(malignant_proba(model, patient)[0])
    assert 0.0 <= res.local_pred <= 1.5
    # Reproducible with the same seed
    res2 = explain_patient_lime(model, split.X_train, patient, seed=42, num_samples=1000)
    pd.testing.assert_series_equal(res.weights, res2.weights)


def test_lime_explains_malignant_class(data, model):
    """For a confidently malignant patient the strongest LIME weights should point toward
    malignant."""
    split = data[3]
    p = malignant_proba(model, split.X_test)
    patient = split.X_test.iloc[[int(np.argmax(p))]]
    res = explain_patient_lime(model, split.X_train, patient, num_samples=1000)
    assert res.table.head(5)["weight"].sum() > 0


# ---------------------------------------------------------------- SHAP
def test_shap_explanation_shape(data, shap_test):
    assert shap_test.values.shape == (114, 30)
    assert list(shap_test.feature_names) == data[2]
    assert len(global_importance(shap_test)) == 30
    tbl = shap_table(shap_test[0])
    assert tbl["abs_shap"].is_monotonic_decreasing


def test_shap_additivity(data, model, shap_test):
    X_test = data[3].X_test
    assert additivity_error(model, shap_test, X_test) < 1e-6
    i = int(np.argmax(malignant_proba(model, X_test)))
    assert shap_test.values[i].sum() > 0  # high-risk patient: contributions push above the
    base rate


def test_shap_format_normalisation():
    vals = np.arange(12, dtype=float).reshape(2, 3, 2)
    v3, b3 = _select_malignant(vals, [0.6, 0.4])
    np.testing.assert_array_equal(v3, vals[:, :, 1])
    assert b3 == pytest.approx(0.4)
    vl, bl = _select_malignant([vals[:, :, 0], vals[:, :, 1]], np.array([0.6, 0.4]))
    np.testing.assert_array_equal(vl, vals[:, :, 1])
    assert bl == pytest.approx(0.4)


# ---------------------------------------------------------------- comparison
def test_comparison_identical_and_opposite():
    idx = [f"f{i}" for i in range(10)]
    w = pd.Series(np.linspace(1, -0.5, 10), index=idx)
    same = compare_explanations(w, w)
```

```
    assert same["jaccard_top_k"] == 1.0 and same["spearman_abs_all_features"] ==
    pytest.approx(1.0)
    assert same["direction_disagreements_union"] == []
    flipped = compare_explanations(-w, w)
    assert flipped["jaccard_top_k"] == 1.0
    assert set(flipped["direction_disagreements_union"]) == set(top_k(w))
    assert jaccard(["a", "b"], ["b", "c"]) == pytest.approx(1 / 3)


def test_case_selection():
    summary = pd.DataFrame({
        "patient_id": [1, 2, 3, 4, 5, 6],
        "p_malignant": [0.99, 0.01, 0.48, 0.80, 0.20, 0.70],
        "jaccard_top_k": [1.0, 1.0, 0.8, 0.1, 0.6, 0.9],
        "spearman_abs_all_features": [0.9, 0.9, 0.7, 0.2, 0.5, 0.8],
        "n_direction_disagreements": [0, 0, 0, 1, 3, 0],
    })
    cases = {c["case_type"]: c["patient_id"] for c in select_case_studies(summary)}
    assert cases == {"Clearly malignant": 1, "Clearly benign": 2, "Borderline": 3,
                     "Ranking divergence": 4, "Direction disagreement": 5}
    summary["n_direction_disagreements"] = 0
    last = select_case_studies(summary)[-1]
    assert last["found"] is False and last["patient_id"] is None


def test_lime_stability(data, model):
    split = data[3]
    stab = lime_stability(model, split.X_train, split.X_test.iloc[[0]], seeds=(0, 1),
    num_samples=500)
    assert 0.0 <= stab["mean_pairwise_jaccard_top_k"] <= 1.0


# ---------------------------------------------------------------- visualisation
def test_plot_creation(data, model, shap_test, tmp_path):
    split = data[3]
    lime_res = explain_patient_lime(model, split.X_train, split.X_test.iloc[[0]],
    num_samples=500)
    cmp = compare_explanations(lime_res.weights, pd.Series(shap_test.values[0],
    index=split.feature_names))
    m = evaluate_classifier(model, split.X_test, split.y_test)
    figs = {
        "cm": viz.plot_confusion_matrix(m["confusion_matrix"]),
        "roc": viz.plot_roc({"RF": (np.array([0, 0.1, 1]), np.array([0, 0.9, 1]), 0.95)}),
        "gauge": viz.plot_risk_gauge(0.7),
        "lime": viz.plot_lime(lime_res.table),
        "bar": viz.plot_shap_global_bar(global_importance(shap_test)),
        "bee": viz.plot_shap_beeswarm(shap_test),
        "waterfall": viz.plot_shap_waterfall(shap_test[0]),
        "force": viz.plot_shap_force_matplotlib(shap_test[0]),
        "cmp": viz.plot_lime_vs_shap(cmp["direction_table"]),
        "arch": viz.plot_architecture(),
    }
    for name, fig in figs.items():
        assert isinstance(fig, Figure), name
        viz.save(fig, tmp_path / f"{name}.png")
        assert (tmp_path / f"{name}.png").stat().st_size > 1000, name
        plt.close(fig)
    html = viz.shap_force_html(shap_test[0])
    assert "<script" in html.lower()


# ---------------------------------------------------------------- invalid input
@pytest.mark.parametrize("mutate, msg", [
    (lambda d: d.update({"mean radius": float("nan")}), "Non-finite"),
    (lambda d: d.update({"mean radius": float("inf")}), "Non-finite"),
    (lambda d: d.update({"mean radius": -1.0}), "Negative"),
    (lambda d: d.update({"mean radius": "abc"}), "numeric"),
    (lambda d: d.pop("mean radius"), "Missing"),
    (lambda d: d.update({"bogus feature": 1.0}), "Unknown"),
```

```
])
def test_invalid_numeric_input(data, mutate, msg):
    names, split = data[2], data[3]
    d = split.X_test.iloc[0].to_dict()
    mutate(d)
    with pytest.raises(ValueError, match=msg):
        validate_patient_input(d, names)


def test_invalid_input_length(data):
    with pytest.raises(ValueError, match="Expected 30"):
        validate_patient_input([1.0, 2.0], data[2])


# ---------------------------------------------------------------- application smoke test
def test_streamlit_app_runs_and_updates_on_patient_change():
    from streamlit.testing.v1 import AppTest

    at = AppTest.from_file(str(ROOT / "app.py"), default_timeout=300).run()
    assert not at.exception, [e.value for e in at.exception]
    assert [t.label for t in at.tabs] == ["Overview", "Diagnostic Prediction", "LIME
    Explanation",
                                          "SHAP Explanation", "LIME vs SHAP", "Model
    Evaluation", "Case Studies"]
    before = {m.label: m.value for m in at.metric}
    bundle = load_bundle()
    p = malignant_proba(bundle["model"], bundle["X_test"])
    current = at.selectbox(key="patient_id").value
    # switch to whichever extreme-probability patient is not currently selected
    lowest, highest = int(bundle["X_test"].index[np.argmin(p)]),
    int(bundle["X_test"].index[np.argmax(p)])
    at.selectbox(key="patient_id").set_value(lowest if current != lowest else
    highest).run()
    assert not at.exception
    after = {m.label: m.value for m in at.metric}
    assert before["Sum of SHAP values"] != after["Sum of SHAP values"] or \
        before["Local fit R²"] != after["Local fit R²"]
```

APPENDIX B: RAW DIAGNOSTIC TRACE LOGS, AUDIT TABLES & VERIFICATION SHEETS

All logs below are unedited console or file outputs from the final build of ClinXAI. Their source files are kept in `report/evidence/` and `outputs/`.

B.1 Environment Record

Python version and package versions (`report/evidence/environment.txt`)

```
Python 3.12.10
joblib==1.6.0
lime==0.2.0.1
matplotlib==3.11.2
numpy==2.5.4
pandas==3.1.0
playwright==1.64.1
pytest==9.1.1
scikit-learn==1.9.1
scipy==1.18.1
shap==0.54.1
streamlit==1.66.1
```

B.2 Training Console Log

`python train_model.py` (`report/evidence/train_model_console.txt`)

```
PS C:\Projects\ClinXAI> python train_model.py
Dataset: 569 samples x 30 features | malignant (1) = 212, benign (0) = 357
Train: 455 | Test: 114 (stratified, random_state=42)

Test-set metrics (positive class = Malignant):
                            accuracy  precision  recall_sensitivity  specificity
    f1  roc_auc  tn  fp  fn  tp
model
Random Forest                  0.9737      1.000              0.9286       1.0000
    0.9630   0.9944  72   0   3  39
Logistic Regression (baseline)    0.9649      0.975              0.9286       0.9861
    0.9512   0.9960  71   1   3  39

RF 5-fold CV ROC-AUC on training set: 0.9880 +/- 0.0073

Saved model bundle -> C:\Projects\ClinXAI\models\clinxai_rf_bundle.joblib
```

B.3 Evaluation Console Log

`python evaluate.py` (`report/evidence/evaluate_console.txt`)

```
PS C:\Projects\ClinXAI> python evaluate.py
SHAP additivity check on 114 test patients: max |error| = 4.44e-16
Running LIME on 114 test patients ...
Case 1 (Clearly malignant), patient 70: Model predicts Malignant with
    P(malignant)=1.000 (actual: Malignant; correct). LIME and SHAP share 5/5 top
    features (Jaccard 1.00; Spearman rho over |attributions| 0.90). No direction
    disagreement within the top-5 union. LIME surrogate local fit R^2=0.63; top-5
    stability across 5 seeds: mean Jaccard 0.87.
Case 2 (Clearly benign), patient 74: Model predicts Benign with P(malignant)=0.000
    (actual: Benign; correct). LIME and SHAP share 5/5 top features (Jaccard 1.00;
```

```
    Spearman rho over |attributions| 0.83). No direction disagreement within the top-5
    union. LIME surrogate local fit R^2=0.17; top-5 stability across 5 seeds: mean
    Jaccard 1.00.
Case 3 (Borderline), patient 100: Model predicts Malignant with P(malignant)=0.513
    (actual: Malignant; correct). LIME and SHAP share 2/5 top features (Jaccard 0.25;
    Spearman rho over |attributions| 0.60). Opposite direction for: area error, worst
    perimeter. LIME surrogate local fit R^2=0.05; top-5 stability across 5 seeds: mean
    Jaccard 0.48.
Case 4 (Ranking divergence), patient 86: Model predicts Benign with P(malignant)=0.297
    (actual: Malignant; INCORRECT). LIME and SHAP share 0/5 top features (Jaccard
    0.00; Spearman rho over |attributions| 0.27). Opposite direction for: worst area,
    worst perimeter, mean concave points, area error. LIME surrogate local fit
    R^2=0.03; top-5 stability across 5 seeds: mean Jaccard 0.39.
Case 5 (Direction disagreement), patient 73: Model predicts Benign with
    P(malignant)=0.160 (actual: Malignant; INCORRECT). LIME and SHAP share 1/5 top
    features (Jaccard 0.11; Spearman rho over |attributions| 0.39). Opposite direction
    for: worst area, worst perimeter, mean concavity, mean concave points. LIME
    surrogate local fit R^2=0.06; top-5 stability across 5 seeds: mean Jaccard 0.60.

Aggregate LIME vs SHAP agreement over the test set:
  n_patients: 114
  mean_top5_jaccard: 0.6230
  mean_top5_overlap: 3.6579
  mean_spearman_abs: 0.6998
  pct_patients_with_direction_disagreement: 28.9474
  mean_lime_local_r2: 0.2739

Outputs written to C:\Projects\ClinXAI\outputs
```

B.4 Automated Test Log

python -m pytest -v (report/evidence/pytest_verbose.txt)

```
============================= test session starts =============================
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.7.0 --
    C:\Projects\ClinXAI\.venv\Scripts\python.exe
rootdir: C:\Projects\ClinXAI
plugins: anyio-4.15.1
collecting ... collected 24 items
tests/test_pipeline.py::test_dataset_loading PASSED                      [  4%]
tests/test_pipeline.py::test_malignant_benign_mapping PASSED             [  8%]
tests/test_pipeline.py::test_feature_ordering PASSED                     [ 12%]
tests/test_pipeline.py::test_split_stratified_without_leakage PASSED     [ 16%]
tests/test_pipeline.py::test_model_prediction_and_probability_range PASSED [ 20%]
tests/test_pipeline.py::test_metrics_use_malignant_as_positive PASSED    [ 25%]
tests/test_pipeline.py::test_bundle_roundtrip PASSED                     [ 29%]
tests/test_pipeline.py::test_lime_explanation PASSED                     [ 33%]
tests/test_pipeline.py::test_lime_explains_malignant_class PASSED        [ 37%]
tests/test_pipeline.py::test_shap_explanation_shape PASSED               [ 41%]
tests/test_pipeline.py::test_shap_additivity PASSED                      [ 45%]
tests/test_pipeline.py::test_shap_format_normalisation PASSED            [ 50%]
tests/test_pipeline.py::test_comparison_identical_and_opposite PASSED    [ 54%]
tests/test_pipeline.py::test_case_selection PASSED                       [ 58%]
tests/test_pipeline.py::test_lime_stability PASSED                       [ 62%]
tests/test_pipeline.py::test_plot_creation PASSED                        [ 66%]
tests/test_pipeline.py::test_invalid_numeric_input[<lambda>-Non-finite0] PASSED [ 70%]
tests/test_pipeline.py::test_invalid_numeric_input[<lambda>-Non-finite1] PASSED [ 75%]
tests/test_pipeline.py::test_invalid_numeric_input[<lambda>-Negative] PASSED [ 79%]
tests/test_pipeline.py::test_invalid_numeric_input[<lambda>-numeric] PASSED [ 83%]
tests/test_pipeline.py::test_invalid_numeric_input[<lambda>-Missing] PASSED [ 87%]
tests/test_pipeline.py::test_invalid_numeric_input[<lambda>-Unknown] PASSED [ 91%]
tests/test_pipeline.py::test_invalid_input_length PASSED                 [ 95%]
tests/test_pipeline.py::test_streamlit_app_runs_and_updates_on_patient_change PASSED
    [100%]
```

```
============================== 24 passed in 17.82s ==============================
```

B.5 Independent Output Validation (35 checks)

Validation of saved outputs (`report/evidence/output_validation.txt`)

```
PASS All 26 expected PNG figures exist and decode
PASS 5 SHAP force HTML files contain embedded JS
PASS No stray/unexpected PNGs in outputs/figures
PASS Confusion matrix in metrics.json matches recomputation | 72,0,3,39
PASS Confusion matrix sums to test size 114 and actual malignant = 42
PASS All RF metrics in metrics.json match recomputation | {'accuracy': 0.9737,
    'precision': 1.0, 'recall_sensitivity': 0.9286, 'f1': 0.963, 'roc_auc': 0.9944,
    'specificity': np.float64(1.0)}
PASS Metrics are internally consistent with the confusion matrix
PASS metrics.csv agrees with metrics.json
PASS Baseline metrics match recomputation
PASS dataset_overview.csv has 30 features
PASS Saved labels: malignant=1 matches sklearn raw target==0
PASS Model classes_ == [0,1] and feature order == sklearn order
PASS SHAP values used = class-1 (malignant) slice
PASS SHAP class-1 values = -(class-0 values) (binary consistency)
PASS SHAP base + sum == P(malignant) for all 114 test patients | max err 4.44e-16
PASS SHAP base value equals mean training P(malignant)-weighted base (0.3747) | 0.3747
PASS Sign sanity: highest-risk patient has positive total SHAP, lowest-risk negative
PASS LIME sign sanity: top-5 weights sum >0 for highest-risk, <0 for lowest-risk
    patient | +0.513 / -0.301
PASS LIME model_proba equals RF P(malignant)
PASS test_set_lime_vs_shap.csv covers all 114 test patients
PASS p_malignant column matches model
PASS 5 case studies, all found
PASS Case 1 = max P(malignant)
PASS Case 2 = min P(malignant)
PASS Case 3 = closest to 0.5
PASS Case 4 (#86) has the lowest top-5 Jaccard among remaining patients
PASS Case 5 (#73) has the most direction disagreements among remaining patients
PASS Case 1 (Clearly malignant, #70) re-runs identically (LIME seed 42, SHAP, labels)
    | P=1.000, actual=Malignant, pred=Malignant, overlap=5/5, rho=0.90, dir=[],
    R2=0.63
PASS Case 2 (Clearly benign, #74) re-runs identically (LIME seed 42, SHAP, labels) |
    P=0.000, actual=Benign, pred=Benign, overlap=5/5, rho=0.83, dir=[], R2=0.17
PASS Case 3 (Borderline, #100) re-runs identically (LIME seed 42, SHAP, labels) |
    P=0.513, actual=Malignant, pred=Malignant, overlap=2/5, rho=0.60, dir=['area
    error', 'worst perimeter'], R2=0.05
PASS Case 4 (Ranking divergence, #86) re-runs identically (LIME seed 42, SHAP, labels)
    | P=0.297, actual=Malignant, pred=Benign, overlap=0/5, rho=0.27, dir=['worst
    area', 'worst perimeter', 'mean concave points', 'area error'], R2=0.03
PASS Case 5 (Direction disagreement, #73) re-runs identically (LIME seed 42, SHAP,
    labels) | P=0.160, actual=Malignant, pred=Benign, overlap=1/5, rho=0.39,
    dir=['worst area', 'worst perimeter', 'mean concavity', 'mean concave points'],
    R2=0.06
PASS Patient #86 is a false negative (actual malignant, predicted benign)
PASS Patient #73 is a false negative (actual malignant, predicted benign)
PASS Aggregate stats match per-patient CSV

35/35 checks passed
```

B.6 Real-Browser Verification Sheet (34 checks)

Table B.1: Real-browser click-through results (Playwright + Microsoft Edge)

No.	Check	Result	Observed detail
1	Seven tabs rendered in the browser	PASS	['Overview', 'Diagnostic Prediction', 'LIME Explanation', 'SHAP Explanation', 'LIME vs SHA
2	Title banner shown	PASS	ClinXAI — Transparent Clinical Diagnostics
3	Educational disclaimer visible	PASS	
4	[Overview] images render (>= 1), none broken, no exceptions/errors	PASS	1 images, broken=[], exceptions=0, errors=[]
5	[Diagnostic Prediction] images render (>= 1), none broken, no exceptions/errors	PASS	1 images, broken=[], exceptions=0, errors=[]
6	[LIME Explanation] images render (>= 1), none broken, no exceptions/errors	PASS	1 images, broken=[], exceptions=0, errors=[]
7	[SHAP Explanation] images render (>= 4), none broken, no exceptions/errors	PASS	4 images, broken=[], exceptions=0, errors=[]
8	[LIME vs SHAP] images render (>= 1), none broken, no exceptions/errors	PASS	1 images, broken=[], exceptions=0, errors=[]
9	[Model Evaluation] images render (>= 2), none broken, no exceptions/errors	PASS	2 images, broken=[], exceptions=0, errors=[]
10	[Case Studies] images render (>= 1), none broken, no exceptions/errors	PASS	6 images, broken=[], exceptions=0, errors=[]
11	Default patient #70: P(malignant) = 1.000 and label Malignant	PASS	1.000
12	SHAP base value shown as 0.375	PASS	0.375
13	Patient #70 LIME vs SHAP overlap 5/5	PASS	5 / 5
14	Patient switch -> #86 header, P=0.297, Benign output, actual Malignant	PASS	0.297
15	LIME tab updated to #86 (R^2 0.03)	PASS	0.03
16	SHAP local values changed after switching patient	PASS	+0.625 -> -0.078
17	Patient #86 comparison: 0/5 overlap, 4 direction disagreements	PASS	0 / 5, 4
18	Comparison figure re-rendered for #86 without errors	PASS	
19	LIME stability for #86 computed in browser (expected 0.39 from case study)	PASS	0.39
20	Changing LIME seed re-runs LIME (R^2 changes or stays plausible)	PASS	seed42 R^2=0.03, seed7 R^2=0.02
21	Seed restored to 42 reproduces R^2 0.03	PASS	0.03
22	Patient #100 borderline P=0.513	PASS	0.513
23	What-if worst area 906.5 -> 500 flips to Benign with P=0.363	PASS	0.363
24	SHAP tab reflects adjusted values	PASS	0.363
25	Interactive SHAP force plot renders SVG inside iframe	PASS	1 frames, 1 svg
26	Case Studies lists 'Case 1: Clearly malignant — Patient #70'	PASS	
27	Case Studies lists 'Case 2: Clearly benign — Patient #74'	PASS	
28	Case Studies lists 'Case 3: Borderline — Patient #100'	PASS	
29	Case Studies lists 'Case 4: Ranking divergence — Patient #86'	PASS	

No.	Check	Result	Observed detail
30	Case Studies lists 'Case 5: Direction disagreement — Patient #73'	PASS	
31	'Open patient #73' button switches the whole app to #73 (P=0.160)	PASS	sidebar='Patient #73 — actual Malignant, P=0.16 ★ Direction disagreement', header='Patien
32	Patient #73 comparison: 1/5 overlap, 4 direction disagreements	PASS	
33	Model Evaluation metrics shown (0.9737 / 1.0000 / 0.9286 / 0.9630 / 0.9944)	PASS	
34	No browser console/page errors	PASS	

B.7 SHAP Additivity Record

outputs/metrics/shap_additivity.json


```
{
  "max_abs_error": 4.440892098500626e-16,
  "tolerance": 1e-06,
  "n_patients": 114,
  "base_value_p_malignant": 0.3746739926739926,
  "explainer": "shap.TreeExplainer(feature_perturbation='tree_path_dependent')",
  "output": "P(malignant) — probability units"
}
```


B.8 Model Metrics Record

outputs/metrics/metrics.json


```
{
  "dataset": {
    "name": "Breast Cancer Wisconsin (Diagnostic) —
    sklearn.datasets.load_breast_cancer",
    "n_samples": 569,
    "n_features": 30,
    "label_encoding": {
      "malignant": 1,
      "benign": 0
    },
    "n_malignant": 212,
    "n_benign": 357,
    "n_train": 455,
    "n_test": 114,
    "test_malignant": 42,
    "test_benign": 72,
    "split": "stratified 80/20, random_state=42"
  },
  "random_forest": {
    "params": {
      "n_estimators": 300,
      "max_depth": null,
      "min_samples_leaf": 1,
      "class_weight": null,
      "random_state": 42,
      "n_jobs": -1
    },
    "positive_class": "Malignant (label 1)",
    "decision_threshold": 0.5,
    "accuracy": 0.9736842105263158,
    "precision": 1.0,
    "recall_sensitivity": 0.9285714285714286,
    "specificity": 1.0,
```

```
    "f1": 0.9629629629629629,
    "roc_auc": 0.9943783068783069,
    "confusion_matrix": {
      "tn": 72,
      "fp": 0,
      "fn": 3,
      "tp": 39
    },
    "cv_roc_auc_mean": 0.9879772961816305,
    "cv_roc_auc_std": 0.007281943183619433
  },
  "logistic_regression_baseline": {
    "positive_class": "Malignant (label 1)",
    "decision_threshold": 0.5,
    "accuracy": 0.9649122807017544,
    "precision": 0.975,
    "recall_sensitivity": 0.9285714285714286,
    "specificity": 0.9861111111111112,
    "f1": 0.9512195121951219,
    "roc_auc": 0.996031746031746,
    "confusion_matrix": {
      "tn": 71,
      "fp": 1,
      "fn": 3,
      "tp": 39
    }
  }
}
```


B.9 Case-Study Audit Table

Table B.2: Contents of outputs/case_studies/case_studies.csv (numeric columns; direction-disagreement lists and observations are given in Section 5.8)

#	Case	Patient	Actual / predicted	P(mal.)	Overlap	Jaccard	ρ	LIME R^2	Stability
1	Clearly malignant	#70	Malignant / Malignant	1.0	5	1.0	0.8981	0.626	0.8667
2	Clearly benign	#74	Benign / Benign	0.0	5	1.0	0.8265	0.1738	1.0
3	Borderline	#100	Malignant / Malignant	0.5133	2	0.25	0.6018	0.0511	0.4798
4	Ranking divergence	#86	Malignant / Benign	0.2967	0	0.0	0.269	0.0274	0.3857
5	Direction disagreement	#73	Malignant / Benign	0.16	1	0.1111	0.3927	0.0606	0.6048

B.10 Descriptive Analysis Record

report/figures/report_descriptive_analysis.json (computed from test_set_lime_vs_shap.csv by report/make_report_figures.py)


```
{
  "spearman_confidence_vs_jaccard": 0.6012900768443163,
  "spearman_confidence_vs_lime_r2": 0.487167832728111,
  "n_confident_ge_0.4": 90,
  "mean_jaccard_confident": 0.6922155555555556,
  "mean_r2_confident": 0.30308999999999997,
  "n_uncertain_lt_0.4": 24,
  "mean_jaccard_uncertain": 0.3634375,
  "mean_r2_uncertain": 0.16422083333333334,
```

```
  "overlap_count_distribution": {
    "0": 1,
    "1": 2,
    "2": 19,
    "3": 18,
    "4": 47,
    "5": 27
  },
  "false_negatives": [
    {
      "patient_id": 73,
      "p_malignant": 0.16,
      "jaccard_top_k": 0.1111,
      "spearman_abs_all_features": 0.3927,
      "lime_local_r2": 0.0606,
      "n_direction_disagreements": 4
    },
    {
      "patient_id": 86,
      "p_malignant": 0.2967,
      "jaccard_top_k": 0.0,
      "spearman_abs_all_features": 0.269,
      "lime_local_r2": 0.0274,
      "n_direction_disagreements": 4
    },
    {
      "patient_id": 385,
      "p_malignant": 0.19,
      "jaccard_top_k": 0.25,
      "spearman_abs_all_features": 0.2427,
      "lime_local_r2": 0.0484,
      "n_direction_disagreements": 3
    }
  ]
}
```